

Automated Software Testing

2023-2024

Indice

1	TESTING	2
2	JUnit	8
2.1	Hamcrest	12
2.2	AssertJ	12
3	TEST-DRIVEN DEVELOPMENT	14
4	CODE COVERAGE	20
5	MUTUATION TESTING	22
6	MAVEN	24
6.1	Installazione	25
6.2	Gestione delle dipendenze	30
6.3	Automazione della build	35
6.4	Packaging con Maven	40
6.5	Plug-In	43
6.6	Profili Maven	51
6.7	Maven e JUnit 5	52
7	MOCKING	54
8	VERSION CONTROL SYSTEM	61
8.1	Git	63
8.2	GitHub Flow	73
8.3	Comandi Git	76
8.3.1	Configurazione	76
8.3.2	Inizializzazione di un Progetto con Git	76
8.3.3	Commit	76
8.3.4	Principi fondamentali di Git	77
8.3.5	Branch	77
8.3.6	Merge	77
8.3.7	Rebasing	78
8.3.8	Annullare	78
8.3.9	Esaminare il repository	78
8.3.10	Stash	79
8.3.11	Sincronizzazione	79

<i>INDICE</i>	1
9 CONTINUOUS INTEGRATION	81
9.1 Coveralls	86
10 DOCKER	88
10.1 Dockerfile	93
10.2 Docker Compose	94
10.3 Docker Hub	95
10.4 Docker su GitHub Actions	96
11 INTEGRATION TEST	97
12 UI TEST	104
12.1 Multi-threading	109
13 END-TO-END TEST	111
14 BEHAVIORAL-DRIVEN DEVELOPMENT	114
15 CODE QUALITY	117

Capitolo 1

TESTING

Gli **Automated Test** (oppure **test automatizzati**) forniscono un grado di fiducia nella correttezza del software, sebbene non possano garantire la sua precisione al 100%. Possono essere identificate diverse classificazioni di test. Tuttavia, è importante notare che non esiste un consenso universale sulla definizione dei test.

Un **test** è un processo *ripetibile* e, nella maggior parte dei casi, *deterministico*. Questo significa che, eseguendo il test più volte, il risultato deve essere sempre identico. Tuttavia, in situazioni di programmazione concorrente, dove l'esecuzione avviene in modo non deterministico, anche i test possono essere non deterministici.

Il test ha lo scopo di *verificare il corretto* comportamento dell'entità sottoposta a verifica all'interno di *uno specifico contesto*, situazione o scenario. A seconda del tipo di test svolto, l'entità che viene testata può variare. Tuttavia, in generale, ogni specifico test mira a verificare la correttezza di un certo tipo di entità in un contesto molto specifico. Inoltre, un test viene condotto con un *input* specifico, asserendo un *output* predeterminato. Questo processo consente di verificare che l'output sia conforme alle aspettative. Per garantire che l'output sia corretto nel contesto del test, eseguiamo delle **asserzioni**.

Gli Automated Test *rappresentano una forma di documentazione eseguibile*. In particolare, quando i test sono ben scritti, offrono una chiara comprensione delle funzionalità del codice.

A differenza della *documentazione tradizionale*, che può diventare obsoleta e perdere sincronia con il codice quando quest'ultimo subisce modifiche significative, i test rimangono sempre attuali. Infatti, in caso di cambiamenti radicali nel codice, i test rivelano tali modifiche attraverso eventuali fallimenti. Pertanto, è sufficiente aggiornare i test per riflettere le nuove implementazioni, garantendo che la documentazione rimanga sempre allineata e mai obsoleta.

Quando i test sono *redatti in modo accurato*, costituiscono una solida **Safety Net** (o *rete di sicurezza*). *Questa rete ci offre la possibilità di eseguire refactoring in modo rapido e sicuro*. Infatti, se si verifica una rottura nel codice durante il processo di refattorizzazione, allora i test rileveranno il fallimento, semplificando l'individuazione del problema.

N.B: con **refactoring** si intende il processo di modifica del codice senza alterarne i comportamenti. Grazie ai test, possiamo essere certi che il comportamento del codice rimanga invariato durante il processo di refactoring. Infatti, se i test continuano ad essere validi, allora possiamo essere sicuri di non aver introdotto alcun problema nel codice.

Gli Automated Test non garantiscono l'assenza di **bug** nel codice, *ma facilitano l'individuazione e la risoluzione di eventuali difetti*. In particolare, il processo di individua-

zione e correzione dei bug segue una metodologia ben definita:

1. Scrivere un test che simuli il bug, causando il fallimento del test.
2. Correggere il codice in modo che il test venga superato con successo, indicando che il bug è stato risolto.
3. Verificare che tutti gli altri test esistenti abbiano esito positivo, garantendo così che la correzione non abbia introdotto nuovi bug.

I test rappresentano pertanto una *condizione necessaria* per la correttezza del codice, sebbene non siano sufficienti. Infatti, se un test fallisce allora è certo che ci sia un problema nel codice. Tuttavia, se il test non fallisce allora non possiamo affermare con certezza che il codice sia corretto.

Tuttavia, per garantire questa proprietà, **i test devono essere ben scritti**, cioè devono essere formulati e progettati in modo accurato, per testare in modo completo ed efficace il comportamento del programma.

La **Test Pyramid** (o **piramide dei test**) è una rappresentazione visiva per descrivere i diversi livelli di test:

- **Unit Test**: si concentrano sulla verifica delle **singole unità di un programma**, come classi o metodi. L'obiettivo degli Unit Test è quello di testare una specifica unità del programma **in modo isolato**, cioè in modo indipendente dagli altri componenti. Pertanto, i componenti esterni all'unità da testare vengono sostituiti da false implementazioni, note come **test double** o **mock**.
- **Integration Test**: **combinano** le singole unità per verificare che continuino a **funzionare come previsto**, considerando che sono state testate separatamente con gli Unit Test. In particolare, gli Integration Test mirano a testare la collaborazione tra almeno due componenti reali.
- **End-to-End Test** (o **System Test**): testano **l'intera applicazione attraverso la sua interfaccia utente**, simulando l'interazione **dell'utente con l'applicazione**. In particolare, gli End-to-End Test verificano che **tutti i componenti dell'applicazione interagiscano correttamente**.

N.B.: rappresentano una categoria speciale di Integration Test in cui tutti i componenti del sistema vengono integrati insieme.

La forma della Pyramid Test suggerisce una distribuzione gerarchica dei test che riflette la dimensione e la velocità. Alla base della piramide, si dovrebbero avere numerosi Unit Test, ciascuno di piccole dimensioni e di rapida esecuzione, misurata in millisecondi. Procedendo verso l'alto, si incontrano alcuni Integration Test di dimensioni più ampie, che potrebbero risultare più lenti rispetto agli Unit Test. Infine, al vertice della piramide, sono presenti pochi End-to-End Test, che operano ad un livello molto alto e possono richiedere più tempo per l'esecuzione.

La scrittura e la manutenzione dei test richiedono diverse considerazioni a seconda del tipo di test. Scrivere Unit Test dovrebbe risultare relativamente semplice, essendo focalizzati su singole unità del codice. Gli Integration Test, pur potendo richiedere un po' più di impegno, si concentrano sulla connessione di diversi componenti e sulla verifica della loro interazione. Gli End-to-End Test, essendo orientati all'interazione con l'interfaccia utente esterna dell'applicazione, possono essere i più complessi da scrivere.

Infatti, questo tipo di test mira a simulare l'esperienza dell'utente finale, ignorando i dettagli interni dell'applicazione.

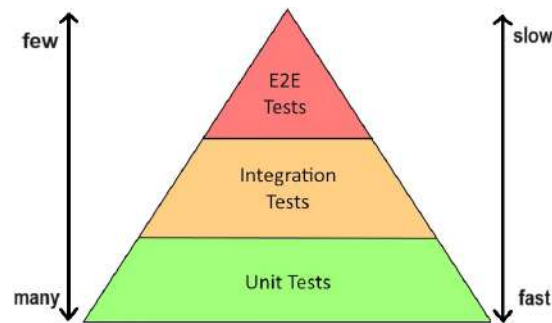


Figura 1.1: Rappresentazione della Pyramid Test. Questa struttura gerarchica fornisce un approccio completo alla verifica del software, partendo dai test più specifici e dettagliati (Unit Test) fino ai test più generici ed orientati all'utente finale (End-to-End Test). La piramide evidenzia l'importanza di garantire una solida base, effettuando test a livelli più bassi prima di passare ai test a livelli superiori. Inoltre, evidenzia che i test alla base della piramide saranno numerosi ma veloci, mentre i test ai livelli superiori saranno meno numerosi ma più lenti.

Il **System Under Test (SUT)**, noto anche come Software Under Test o Subject Under Test, rappresenta il componente sottoposto all'analisi durante un processo di test. La sua natura varia a seconda del tipo di test eseguito:

- Negli Unit Test Java, il SUT si configura come una singola classe. In questo contesto, vengono effettuate chiamate ai metodi su un'istanza specifica di tale classe.
- Negli Integration Test, il SUT può essere configurato come una singola istanza che interagisce con altri componenti esterni oppure come diverse istanze, corrispondenti a diverse classi.
- Negli End-to-End Test, il SUT si estende a tutta l'applicazione. In questo contesto, interagiamo con l'applicazione attraverso l'interfaccia utente (UI), simulando l'esperienza dell'utente finale.

Gli Unit Test si basano sulla comprensione dettagliata della logica interna di un componente, pertanto possono essere considerati come test **white-box**.

Al contrario, gli End-to-End Test ignorano completamente i dettagli di basso livello, concentrandosi soltanto sull'interazione dell'utente finale con l'applicazione. Pertanto, possono essere classificati come test **black-box**.

Gli Integration Test rappresentano un punto intermedio. Pur non essendo veramente test black-box, poiché possono coinvolgere alcuni dettagli interni a basso livello, non raggiungono la conoscenza interna degli white-box. Pertanto, pur dipendendo da alcuni aspetti interni, gli Integration Test non rivelano ogni dettaglio del funzionamento interno del componente testato.

Gli Unit Test costituiscono la base della piramide, poiché si concentrano sulla verifica del funzionamento delle singole unità del codice. Pertanto, può risultare poco significativo scrivere Integration Test o End-to-End Test per componenti che non hanno ancora superato gli Unit Test.

Invece, gli Integration Test e gli End-to-End Test dovrebbero concentrarsi esclusivamente sulla verifica del comportamento di componenti già testate in modo isolato attraverso gli Unit Test. Pertanto, ci aspettiamo che tali test siano positivi fin dall'inizio.

Il **Test-Driven Development (TDD)** rappresenta una metodologia di sviluppo basata sulla scrittura dei test prima dell'implementazione effettiva del codice. Pertanto, in questa metodologia gli Unit Test vengono progettati per fallire inizialmente. In questo contesto, i test svolgono diversi ruoli fondamentali nel processo di sviluppo:

- Garantiscono la qualità del codice.
- Guidano la progettazione e lo sviluppo del codice.
- Agiscono come specifiche, delineando le aspettative e le funzionalità desiderate.
- Forniscono una documentazione sul comportamento atteso del codice.
- Contribuiscono a scrivere codice che sia modulare, pulito e facilmente manutenibile.

Gli **User-Interface Test** (o **UI Test**) sono progettati per testare l'interfaccia utente di un'applicazione, che può presentarsi come un'applicazione da riga di comando, un'applicazione Desktop con interfaccia grafica (GUI) oppure un'applicazione web (ad esempio, possono testare il corretto funzionamento dell'UI quando l'utente inserisce dei dati in un campo di testo e preme un pulsante, comportando l'aggiunta di un elemento in una lista).

La scrittura di UI Test può essere piuttosto complessa, a causa della natura event-driven delle GUI (Graphical User Interface). Tuttavia, l'impiego di framework di testing dedicati semplifica notevolmente questa operazione (ad esempio, AssertJ Swing offre un'API per simulare gli eventi dell'utente ed accedere ai controlli della GUI, mentre Selenium fornisce un'API per testare l'interfaccia web attraverso un vero browser). **N.B.**: pur essendo intrinsecamente più lenti a causa dell'overhead delle GUI, questi test sono fondamentali per garantire il corretto funzionamento dell'interfaccia utente (UI) dell'applicazione.

N.B.: gli UI Test e gli End-to-End Test (E2E Test) sono concetti ortogonali (cioè, sono concetti distinti). Mentre gli E2E Test spesso corrispondono agli UI Test, poiché esaminano l'intera applicazione come un'entità black-box attraverso la sua UI, l'inverso non è necessariamente vero. Infatti, è possibile testare l'UI anche attraverso Unit Test, isolandola dalle sue dipendenze.

Quando eseguire i test è una decisione fondamentale nel processo di sviluppo del software:

- Gli Unit Test sono progettati per essere veloci. Pertanto dovrebbero essere eseguiti con frequenza dagli sviluppatori, tipicamente dopo ogni modifica o refactoring del codice.
- Gli Integration Test e gli E2E Test sono tipicamente più lenti. Pertanto, possono essere eseguiti con meno frequenza dagli sviluppatori. Solitamente, vengono eseguiti in un server di *Continuous Integration* dedicato.

Gli Unit Test rappresentano un metodo di verifica a basso livello, orientato verso i dettagli interni del *System Under Test* (SUT). Questi test sono concepiti per essere completamente esaustivi.

Sebbene testare un metodo su ogni possibile input risulti impraticabile, gli Unit Test dovrebbero essere in grado di testare almeno tutti i possibili *percorsi* (o *path*) della logica di tale metodo.



ES:

Se un metodo è progettato per generare un'eccezione in determinate circostanze, allora è essenziale implementare Unit Test che riproducano queste condizioni specifiche e verifichino con certezza che l'eccezione venga effettivamente sollevata.

Nel caso in cui il metodo contenga una struttura condizionale (ovvero, una struttura `if-then-else`), allora è necessario testare entrambi i rami della condizione per assicurare una copertura completa.

Le condizioni come `>`, `<`, `>=` e `<=` dovrebbero essere testate anche nei casi limite. Ad esempio, una condizione del tipo `i > 0`, dovrebbe essere testata quando `i` è minore di 0, quando `i` è maggiore di 0 e quando `i` è uguale a 0.



ES: Consideriamo il testing sull'utilizzo di una collezione. Allora, per verificare tutti i possibili percorsi, dobbiamo scrivere tre Unit Test distinti:

- Test sulla collezione vuota.
- Test sulla collezione contenente un solo elemento.
- Test sulla collezione contenente più elementi (tipicamente si considerano due elementi).

In questo contesto, è preferibile iniziare il test dal cosiddetto **happy path**, ovvero dal caso più semplice (cioè, il test sulla collezione vuota).

A differenza degli Unit Test, gli Integration Test e gli E2E Test non mirano ad esplorare tutti i possibili percorsi, ma si concentrano esclusivamente sui casi rilevanti, in base alla natura specifica dell'applicazione. Infatti, è inutile scrivere Integration Test per casi eccezionali e sbagliati, poiché tali situazioni sono già state verificate dagli Unit Test.

Notiamo che apportare modifiche al comportamento di singoli componenti rende improbabile il fallimento degli Integration Test, a meno che non avvengano modifiche significative all'interfaccia esterna (come il cambio di nome di una classe). Al contrario, anche la più piccola modifica in un componente dovrebbe provocare il fallimento di almeno uno Unit Test. L'assenza di tale fallimento indica un potenziale problema nella progettazione degli Unit Test, suggerendo che potrebbero non testare in modo completo e accurato il codice. Per condurre questa verifica, vengono impiegati framework di **Mutation Testing**.

Essendo una forma di documentazione eseguibile, i test devono presentarsi in una forma chiara e di facile comprensione (cioè, è necessario rendere chiaro cosa sta verificando un test e il contesto in cui viene eseguito). Pertanto, l'uso eccessivo di astrazioni nei test potrebbero rappresentare un anti-pattern.

La *duplicazione del codice*, se contribuisce a migliorare la chiarezza e la leggibilità dei test, è ammissibile e persino consigliabile.





Indipendentemente dal tipo di test, è fondamentale scrivere test che replicano e verificano con precisione un comportamento specifico. Negli Automated Test, i falsi positivi costituiscono un problema critico, spesso anche più grave dei falsi negativi. Questo perché i falsi positivi (cioè, test che dovrebbero essere negativi ma vengono erroneamente interpretati come positivi), forniscono una falsa sicurezza sulla correttezza del codice. Per valutare l'efficacia degli Automated Test, si ricorre a strumenti come la **Code Coverage** e il **Mutation Testing**.

I test, dovrebbero concentrarsi esclusivamente sulla verifica di porzioni di codice che richiedono una logica specifica. Pertanto, dobbiamo evitare di scrivere test per i metodi *getter* e *setter*, a meno che non implichi condizioni o comportamenti particolari.

Lo Static Type Checking e gli Automated Test sono due aspetti ortogonali che mirano a garantire la correttezza del codice a livelli diversi. Infatti, il controllo statico dei tipi si focalizza sul comportamento statico del codice, mentre i test automatici si occupano del comportamento a runtime. Entrambi sono elementi essenziali per assicurare la robustezza e la precisione del software, ma è fondamentale sottolineare che né il controllo statico dei tipi né i test automatici da soli sono sufficienti a garantire la correttezza del codice.

Capitolo 2

JUnit

Distinguiamo il processo di testing in quattro fasi ordinate:

1. **Setup (preparazione)**: in questa fase, si crea l'ambiente di test. Negli Unit Test, questa fase comprende la creazione del SUT (ad esempio, si genera un'istanza della classe da testare). In particolare, tale fase stabilisce lo stato iniziale del SUT prima dell'esecuzione di ogni test.

Lo stato creato in questa fase, che comprende l'istanza della SUT ed eventuali altri oggetti correlati necessari durante i test, è denominato **Test Fixture**.

Una Test Fixture rappresenta tutto ciò che è necessario per testare il SUT. Essa costituisce uno stato predefinito di un insieme di oggetti, funzionando come base per l'esecuzione dei test. La Test Fixture ha il compito di garantire che l'ambiente in cui vengono eseguiti i test sia noto e stabile, assicurando la ripetibilità dei risultati dei test.

La Test Fixture è costituita principalmente dall'istanza del SUT. Tuttavia, quando il SUT richiede la collaborazione di oggetti esterni, la Test Fixture include anche le istanze di questi collaboratori.

N.B.: negli Unit Test, le istanze dei collaboratori nella Test Fixture vengono sostituite da false implementazioni, dette **mock**. Invece, negli Integration Test, i collaboratori devono essere reali (ad esempio, il caricamento di un database con un insieme specifico e noto di dati).

2. **Exercise (esercizio)**: durante questa fase, si interagisce con il SUT. Questo può comprendere la chiamata di un metodo per ottenere un risultato specifico oppure la generazione di *side effect* in altre istanze.
3. **Verify (verifica)**: consiste nel confrontare il risultato ottenuto durante la precedente fase con il comportamento atteso. In particolare, attraverso l'utilizzo delle asserzioni sullo stato modificato del Test Fixture, si verifica che il valore restituito dal metodo invocato ed i side effect su di esso corrispondano alle aspettative.
4. **Teardown (ripristino)**: questa fase ha l'obiettivo di ripulire l'ambiente del test, riportandolo alle condizioni iniziali (ad esempio, se sono stati creati file durante il test, questi dovrebbero essere rimossi in questa fase).

N.B.: le prime tre fasi sono note anche come “arrange, act, assert” oppure “given, when, then” (nella metodologia Behavioral-Driven Development).

JUnit è un framework di Unit Test per il moderno Java. In JUnit, un **test case** è rappresentato da una classe Java standard (non è obbligatorio ereditare da una classe base esistente). Ciascun test è identificato da un metodo pubblico annotato con **@Test**.



Ogni metodo di test dovrebbe verificare un singolo scenario, interagendo con una parte specifica del SUT (ad esempio, invocando un metodo del SUT). La fase di verifica avviene attraverso l'analisi dell'output o del risultato, confrontandolo con quanto atteso, mediante l'uso di **metodi assert**.



La struttura di una classe di test in JUnit segue un modello ben definito:

- **Setup a livello di classe:** questa fase è gestita da un metodo statico annotato con `@BeforeClass` per configurare l'ambiente prima dell'esecuzione dei test. Questo metodo viene eseguito una sola volta prima di iniziare l'esecuzione dei test. Qui è possibile configurare risorse condivise o configurazioni globali.
- **Setup a livello di metodo:** questa fase è gestita da un metodo non statico annotato con `@Before` per configurare l'ambiente prima dell'esecuzione di ciascun test. Questo metodo consente di configurare delle condizioni specifiche per il test in questione.
- **Exercise e Verify:** l'esecuzione del test avviene attraverso i metodi annotati con `@Test`. Durante questa fase, il SUT viene esercitato ed i risultati vengono verificati utilizzando asserzioni. Questo è il cuore del processo di testing.
- **Teardown:** questa fase è gestita dai metodi annotati con `@AfterClass` (statico) e `@After`. Questi metodi vengono eseguiti rispettivamente dopo la conclusione di tutti i test della classe e dopo ciascun metodo di test. Qui avviene il rilascio delle risorse ed il ripristino dello stato iniziale.

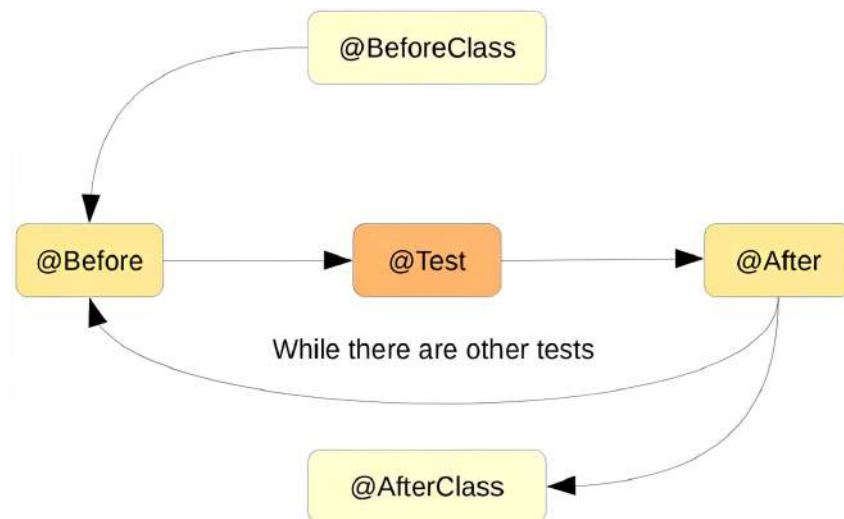


Figura 2.1: Ciclo di vita di JUnit.

I metodi di asserzione in JUnit, definiti nella classe `org.junit.Assert`, sono statici e consentono di verificare le aspettative durante l'esecuzione dei test. Analizziamo alcuni dei principali metodi di asserzione:

- `assertEquals(expected, actual)`: questo metodo confronta il valore attuale (`actual`) con il valore atteso (`expected`) utilizzando il metodo `equals`. L'asserzione fallisce se i due valori sono diversi.

- `assertSame(expected, actual)`: questo metodo verifica se il riferimento attuale (`actual`) è lo stesso del riferimento atteso (`expected`), utilizzando l'operatore `==`. L'asserzione fallisce se i riferimenti non sono gli stessi.
- `assertTrue(expression)` e `assertFalse(expression)`: questi metodi verificano rispettivamente se l'espressione booleana è `true` o `false`. L'asserzione fallisce se l'espressione non soddisfa le aspettative.
- `assertNull(expression)` e `assertNotNull(expression)`: questi metodi verificano rispettivamente se l'espressione è `null` oppure no. L'asserzione fallisce se l'espressione non soddisfa le aspettative.

N.B.: nel caso in cui un'asserzione fallisca, il test termina con uno stato di fallimento, fornendo informazioni dettagliate sul motivo del fallimento. Se durante il test viene lanciata un'eccezione, il test termina con un errore. In caso contrario, il test ha successo.

N.B.: tutti i metodi di asserzione prevedono una versione *overload*, che accetta una stringa come primo argomento. Questa stringa rappresenta un messaggio personalizzato che sarà visualizzato in caso di fallimento del test, offrendo ulteriori dettagli per la diagnosi del problema.

JUnit adotta un approccio "all or nothing", ovvero se anche una sola asserzione fallisce all'interno di un metodo di test, allora il metodo stesso viene contrassegnato come fallito, l'intero caso di test in cui il metodo è incluso è considerato non superato e l'intera classe di test a cui il caso di test appartiene è segnalata come fallita.

Dobbiamo garantire che ogni singolo metodo di test possa essere eseguito in modo isolato. Perciò, dobbiamo evitare di scrivere metodi di test che dipendano dai side effect di altri test, soprattutto perché JUnit non specifica l'ordine di esecuzione dei test (cioè, i test vengono eseguiti in un ordine non deterministico e non è possibile garantire che JUnit utilizzi la stessa istanza per eseguire i vari metodi di test in esecuzioni successive). Mantenere l'indipendenza tra i test assicura che ognuno possa essere eseguito in modo indipendente dagli altri, contribuendo ad un ambiente di testing affidabile e ripetibile.



ES:

Consideriamo la seguente classe di test:

```
1 public class JunitWrongExample {
2     private int count = 1;
3
4     @Test
5     public void test1() {
6         count++;
7         assertEquals(2, count);
8     }
9
10    @Test
11    public void test2() {
12        count++;
13        assertEquals(2, count);
14    }
15 }
```

Entrambi i test avranno successo perché il framework di testing JUnit non specifica l'ordine di esecuzione dei test. In particolare, entrambi i metodi di test (`test1` e `test2`) modificano la variabile condivisa `count`, ma poiché JUnit può eseguire i test in qualsiasi ordine e con diverse istanze della classe di test, il risultato può variare.

Se JUnit esegue prima il metodo `test1`, allora la variabile `count` viene incrementata a 2. Quando successivamente esegue il metodo `test2`, la variabile `count` vale 2, quindi il test `assertEquals(2, count)` in `test2` avrà successo.

Invece, se JUnit esegue prima il metodo `test2`, allora la variabile `count` viene incrementata a 2. Quando successivamente esegue il metodo `test1`, la variabile `count` è già 2 e anche il test `assertEquals(2, count)` in `test1` avrà successo.

In entrambi i casi, tutti e due i test avranno successo anche se ci si aspettava che uno dei test fallisse. Questa è una delle ragioni per cui è importante scrivere test indipendenti che possano essere eseguiti in modo isolato.

Per convenzione, è consigliato organizzare tutti i test in una directory separata dal codice sorgente, comunemente denominata `test`. Questa pratica consente di facilitare il rilascio dell'applicazione, in quanto durante questa fase vogliamo distribuire solo il codice e non i test.

Inoltre, è consigliato chiamare la classe di test con lo stesso nome della classe originale che si intende testare, seguito dalla dicitura `Test`.

I metodi privati non dovrebbero essere oggetto di test diretti, poiché rappresentano dettagli implementativi. Questi metodi costituiscono una suddivisione logica del problema in unità più piccole. Nel caso in cui un metodo risulti eccessivamente ampio, è preferibile suddividerlo in metodi privati, semplificandone così la lettura. I metodi privati vengono implicitamente testati attraverso i metodi pubblici che li chiamano. Eseguendo un test sul metodo pubblico e verificando le asserzioni sul risultato o sullo stato dell'oggetto, possiamo garantire che i metodi privati funzionino correttamente. Pertanto, dobbiamo evitare di trasformare un metodo privato in un metodo pubblico solo per scopi di testing.

I metodi di asserzione forniti da JUnit sono sufficienti per **asserzioni di base**, ma possono diventare meno pratici in alcune situazioni più complesse. Infatti, questi metodi possono risultare controintuitivi nella loro formulazione, seguendo una struttura “verbo, oggetto, soggetto”. Invece, la formulazione più naturale per gli esseri umani è costituita dalla struttura “soggetto, verbo, oggetto”.

In situazioni in cui le asserzioni di JUnit non sono sufficienti o risultano poco intuitive, è possibile ricorrere all'uso di librerie aggiuntive specializzate per le asserzioni. In questo caso, JUnit può ancora essere utilizzato per l'esecuzione dei test, mentre la libreria aggiuntiva gestisce le asserzioni in modo più chiaro e efficace.

Di seguito analizziamo alcuni strumenti che possono essere integrati con JUnit per agevolare la scrittura di metodi di test più complessi, rendendoli più comprensibili e inerenti al linguaggio naturale.

2.1 Hamcrest

Hamcrest è una libreria di test aggiuntiva che si integra con JUnit 4, offrendo una sintassi più espressiva e leggibile per le asserzioni. Di seguito, effettuiamo un confronto tra le asserzioni di JUnit e le equivalenti utilizzando Hamcrest:

```
1 // Utilizzando JUnit 4
2 assertEquals(expected, actual); // verbo, oggetto, soggetto
3
4 // Utilizzando Hamcrest
5 assertThat(actual, equalTo(expected)); // soggetto, verbo, oggetto
```

2.2 AssertJ

AssertJ è una libreria che offre un modo più intuitivo e leggibile per scrivere le asserzioni nei test. Possiamo integrare AssertJ nel progetto nel seguente modo:

1. Scarichiamo il jar dalla pagina ufficiale di AssertJ. Alternativamente, possiamo ottenere il jar direttamente da Maven.
2. Creiamo una cartella nel progetto per ospitare i jar delle librerie esterne, ad esempio `libs`.
3. Copiamo il jar scaricato nella cartella creata.
4. Aggiungiamo il jar al classpath del progetto. In Eclipse, possiamo farlo cliccando con il pulsante destro del mouse sul jar e selezionando “Build Path” → “Add to Build Path”.

N.B.: questo non è il modo ideale di gestire le librerie esterne e discuteremo di una soluzione migliore utilizzando Maven in seguito.

AssertJ segue un approccio che si basa su un singolo metodo `assertThat`, simile a Hamcrest. Tuttavia, segue uno stile più fluido che rende la sintassi più chiara e comprensibile. In particolare, il metodo `assertThat` richiede un singolo argomento, rappresentato dall’oggetto su cui eseguire le asserzioni. Quindi, consente chiamare i metodi direttamente sul valore restituito da `assertThat`.

L’oggetto restituito da `assertThat` offre una serie di metodi di asserzione specifici per il tipo statico dell’oggetto (ad esempio, se l’oggetto è una lista, allora possiamo chiamare asserzioni specifiche per le liste). In particolare, osserviamo che AssertJ rende le asserzioni più leggibili e intuitive rispetto a JUnit e Hamcrest (possono essere lette come una frase in linguaggio naturale):

```
1 // Utilizzando JUnit 4
2 assertEquals(expected, actual);
3
4 // Utilizzando Hamcrest
5 assertThat(actual, equalTo(expected));
6
7 // Utilizzando AssertJ
8 import static org.assertj.core.api.Assertions.*;
9 assertThat(actual).isEqualTo(expected);
```

Inoltre, con AssertJ è possibile concatenare i metodi di asserzione. Naturalmente, il primo fallimento di qualsiasi asserzione causa il fallimento dell'intero test. Vediamo il seguente esempio di asserzione, in cui verifichiamo che la lista dei conti bancari ha un solo elemento che corrisponde ad uno specifico id:

```
1|assertThat(bankAccounts)
2|    .hasSize(1)
3|    .extracting(BankAccount::getId)
4|    .contains(newAccountID);
```



In caso di fallimento, le asserzioni di AssertJ forniscono messaggi di errore molto chiari e informativi, notevolmente più comprensibili rispetto agli errori di JUnit.

Capitolo 3

TEST-DRIVEN DEVELOPMENT



Il **Test-Driven Development (TDD)** rappresenta una metodologia di sviluppo basata sulla scrittura dei test prima dell'implementazione effettiva del codice. Pertanto, in questa metodologia gli Unit Test vengono progettati per fallire inizialmente. In questo contesto, i test svolgono diversi ruoli fondamentali nel processo di sviluppo:

- Garantiscono la qualità del codice.
- Guidano la progettazione e lo sviluppo del codice.
- Agiscono come specifiche, delineando le aspettative e le funzionalità desiderate.
- Forniscono una documentazione sul comportamento atteso del codice.
- Contribuiscono a scrivere codice che sia modulare, pulito e facilmente manutenibile.

Il **ciclo Red-Green-Refactor** è un processo iterativo che guida lo sviluppo attraverso tre stati ricorrenti:

1. **Red**: in questa fase dobbiamo scrivere un singolo test per una funzionalità o una caratteristica non ancora implementata. L'obiettivo è verificare che il test appena creato fallisca, poiché la funzionalità desiderata non è ancora presente.



N.B.: se il test implementato in questa fase non fallisce, allora potrebbe rappresentare un *falso positivo*. Questo significa che il test non sta verificando correttamente ciò che intendiamo testare.

2. **Green**: in questa fase dobbiamo scrivere il codice necessario a superare con successo il test precedentemente creato. L'obiettivo è trasformare il test dallo stato rosso allo stato verde. Dopo l'implementazione di questa nuova funzionalità, dobbiamo garantire che tutti i test esistenti siano ancora validi.

N.B.: in questa fase, il codice può essere scritto in modo grezzo.

3. **Refactoring**: rappresenta una fase opzionale che si concentra sulla pulizia e sul miglioramento del codice. Durante il refactoring, è possibile ottimizzare e migliorare la struttura del codice senza modificarne il comportamento. Al termine di questa fase dobbiamo ancora garantire che tutti i test vengano superati con successo, preservando l'integrità delle funzionalità implementate.



N.B.: i test rappresentano una Safety Net. Pertanto, durante il refactoring possiamo migliorare la qualità del codice senza preoccuparsi di rompere le funzionalità esistenti (il refactoring non deve cambiare la semantica o il comportamento del SUT). Qualora un test dovesse fallire, si presume che il refactoring abbia

introdotto una rottura ed è consigliabile eseguire un rollback ad una versione precedente.

N.B.: è consigliato eseguire il refactoring prima di avviare un nuovo ciclo, mantenendo così il codice pulito e manutenibile nel tempo.

N.B.: il refactoring non si limita al SUT ma anche ai test. Anche in questo caso, la modifica nei test non deve alterarne il comportamento previsto, ma deve concentrarsi sul miglioramento della leggibilità. Ricordiamo che, nella scrittura dei test, la duplicazione del codice è accettabile se contribuisce a renderli più comprensibili e chiari.

Al termine di ogni ciclo, viene testata ed implementata una singola funzionalità. Questo processo può essere ripetuto per aggiungere ulteriori funzionalità, garantendo che ogni iterazione introduca miglioramenti e mantenga l'affidabilità complessiva del sistema.

N.B.: ad ogni iterazione del ciclo, dobbiamo eseguire tutti i test per garantire che la nuova funzionalità non abbia compromesso il sistema. Per questo motivo, gli Unit Test devono essere veloci (ogni ciclo deve essere completato in pochi minuti). Se il processo richiede più tempo del previsto, allora potrebbe essere utile suddividere il problema in sottoproblemi più piccoli (la metodologia TDD ci costringe a concentrarci su compiti di piccole dimensioni).

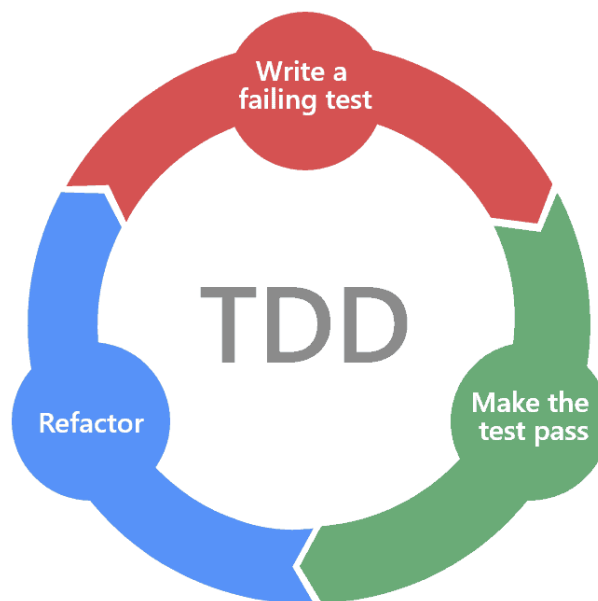


Figura 3.1: ciclo Red-Green-Refactor.

Affrontare la verifica e la correzione di un'applicazione già esistente può risultare una sfida complessa per diverse ragioni:

- L'applicazione potrebbe avere una struttura complessa e un codice di difficile comprensione, il che rende complicato introdurre nuovi test in modo efficace.
- Il processo di test del codice richiede una fase di pulizia attraverso il refactoring. Tuttavia, per effettuare il refactoring in modo sicuro, è fondamentale disporre di una Safety Net costituita dai test.

N.B.: questo porta ad un ciclo di dipendenze, in cui l'applicazione necessita di una rete di test che è difficile da implementare.

- I test potrebbero essere parziali, concentrandosi principalmente su ciò che il codice attuale fa piuttosto che rispecchiare i requisiti reali dell'applicazione.
- I test potrebbero tendere a confermare il comportamento esistente del codice, senza necessariamente testare se il codice soddisfa effettivamente i requisiti del cliente.
- I requisiti dovrebbero essere definiti in anticipo, in quanto delineano il comportamento del codice stesso. Questo principio è analogo per i test, i quali agiscono come specifiche eseguibili del sistema.

La metodologia TDD (Test-Driven Development) non è solamente una pratica di verifica del codice, ma rappresenta anche un approccio alla progettazione del codice stesso. Infatti, ci impone di ragionare sul comportamento desiderato del codice prima di procedere con la sua implementazione effettiva. Inoltre, TDD ci costringe a concentrarci su una singola funzionalità alla volta, organizzando il pensiero e strutturando il codice in modo chiaro.

L'utilizzo dei test consente uno sviluppo del codice rapido e pulito, limitando la possibilità di introdurre errori nel codice esistente. La creazione dei test prima dell'implementazione effettiva del codice guida il processo di sviluppo in modo controllato e metodico, garantendo una maggiore sicurezza nel processo di costruzione del software.

L'approccio di scrivere i test prima del codice ci impone di affrontare i problemi in modo graduale e semplificato, seguendo il principio di **Keep It Simple Stupid (KISS)**. Questo principio di design sottolinea che la maggior parte dei sistemi funziona in modo ottimale quando sono mantenuti in modo semplice invece che complesso. In particolare, la metodologia TDD rispetta appieno questo principio (poiché ci costringe a concentrarci su piccoli sottoproblemi e semplici).

Inoltre, TDD impone la creazione di codice modulare, aderendo al principio di **Modular Code**. Questo principio suggerisce di implementare il codice in modo che sia facilmente suddivisibile in moduli. In particolare, un codice modulare è più facile da testare, contribuendo così alla qualità e alla robustezza complessiva del sistema.

La metodologia TDD adotta un approccio in cui i test e il codice di produzione vengono sviluppati contemporaneamente, richiedendo un continuo passaggio tra la scrittura dei test e l'implementazione del codice di produzione. La frequenza di questo passaggio è governata dalle **3 leggi del TDD**. Tali leggi stabiliscono delle regole chiare per rendere semplice ed efficace lo sviluppo del codice, poiché ci costringono a modificare una sola funzione alla volta su problemi molto piccoli e semplici. In particolare, queste leggi definiscono quanto segue:

1. Non è consentito scrivere alcun codice di produzione, a meno che non serva a far passare uno Unit Test che fallisce.
N.B: finché non scriviamo un test che fallisce, non possiamo implementare nessuna funzionalità nel codice di produzione (neanche una classe vuota).
2. Non è consentito scrivere più codice di test di quello sufficiente a farlo fallire. Appena lo Unit Test fallisce, è necessario procedere alla scrittura del codice di produzione per superare il test.
N.B: gli errori di compilazione sono considerati come dei fallimenti. Infatti, quando iniziamo a scrivere il test, il codice di produzione relativo non è ancora presente (ad esempio, il test può fare riferimento a una classe mancante che non esiste).



Pertanto, nel momento in cui il test non viene compilato, è necessario sospendere la scrittura del test e procedere con la scrittura del codice di produzione per garantire la compilazione del test.

3. Non è consentito scrivere più codice di produzione di quello sufficiente a superare l'unico Unit Test fallito.

N.B.: una volta ottenuto superato il test, è necessario evitare di aggiungere ulteriore codice di produzione.

Durante ogni ciclo, tutti i test devono essere superati con successo. Tuttavia, se si verifica un problema in un ciclo, grazie alle 3 leggi del TDD dovrebbe essere facile tornare ad un precedente stato di funzionamento. Infatti, basta annullare ciò che è stato fatto nell'ultimo ciclo in cui tutti i test erano positivi.

Per far superare il test che sta fallendo, è essenziale apportare delle **trasformazioni**. Durante questo processo, il codice segue una regola fondamentale:



 **Transformation Priority Premise:** all'aumentare della specificità dei test, il codice di produzione deve diventare più generico.

Questa regola suggerisce che le trasformazioni dovrebbero seguire una priorità ben definita, mirando ad implementare con successo il ciclo di sviluppo Red-Green-Refactor. Quindi, l'ordine con cui applichiamo le trasformazioni diventa significativo, influenzando anche l'ordine dei test da scrivere (poiché la scrittura dei test precede l'applicazione di qualsiasi trasformazione).

Di seguito viene elencata una lista parziale di possibili trasformazioni, ordinate in base alla loro complessità (cioè, da semplici a complesse). Tutte queste trasformazioni mirano a rendere il comportamento del codice più generico:

1. `{}` → `null`: inizialmente si scrive il codice necessario per far compilare il test (ad esempio, la creazione della classe non esistente). La trasformazione più semplice consiste nel far restituire `null` dal codice.
2. `null` → `constant`: si sostituisce il codice che restituisce `null` con una costante.
3. `constant` → `scalar`: si sostituisce il codice che restituisce costante con una variabile o un argomento.
4. `statement` → `statements`: si sostituisce una sola istruzione con un blocco di istruzioni.
5. `unconditional` → `if`: si sostituisce un blocco di istruzioni con una condizione.
6. `statement` → `recursion`: si sostituisce un'istruzione con una ricorsione.
7. `if` → `while`: si sostituisce un blocco `if` con un ciclo `while`.
8. `expression` → `function`: si sostituisce un'espressione con una funzione o un algoritmo.

N.B.: le trasformazioni più semplici devono essere privilegiate rispetto a quelle più complesse. Inoltre, le trasformazioni più complesse devono essere applicate solo dopo aver implementato con successo quelle più semplici.

Questo principio guida anche la sequenza di scrittura dei test:

1. Innanzitutto, dobbiamo scrivere i test che si prevede possano essere soddisfatti applicando le trasformazioni più semplici.
2. Quando i test diventano più specifici, si applicano trasformazioni più complesse che trasformano il codice in qualcosa di più generico.



N.B.: la tecnica di **Transformation Priority Premise** ci consente di evitare una situazione di *vicolo cieco*, in cui test provoca la riscrittura di un intero metodo.

Il TDD costringe a concentrarci su problemi di piccole dimensioni, ma non impone la dimensione dei passi. In particolare, possiamo procedere con passi più piccoli o più grandi senza violare i principi e le regole del TDD:

- Quando ci sentiamo abbastanza sicuri dell'implementazione, possiamo procedere con passi più grandi.
- Quando siamo in difficoltà con il compito in corso o non siamo sicuri di come proseguire, possiamo procedere con passi più piccoli.



ES:

Consideriamo il seguente test che desideriamo soddisfare:

```
1 | @Test
2 | public void testDepositWhenAmountIsCorrectShouldIncreaseBalance() {
3 |     // Setup
4 |     BankAccount bankAccount = new BankAccount();
5 |     bankAccount.setBalance(5);
6 |
7 |     // Exercise
8 |     bankAccount.deposit(10);
9 |
10 |    // Verify
11 |    assertEquals(15, bankAccount.getBalance(), 0);
12 | }
```

Se procediamo con passi più grandi, otteniamo il seguente codice di produzione:

```
1 | public void deposit(double amount) {
2 |     balance += amount;
3 | }
```

Invece, se procediamo con passi più piccoli, otteniamo il seguente codice di produzione:

```
1 | public void deposit(double amount) {
2 |     balance = 15;
3 | }
```

Tuttavia, dobbiamo evitare di scrivere più codice di quello necessario per far passare il test (in questo caso scriviamo codice di produzione per un test che ancora non esiste, violando i principi del TDD). Un esempio di questa violazione è rappresentato dal seguente caso (non è stato scritto alcun test che verifica il lancio dell'eccezione):

```
1 public void deposit(double amount) {  
2     if (amount < 0) {  
3         throw new IllegalArgumentException("Negative amount: " +  
           ↪ amount);  
4     }  
5     balance += amount;  
6 }
```

Descriviamo tre possibili strategie per raggiungere lo stato Green:

- **Fake It** (*fingere*): inizialmente si restituisce una costante per far passare il test. Successivamente, si sostituisce gradualmente le costanti con le variabili.
- **Obvious Implementation** (*implementazione ovvia*): nel caso in cui si conosca l'implementazione reale, possiamo scriverla direttamente.
- **Triangulation** (*triangolazione*): inizialmente si implementa un singolo caso specifico. Quindi, si aggiunge un secondo caso e si adatta l'implementazione per gestire entrambi i casi. Gradualmente, si aggiungono ulteriori casi generalizzando il codice.

Le 3 leggi del TDD ci costringono a scrivere solo il codice di produzione necessario per far passare un test che fallisce, evitando di violare i test esistenti. La scrittura di un test che viene superato invece di fallire rappresenta un *falso positivo*, rappresentando un serio rischio per l'integrità del codice. Infatti, ci aspettiamo che il test fallisca poiché dovrebbe testare uno scenario che il nostro codice non è ancora in grado di gestire.

D'altra parte, le 3 leggi del TDD non vietano di scrivere test che ci aspettiamo abbiano esito positivo. Questi test hanno lo scopo di verificare uno scenario che il nostro codice di produzione dovrebbe essere già in grado gestire. In particolare, tali test possono essere utili per applicare la strategia di Triangulation, assicurandoci che il codice di produzione interagisca correttamente con i dati sperimentali (rappresentati dai test). Inoltre, la scrittura di test con esito positivo può risultare utile per documentare casi non immediatamente evidenti osservando altri test.

Tuttavia, se scriviamo un test di successo perché non siamo sicuri del funzionamento del nostro codice di produzione, potremmo allontanarci dalla corretta applicazione della metodologia TDD. Questo potrebbe indicare che stiamo affrontando un problema troppo grande per la parte di codice su cui stiamo lavorando, perdendo il focus sui piccoli problemi. Inoltre, potrebbe indicare che non stiamo seguendo la Transformation Priority Premise, applicando trasformazioni complesse prima di quelle semplici che portano ad un codice più complicato e difficile da comprendere.

Capitolo 4

CODE COVERAGE

La **Code Coverage** (*copertura del codice*) rappresenta la misura del numero di linee di codice eseguite durante l'esecuzione dell'applicazione.

In particolare, se l'applicazione è rappresentata dalla suite di test, allora la Code Coverage rappresenta la misura del numero di linee di codice eseguite durante i test. In questo caso si parla di **Test Coverage** (*copertura dei test*). In altre parole, la Test Coverage misura la quantità di codice testata durante l'esecuzione dei test.

N.B.: in ogni caso, utilizzeremo il termine generico Code Coverage per riferirci alla Test Coverage.

La Code Coverage viene espressa in percentuale e fornisce un'indicazione sulla quantità di codice che è stata sottoposta a testing. Tuttavia, questa valutazione dipende molto dall'efficacia dei test e dalla correttezza della loro implementazione.

In particolare, un codice di produzione con un'elevata Code Coverage e test ben scritti ha una minore probabilità di contenere bug non rilevati rispetto ad un codice con una bassa Code Coverage.

N.B.: se i test sono ben scritti, allora la presenza di un bug nel codice suggerisce con una certa sicurezza che tale bug si trovi in una porzione di codice che non è stata sottoposta a testing.

Quando la Code Coverage rileva linee di codice non coperte o rami mancanti, potrebbe essere presente un bug nel codice. Pertanto, un codice non coperto completamente deve essere considerato un allarme (specialmente se crediamo di aver testato tutti i percorsi possibili).

Per questo motivo, l'obiettivo che dobbiamo raggiungere con la Code Coverage è una copertura del 100%.

Tuttavia, tale obiettivo è richiesto nei contesti di codice contenente logica e non per classi con semplici getter e setter che non introducono logica significativa. Inoltre, siamo interessati soltanto alla copertura del codice di produzione e non alla copertura del codice di test. Pertanto possiamo escludere queste classi dall'analisi della Code Coverage.

N.B.: per escludere queste classi dall'analisi della Code Coverage, è sufficiente modificare la configurazione di lancio (ad esempio, rimuovendo la cartella sorgente dei test e lasciando solo la sorgente del codice).

N.B.: il 100% di Code Coverage non garantisce la qualità dei test, poiché non costituisce una condizione sufficiente per garantire la correttezza del codice. Tuttavia, è considerato una condizione necessaria. Infatti, se i test sono ben scritti e testano il corretto comportamento del codice, allora si ottiene una Code Coverage del 100%.

N.B.: se applichiamo correttamente la TDD, dovremmo ottenere automaticamente

una copertura del 100%, poiché questa metodologia richiede la scrittura dei test prima dell'implementazione del codice di produzione (pertanto, il codice implementato deve coprire i test scritti, altrimenti abbiamo qualche bug).

La misurazione della Code Coverage viene effettuata mediante uno strumento specializzato. Tale strumento analizza i file binari per monitorare le istruzioni in esecuzione, considerando anche scenari complessi (ad esempio, istruzioni `if-then-else`, cicli, `switch`, espressioni booleane, ecc...). In questo modo è in grado di analizzare la copertura su tutti i rami (*branch*) del codice.

In Java, lo strumento principale utilizzato per la Code Coverage è **JaCoCo**, integrato in Eclipse con **EclEmma**. In particolare, EclEmma funge da interfaccia per JaCoCo, semplificando la visualizzazione e l'analisi della Code Coverage.

JaCoCo implementa un *agente Java* progettato per eseguire qualsiasi applicazione Java. Questo agente genera un rapporto dettagliato delle righe di codice eseguite durante l'applicazione. Se sono presenti istruzioni *branch*, il rapporto indica anche se tutte le ramificazioni sono state eseguite durante l'applicazione.

N.B: un agente Java è un *plugin* della JVM, che si basa sull'Instrumentation API di Java.

EclEmma sfrutta JaCoCo per monitorare le istruzioni eseguite, ma offre anche la possibilità di generare un rapporto visuale nella vista “Coverage” dell'IDE. In particolare, l'editor viene colorato in base al rapporto, utilizzando i seguenti colori:

- **Verde**: indica che l'istruzione è stata eseguita (perciò, l'istruzione è coperta dai test).
- **Rosso**: indica che l'istruzione non è stata eseguita (perciò, l'istruzione non è coperta dai test).
- **Giallo**: indica che alcuni *branch* (ad esempio, quelli presenti nella struttura `if-then-else`) non sono stati eseguiti (perciò, l'istruzione è stata eseguita solo parzialmente).

Possiamo utilizzare EclEmma in Eclipse attraverso il suo launcher, per eseguire i test con la Code Coverage:

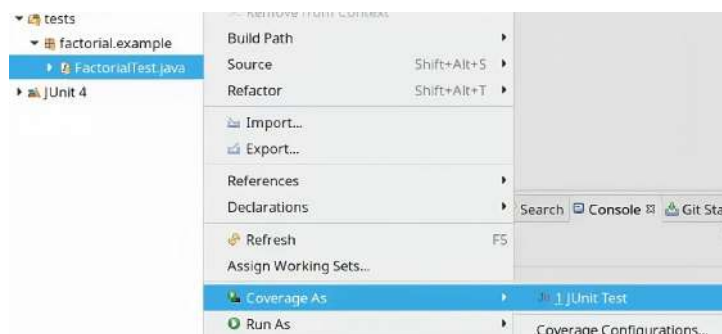


Figura 4.1: Launcher di EclEmma in Eclipse.

Capitolo 5

MUTATION TESTING

La valutazione della Code Coverage è fondamentale per rilevare le porzioni di codice che non sono state sottoposte a test. Di conseguenza, una bassa percentuale di Code Coverage può indicare un potenziale problema.

Tuttavia, anche una Code Coverage del 100% non garantisce che il codice sia corretto e ben testato (ad esempio, il comportamento del codice può essere modificato, ma i test continuano ad essere positivi a causa di un bug che non riescono a rilevare). In altre parole, ottenere una Code Coverage completa non è sufficiente per garantire che il sistema sia privo di errori o che i test siano esaustivi nella verifica del comportamento corretto del codice.

Il **Mutation Testing** (*test di mutazione*) rappresenta un meccanismo utile per valutare la qualità dei test. Questa metodologia si basa sulla realizzazione di piccole “mutazioni” del SUT. Ogni mutazione del SUT genera una variante chiamata **mutante**. Quindi, eseguiamo tutti i test sul mutante:

- Se almeno un test fallisce durante l'esecuzione sul mutante, allora abbiamo un esito positivo (non è necessario che tutti i test falliscano, ma almeno uno deve farlo). Se almeno un test fallisce, diciamo che “*il mutante è stato ucciso*”.
- Se tutti i test hanno successo durante l'esecuzione sul mutante, allora i test potrebbero non verificare in modo completo la complessità del codice sorgente. Se tutti i test vengono soddisfatti, diciamo che “*il mutante è sopravvissuto*”.

N.B.: esistono vari approcci per realizzare mutazioni del codice. Ogni mutazione genera un nuovo mutante.

N.B.: è fondamentale applicare una mutazione alla volta per valutare l'impatto in modo separato.



ES:

Supponiamo di avere nel SUT la seguente linea di codice:

```
1| if(a && b) { ... }
```

Allora, un mutante potrebbe consistere nella seguente riga di codice:

```
1| if(a || b) { ... }
```

Successivamente, eseguiamo tutti i test sul mutante. Se almeno un test fallisce, diciamo che “il mutante è stato ucciso”. In caso contrario, diciamo che “il mutante è sopravvissuto”.

N.B.: se i test verificano effettivamente il comportamento complessivo del SUT, allora tutti i mutanti devono essere “uccisi”.

Il Mutation Testing può essere eseguito attraverso alcuni strumenti automatici che generano mutanti in modo dinamico (*on the fly*). Per ogni mutante generato, questi strumenti eseguono tutti i test e segnalano quali mutanti sopravvivono. La creazione di mutanti si basa sull’impiego di un insieme di operatori di mutazione, noti come **mutatori**. Di seguito elenchiamo alcuni possibili mutatori:

- Cambiare l’operatore logico da `&&` a `||` in un’espressione booleana (o viceversa).
- Sostituire il risultato di un’espressione booleana con `true` (o `false`).
- Rimuovere un’assegnazione ad una variabile.
- Modificare i letterali (ad esempio, trasformando 0 in 1 o viceversa).
- Rimuovere completamente una chiamata ad un metodo.

PIT è un framework per il Mutation Testing in Java, disponibile anche come plugin per Eclipse chiamato **Pitclipse**. Questo strumento crea un mutante per ogni classe nel codice principale e per ogni mutatore disponibile. Quindi, per ogni mutante creato esegue tutti i test disponibili, raccogliendo ed aggregando i risultati ottenuti.

L’utilizzo di questi strumenti comporta un costo cubico. Pertanto, se si dispone di molte classi, il numero di mutanti generati sarà elevato. Poiché ogni mutante deve essere sottoposto a tutti i test, allora si avrà un aumento significativo del tempo di esecuzione dei test.

Per mitigare questo costo, è possibile adottare alcune strategie:

- Limitare il Mutation Testing ad un insieme ridotto di classi, concentrandosi soltanto sulle parti di codice che presentano una logica interessante e cruciale.
- Eseguire il Mutation Testing solo occasionalmente oppure su un Continuous Integration server.

N.B.: la Code Coverage ed il Mutation Testing introducono un overhead nel processo di testing, ma sono necessari per garantire la qualità del test. Pertanto, mentre gli Unit Test devono essere eseguiti continuamente, la Code Coverage ed il Mutation Testing possono essere eseguiti occasionalmente.

N.B.: in generale, se non si ha una Code Coverage del 100%, alcuni mutanti potrebbero sopravvivere. Tuttavia, il contrario non è necessariamente vero.

N.B.: nei cicli (*loop*), potrebbe verificarsi che alcuni mutanti vengano segnalati come **Timeout**. In questi casi, è possibile considerare questi mutanti come “uccisi”, poiché il test sul mutante non è riuscito a causa di un ciclo infinito che non restituisce un risultato entro il limite di tempo, causando il fallimento del test.

Capitolo 6

MAVEN

Per gestire correttamente le dipendenze di un progetto ed eseguire le operazioni di compilazione (*build*) e test, è necessario utilizzare uno strumento da *linea di comando* in grado di scaricare tutte le dipendenze del progetto, compilare l'intera applicazione, eseguire tutti i test e generare i report.

N.B: è importante separare le dipendenze del codice principale da quelle del codice di test, per garantire un ambiente di testing coerente.

N.B: l'automatizzazione di queste operazioni semplifica notevolmente l'integrazione con server di Continuous Integration, che risulta particolarmente utile per gestire progetti con numerosi test che richiedono tempi considerevoli per essere eseguiti. Inoltre consente di gestire un ambiente comune tra più sviluppatori.

Quando il numero di dipendenze cresce, la gestione manuale del download dei file JAR può diventare complessa e soggetta ad errori, specialmente considerando che alcune librerie esterne potrebbero richiedere dipendenze aggiuntive (note come **dipendenze transitive**).

L'inclusione dei file JAR nel progetto e l'aggiunta al classpath introduce ulteriori complessità nella distribuzione del progetto. Infatti, dopo aver rilasciato il progetto, dovremmo fornire agli utenti le informazioni necessarie per scaricare tutte le dipendenze. Nel caso di una libreria o di un framework, gli sviluppatori interessati devono procedere al download di tutte le dipendenze.

Invece, nel caso di un prodotto destinato agli utenti finali, è possibile incorporare tutte le dipendenze nel file JAR finale. Tuttavia, dobbiamo includere soltanto le dipendenze necessarie all'esecuzione dell'applicazione e non quelle specifiche per i test.

In ogni caso, la gestione manuale delle dipendenze rappresenta uno sforzo soggetto ad errori.

Maven è uno strumento di compilazione automatica (**automatic build**) e di gestione delle dipendenze (**dependency management**) per Java.

In particolare, Maven fornisce un repository centrale remoto, noto come **Maven Central**, in cui sono disponibili tutte le librerie Java esistenti.

Maven segue un ciclo di vita ed altri meccanismi ben definiti basati su *convenzioni*. Nonostante sia uno strumento molto rigido, consente alcune personalizzazioni.

Maven rappresenta un tentativo di applicare modelli alla complessità dell'infrastruttura di sviluppo di un progetto per promuovere la comprensione e la produttività.

Fondamentalmente, Maven funge da strumento di gestione e comprensione dei progetti, offrendo un approccio strutturato per gestire build, documentazioni, report, dipendenze, Version Control Systems, release e distribution.

Attraverso questa organizzazione, Maven mira a semplificare il processo di sviluppo e migliorare la collaborazione tra membri del team.

N.B: non combattere contro Maven, perderai.

Gli obiettivi di Maven includono:

- Creare **artefatti** (come JAR, WAR, EAR, ecc...).
- Eseguire operazioni collaterali (come la creazione di una release completa e la sua distribuzione su un repository remoto).
- Mantenere il processo di compilazione leggibile, riproducibile e consistente.

Maven promuove un approccio basato sulle **convenzioni** anziché sulla **configurazione**. Ovvero, invece di scrivere comandi specifici per la compilazione ed il testing, è sufficiente descrivere la struttura del progetto seguendo le convenzioni predefinite. Ad esempio:

- I file sorgente devono essere collocati nella directory `src/main/java`.
- I test devono essere collocati nella directory `src/test/java`.

Maven conosce già i comandi da eseguire e li applica automaticamente in base alla struttura del progetto. Inoltre, è possibile estendere le funzionalità di Maven attraverso l'uso di **plug-in**.

Maven semplifica il processo di gestione delle dipendenze e dei plug-in scaricandoli automaticamente dai repository Maven. Questi repository sono essenzialmente collezioni remote accessibili tramite il web. Per impostazione predefinita, Maven utilizza il repository principale **Maven Central**. Questo repository contiene tutte le librerie Java ed i plug-in Maven disponibili.

N.B: è possibile configurare Maven per utilizzare altri repository.

N.B: qualsiasi sviluppatore ha la possibilità di pubblicare i propri artefatti su Maven Central, anche se questo processo è regolamentato da una serie di procedure burocratiche (disponibili nella Central Repository Documentation).



6.1 Installazione

Per installare Maven, seguire questi passaggi:

1. Scaricare i file binari dal sito ufficiale di Maven.
2. Dopo aver scaricato i file, aggiungerli al **PATH** del Sistema Operativo.
3. Assicurarsi di avere installato il compilatore Java, cioè **JDK (Java Development Kit)**.

Possiamo anche utilizzare Maven da Eclipse, seguendo questi passaggi:

1. Assicurarsi di avere installato il plug-in di Eclipse chiamato **m2e - Maven Integration for Eclipse**.

N.B: tipicamente è incluso nelle distribuzioni principali di Eclipse. Altrimenti, può essere installato dal sito principale di Eclipse.

2. Una volta installato il plug-in m2e, non è necessario installare Maven separatamente nel Sistema Operativo. Il plug-in m2e contiene già una versione di Maven integrata.
3. Possiamo eseguire Maven direttamente da Eclipse utilizzando il plug-in m2e.
N.B.: non è richiesta l'esecuzione da riga di comando, a meno che non si voglia eseguire Maven al di fuori dell'ambiente Eclipse (in questo caso, Maven deve essere installato seguendo la precedente procedura).

Per creare un nuovo progetto Maven, è possibile utilizzare gli **archetipi**, che sono dei template di progetti iniziali. Ad esempio, è possibile creare un progetto Maven per una semplice applicazione Java o un'applicazione web Java.

Se Maven è installato nel Sistema Operativo, possiamo inizializzare un nuovo progetto Maven direttamente da riga di comando:

```
1| mvn archetype:generate \
2|   -DarchetypeGroupId=org.apache.maven.archetypes \
3|   -DarchetypeArtifactId=maven-archetype-quickstart \
4|   -DarchetypeVersion=1.4 \
5|   -DgroupId=com.mycompany.app \
6|   -DartifactId=my-app \
7|   -DinteractiveMode=false
8|
```



Questo comando utilizza l'archetipo `maven-archetype-quickstart` per creare un progetto Maven. Vediamo i parametri specificati cosa rappresentano:

- `-DarchetypeGroupId` `-DarchetypeArtifactId` e `-DarchetypeVersion` specificano l'archetipo da utilizzare.
- `-DgroupId` specifica il `groupId` del progetto.
- `-DartifactId` specifica l'`artifactId` del progetto.
- `-DinteractiveMode=false` evita l'interazione interattiva durante la creazione del progetto.



In alternativa, possiamo creare un nuovo progetto Maven direttamente da Eclipse (non richiede l'installazione di Maven nel Sistema Operativo) seguendo questi passaggi:

1. Aprire Eclipse ed assicurarsi che il plug-in m2e sia installato.
2. Selezionare “File” → “New” → “Other” nel menu di Eclipse.
3. Nella finestra di dialogo “Select a wizard”, espandere la categoria “Maven” e selezionare “Maven Project”. Fare clic su “Next”.

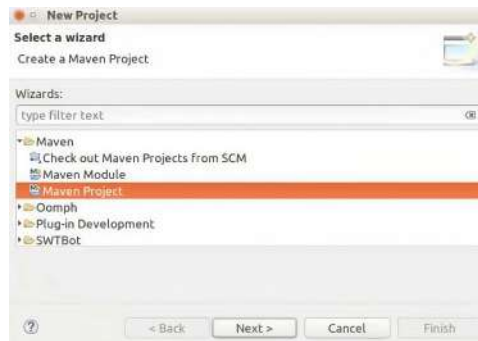


Figura 6.1: Finestra di dialogo “Select a wizard”.

4. Assicurarsi che l’opzione “Create a simple project (skip archetype selection)” sia selezionata. Fare clic su “Next”.
5. Nella schermata successiva, inserire le informazioni del gruppo e dell’artifact, come ad esempio `groupId` e `artifactId`. Queste informazioni identificano univocamente il progetto. Fare clic su “Finish”.

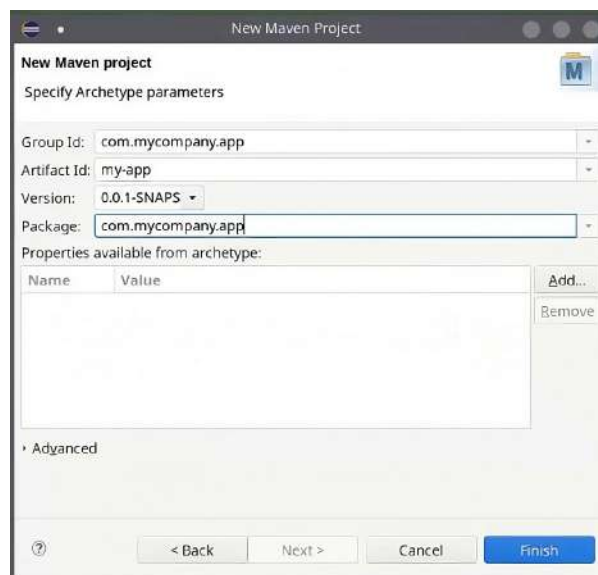


Figura 6.2: Schermata per specificare i parametri dell’archetipo.

6. Maven utilizzerà un archetipo predefinito per creare la struttura di base del progetto.
N.B.: se si desidera utilizzare un archetipo specifico, è possibile selezionare “Create from archetype” nella schermata delle informazioni del progetto e scegliere l’archetipo desiderato. In alternativa, tornare al passo 4 e deselectare l’opzione “Create a simple project (skip archetype selection)”. Quindi scegliere l’archetipo desiderato.

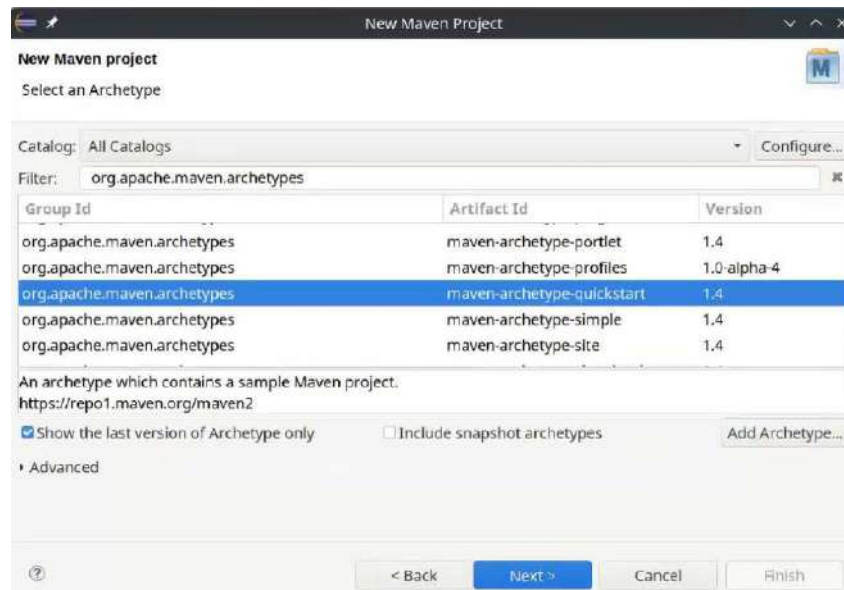


Figura 6.3: Schermata di selezione dell'archetipo desiderato.

Una volta inizializzato il progetto, verranno create le cartelle ed i file necessari per il progetto Maven all'interno della directory specificata dall'`artifactId`. La struttura delle cartelle può variare in base all'archetipo utilizzato.



Figura 6.4: Struttura del progetto Maven utilizzando l'archetipo `maven-archetype-quickstart`.

Un progetto Maven creato al di fuori di Eclipse non contiene i metadati del progetto Eclipse (cioè i file `.project` e `.classpath`). Tuttavia, possiamo importare un progetto Maven esistente in Eclipse seguendo i seguenti passaggi:

1. Aprire Eclipse e selezionare "File" → Import.
2. Nella finestra di dialogo di importazione, espandere la categoria "Maven" e selezionare "Existing Maven Projects". Fare clic su "Next".
3. Selezionare la Root Directory del progetto Maven esistente e fare clic su "Next".
4. Fare clic su "Finish" per completare il processo di importazione. Eclipse importerà automaticamente il progetto Maven esistente, creando i metadati del progetto (come `.project` e `.classpath`) e convertendolo in un progetto Eclipse.

La struttura del progetto Maven segue precise convenzioni per il codice principale e per i test, con lo scopo di organizzare in modo ordinato il codice di produzione ed i relativi test. In particolare, dobbiamo creare due cartelle separate (cioè, una per il codice principale ed una per i test). In questo contesto, Maven definisce le seguenti convenzioni:

- Ciò che si trova in `src/main/java` rappresenta il codice principale, ovvero il codice di produzione. Il codice presente in questa cartella sarà incluso nel JAR finale dell'applicazione. In particolare, il codice contenuto in questa cartella verrà compilato nella directory `target/classes` e impacchettato in un JAR.
- Ciò che si trova in `src/test/java` contiene i test. Il codice presente in questa cartella non sarà incluso nel JAR finale dell'applicazione. In particolare, il codice contenuto in questa cartella verrà compilato nella directory `target/test-classes`. Quindi, gli Unit Test verranno eseguiti seguendo una convenzione di denominazione, ma non saranno impacchettati nel JAR finale dell'applicazione.

N.B: Maven è un framework test-oriented. Pertanto, il plug-in `m2e` imposta automaticamente il percorso della cartella dei test in Eclipse.

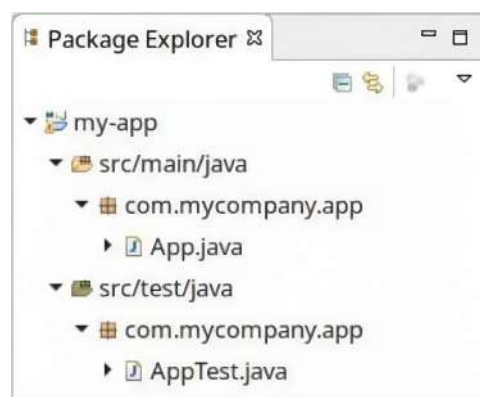


Figura 6.5: Convenzione delle cartelle relative al codice principale ed ai test.

Il file `pom.xml` rappresenta il modello ad oggetti del progetto Maven ed è collocato nella root del progetto. All'interno di questo file sono specificate diverse informazioni fondamentali per la gestione del progetto, tra cui:

- Elenco delle dipendenze esterne necessarie per il progetto.
- Configurazioni generali, come le opzioni di compilazione.
- Indicazione dei plug-in Maven utilizzati per eseguire determinate attività.

Il file `pom.xml` segue un'impostazione predefinita basata su convenzioni, le quali possono essere personalizzate secondo le esigenze specifiche del progetto.



ES (versione di Java):

Di default, il POM utilizza una vecchia versione di Java. Possiamo modifi-

care la versione direttamente dalle proprietà del progetto Eclipse seguendo i seguenti passi:

1. Clic destro sul progetto nel “Package Explorer”.
2. Selezionare “Properties”.
3. Andare su “Java Compiler”.

Tuttavia, questa soluzione modificherà la versione Java su Eclipse ma non su Maven. Pertanto, conviene agire direttamente sul file `pom.xml`:

```

1|<properties>
2|  <project.build.sourceEncoding>UTF-8</project.build.
   |    ↪ sourceEncoding>
3|  <!-- Change the Java compilation level to 8 -->
4|  <maven.compiler.source>1.8</maven.compiler.source>
5|  <maven.compiler.target>1.8</maven.compiler.target>
6|</properties>

```

Una volta salvata la modifica del file `pom.xml`, è possibile aggiornare il progetto Eclipse con il menu contestuale (“Maven” → “Update Project”). In questo modo, Eclipse aggiornerà la configurazione del progetto per utilizzare la versione specificata di Java.

Un progetto Maven è essenzialmente una directory contenente il file `pom.xml`, che rappresenta il **Project Object Model (POM)** in formato XML. Per eseguire Maven dalla riga di comando, è possibile utilizzare il comando `mvn` dalla directory che contiene il file `pom.xml`. In alternativa, è possibile specificare il percorso del file `pom.xml` utilizzando l'opzione `-f`.

6.2 Gestione delle dipendenze

Le dipendenze di un progetto devono essere indicate in una sezione specifica del file `pom.xml`, chiamata `<dependencies>`. Maven scaricherà automaticamente dal repository Maven Central le dipendenze specificate, a meno che non sia configurato diversamente (durante questo processo è necessaria una connessione ad Internet).

Le dipendenze scaricate saranno memorizzate localmente nell'apposita cache `.m2` della directory home.

N.B: se le dipendenze sono già presenti nella cache locale, queste non verranno nuovamente scaricate.

N.B: la directory `.m2` potrebbe crescere notevolmente nel tempo, raggiungendo anche alcuni gigabyte. In questo caso, può essere utile pulire periodicamente la cartella `.m2` e consentire a Maven di scaricare nuovamente gli artefatti necessari da zero.

Le **coordinate GAV (GroupId, ArtifactId, Version)** identificano univocamente ogni artefatto Maven, che può rappresentare un progetto, una dipendenza oppure un plug-in. Queste coordinate sono costituite da tre parti:

- **groupId:** identifica un gruppo, un'azienda, un'organizzazione, un team oppure un progetto che è il proprietario dell'artefatto. Rappresenta una sorta di namespace

ce per raggruppare gli artefatti. La convenzione suggerisce l'uso di una struttura simile ai pacchetti Java. Tale convenzione (accessibile dalla documentazione di Java) consiglia alle aziende di utilizzare il nome del loro dominio Internet (ad esempio, `com.example.mypackage` è un pacchetto chiamato `mypackage` creato da un programmatore di `example.com`).

N.B.: possiamo decidere la granularità del `groupId` in base alla struttura del progetto.

N.B.: prima di rilasciare l'artefatto su Maven Central, è necessario dimostrare di essere il proprietario del dominio Internet associato al `groupId`.

N.B.: se i progetti sono ospitati su GitHub, si può utilizzare `io.github.<user_name>` come `groupId` dell'artefatto.

- **artifactId:** identifica in modo univoco il nome dell'artefatto all'interno dell'organizzazione o del progetto. Per convenzione, si utilizzano lettere minuscole e il trattino per separare le parole (ad esempio, `log4j-core`, `log4j-api` o `assertj-core`).
 - **version:** identifica la versione specifica dell'artefatto. Per convenzione, si utilizzano lettere alfanumeriche e punti. (ad esempio, `1.0`, `2.0.1-RC1` o `2.0.0.1-alpha`).
- N.B.:** se termina con `-SNAPSHOT`, indica che la versione è temporanea (ovvero, è ancora in fase di sviluppo e non è stata rilasciata in modo stabile). Pertanto, è fondamentale rimuovere `-SNAPSHOT` prima del rilascio ufficiale.

Questo sistema di coordinate GAV è essenziale per identificare in modo univoco e recuperare le dipendenze necessarie durante il processo di compilazione di un progetto Maven.

Le dipendenze devono essere dichiarate nel file `pom.xml`, all'interno della sezione delle `<dependencies>`. Ciascuna dipendenza viene specificata come un elemento `<dependency>`, indicando le sue coordinate GAV ed opzionalmente lo scope.

Lo **scope** (*ambito*) determina in quale classpath sarà disponibile la dipendenza e come sarà propagata verso i progetti dipendenti. Sono disponibili diversi scope, ma ne utilizzeremo principalmente due:

- **compile:** la dipendenza è disponibile in tutti i classpath ed è transitiva verso altri progetti.
- N.B.:** se non viene specificato nessuno scope, allora `compile` sarà lo scope di default.
- **test:** la dipendenza è disponibile solo nel classpath di test e non è transitiva verso altri progetti.



ES:

Consideriamo la seguente dichiarazione delle dipendenze nel file `pom.xml`:

```

1 | <dependencies>
2 |   <dependency>
3 |     <groupId>com.example</groupId>
4 |     <artifactId>my-library</artifactId>
5 |     <version>1.0.0</version>

```

```

6      <!-- The default scope is "compile" -->
7  </dependency>
8
9  <dependency>
10     <groupId>junit</groupId>
11     <artifactId>junit</artifactId>
12     <version>4.12</version>
13     <scope>test</scope>
14 </dependency>
15 </dependencies>

```

La dipendenza `my-library` ha uno scope `compile`, perciò sarà disponibile in tutti i classpath. Invece, la dipendenza `junit` ha uno scope `test`, perciò sarà disponibile solo nei test del progetto e non sarà transitiva verso altri progetti.

Eclipse m2e funge da intermediario tra la gestione delle dipendenze di Maven ed il compilatore di Eclipse. Durante la prima esecuzione, è necessario attendere che le dipendenze vengano scaricate e memorizzate nella cache `.m2`. Inoltre, m2e imposta automaticamente lo scope delle dipendenze di Maven nel progetto Eclipse.

Eclipse m2e offre un editor multi-tab per il file `pom.xml`.

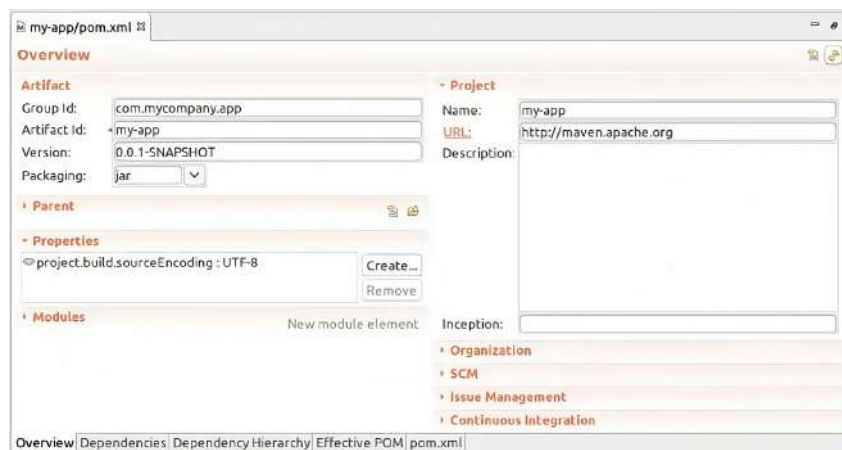


Figura 6.6: Editor multi-tab di m2e.

Inoltre, m2e consente di esaminare le sorgenti delle dipendenze. Quando si apre una classe da un file JAR per la prima volta, le sue sorgenti vengono automaticamente scaricate.

N.B.: di default, sia le sorgenti che i Javadoc vengono scaricati insieme ai file binari. Tuttavia, possiamo personalizzare questa configurazione dalle preferenze di Eclipse.

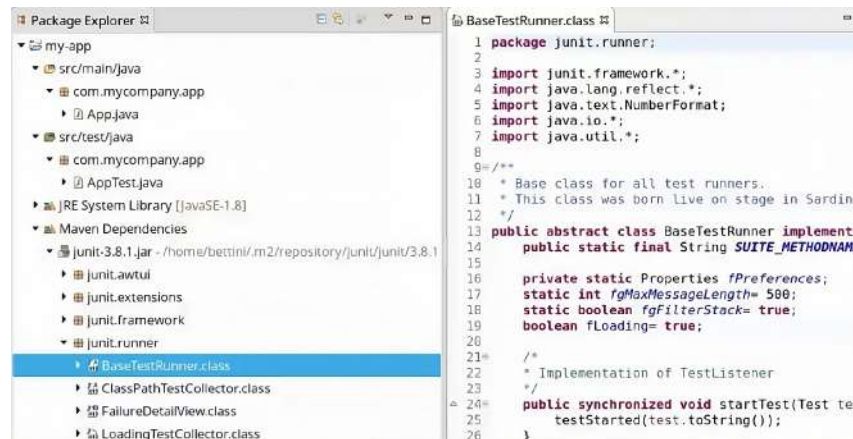


Figura 6.7: Analisi della sorgente di una dipendenza.

Possiamo specificare una versione diversa di una dipendenza modificando la sua versione nel file `pom.xml`. In questo caso, Eclipse scaricherà automaticamente la nuova versione quando si salva il `pom.xml` (a meno che non sia già presente nella cache locale) ed utilizzerà la nuova versione nel classpath del progetto.

Inoltre, Maven gestisce automaticamente le dipendenze transitive. Infatti, basta specificare le dipendenze di primo livello e tutte le dipendenze transitive saranno scaricate automaticamente (ad esempio, se specifichiamo JUnit 4.12 come dipendenza, Maven scaricherà automaticamente anche la dipendenza transitiva hamcrest-core).

N.B: Eclipse m2e utilizza il file `pom.xml` per scaricare le dipendenze, ma la compilazione avviene comunque automaticamente da Eclipse. Pertanto, alcune configurazioni devono essere duplicate e mantenute sincronizzate (ad esempio, è necessario assicurarsi che la versione del compilatore Java specificata nel file `pom.xml` sia coerente con quella configurata in Eclipse).

Le **proprietà** di Maven vengono definite nella sezione `<properties>`. Tale sezione contiene coppie key/value per definire le costanti utilizzate durante la compilazione. Le proprietà di Maven possono essere utilizzate per rendere la compilazione parametrica, in modo da generare build con differenti valori senza modificare il file `pom`. Inoltre, le proprietà incrementano la leggibilità del codice e mantengono la consistenza delle versioni nello stesso progetto.

Possiamo riferirci alle proprietà definite nel file `pom.xml` con la sintassi `${<property_name>}`. Inoltre, le proprietà possono essere ridefinite dalla riga di comando con la sintassi standard di Java.



ES:

Definiamo le seguenti proprietà nel file `pom.xml`:

```
1 <properties>
2   <project.build.sourceEncoding>UTF-8</project.build.
   ↪ sourceEncoding>
3   <junit.version>4.11</junit.version>
4 </properties>
```

Nel file `pom.xml`, possiamo fare riferimento alla proprietà `junit.version` utilizzando la sintassi `${junit.version}`.

```

1| <dependencies>
2|   <dependency>
3|     <groupid>junit</groupid>
4|     <artifactid>junit</artifactid>
5|     <version>${junit.version}</ version>
6|     <scope>test</ scope>
7|   </dependency>
8| </dependencies>

```

Inoltre, possiamo ridefinire questa proprietà direttamente dalla riga di comando con la sintassi Java (ad esempio, `-junit.version=4.12` cambia il valore della proprietà `junit.version` definita nel file `pom.xml` da 4.11 a 4.12).

N.B: possiamo anche definire una proprietà dalla riga di comando con la stessa sintassi.



Maven mette a disposizione delle proprietà standard che possono essere utilizzate nel file `pom.xml` per fare riferimento a cartelle standard, semplificando il riferimento a percorsi relativi che Maven convertirà in percorsi assoluti. Alcune di queste proprietà standard includono:

- `${project.basedir}`: fa riferimento alla cartella principale del progetto (*root*), ovvero la posizione del file `pom.xml`.
- `${project.build.directory}`: di default, fa riferimento alla cartella `target` in cui vengono generati tutti gli artefatti.
- `${project.build.outputDirectory}`: di default, fa riferimento alla cartella `target/classes` dove viene compilato tutto il codice principale di Java.
- `${project.build.sourceDirectory}`: di default, fa riferimento alla cartella `src/main/java` contenente il codice sorgente di produzione.

Alcuni plug-in Maven consentono di impostare delle proprietà, altrimenti utilizzano dei valori predefiniti. Queste informazioni sono generalmente documentate sul sito web del plug-in. Possiamo configurare queste proprietà per i plug-in in modo simile a quanto fatto per il plug-in del compilatore Java di Maven



ES:

Possiamo specificare la versione del compilatore Java di Maven dalle proprietà del plug-in:

```

1| <properties>
2|   <project.build.sourceEncoding>UTF-8</project.build.
   |   ↪ sourceEncoding>
3|   <!-- Change the Java compilation level to 8 -->
4|   <maven.compiler.source>1.8</maven.compiler.source>
5|   <maven.compiler.target>1.8</maven.compiler.target>
6| </properties>

```

Le **risorse** in un progetto Maven sono gestite seguendo il principio di separazione tra il codice sorgente e le risorse. Per convenzione, le risorse vengono collocate nelle seguenti directory:

- **src/main/resources**: contiene le risorse utilizzate dal codice principale. 
- **src/test/resources**: contiene le risorse utilizzate esclusivamente durante i test. **N.B.**: durante l'esecuzione dei test, le risorse presenti in **src/test/resources** sovrascrivono eventuali risorse con lo stesso nome presenti in **src/main/resources**.

6.3 Automazione della build

Durante il processo di compilazione (*build*) con Maven, vengono eseguiti diversi task:

- Le dipendenze vengono scaricate, a meno che non siano già presenti nella cache.
- Il codice principale, di solito situato in **src/main/java**, viene compilato utilizzando solo le dipendenze con scope **compile**.
N.B.: le dipendenze con scope **test** non vengono compilate poiché servono solo a scopo di testing.
- Il codice di test, di solito situato in **src/test/java**, viene compilato insieme al codice principale ed utilizzando sia le dipendenze con scope **test** che **compile**.
N.B.: i test vengono compilati in un classpath contenente le dipendenze con entrambi gli scope **test** e **compile**. Inoltre, nel classpath del test viene compilato anche il codice principale per consentire ai test di accederci.
- Vengono eseguiti i test, e i loro risultati vengono registrati.
- Viene creato un file JAR contenente solo il codice principale.

Per impostazione predefinita, tutti gli artefatti generati saranno posizionati nella directory **target** situata nella cartella *root* contenente il file **pom.xml**. Questa cartella è comunemente nota come **build directory** (*directory di compilazione*).

N.B.: possiamo rimuovere questa directory in modo sicuro in quanto il suo contenuto verrà ricreato o sovrascritto durante la successiva compilazione.

N.B.: dobbiamo escludere o ignorare questa directory quando si utilizza un Version Control System (ad esempio Git).

Il ciclo di vita di Maven, noto come **Maven Lifecycle**, rappresenta il cuore del processo di compilazione in cui vengono connessi tutti i concetti ed i plug-in di Maven. Esso rappresenta un elenco ordinato di *fasi* (**phases**), ciascuna delle quali ha associati degli *obiettivi* (**goal**) forniti dai plug-in Maven.

I plug-in definiscono dei goal associati a determinate fasi del ciclo di vita. Alcuni goal dei plug-in sono vincolati di default a delle fasi specifiche, ma è possibile anche vincolare esplicitamente i goal a fasi specifiche nel file **pom.xml**.

Maven definisce tre cicli di vita principali: 

- **clean**: rappresenta il ciclo di vita in cui **si rimuovono tutti i file generati dalle build precedenti**, consentendo di iniziare una nuova build da uno stato pulito.

- **default**: rappresenta il ciclo di vita predefinito e gestisce la compilazione e la distribuzione del progetto. Genera tutti gli artefatti della build e li rende disponibili per l'utilizzo oppure per il rilascio.
- **site**: rappresenta il ciclo di vita in cui si genera un sito web per includere la documentazione generata automaticamente, i report ed altri dettagli utili (ad esempio, può contenere report di Code Coverage generati da strumenti come JaCoCo).

Ogni ciclo di vita definisce un elenco ordinato di fasi. Quando si richiede l'esecuzione di una specifica fase, Maven esegue in ordine tutte le fasi precedenti definite nel ciclo di vita. Per ogni fase, vengono eseguiti tutti i goal associati ad essa e definiti dai plug-in che sono attivi nel `pom.xml`.

N.B.: i nomi dei cicli di vita, delle fasi e dei goal possono coincidere.

N.B.: l'associazione predefinita tra i goal dei plug-in e le fasi dipende anche dal tipo di packaging del progetto.

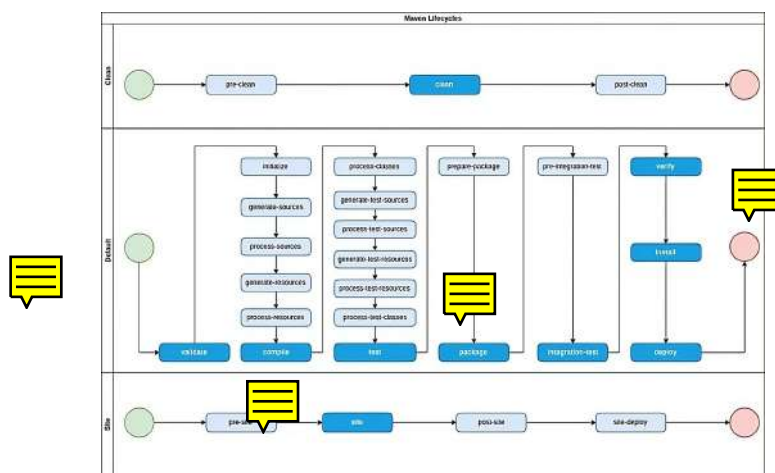


Figura 6.8: cicli di vita e relative fasi di Maven.

Per avviare Maven, è possibile utilizzare il comando `mvn` seguito dal nome di una o più fasi, oppure dal nome di un goal (utilizzando una sintassi speciale).

Per impostazione predefinita, la compilazione termina con un fallimento se si verifica uno qualsiasi dei seguenti scenari:

- Un file sorgente principale di Java non può essere compilato.
- Un file sorgente dei test Java non può essere compilato.
- Uno Unit Test fallisce durante l'esecuzione.

Altrimenti, la compilazione viene considerata riuscita.



ES (clean):

Il ciclo di vita `clean` definisce le fasi `pre-clean`, `clean`, e `post-clean` nell'ordine specificato.

Se lanciamo il comando `mvn clean`, allora verranno eseguite le fasi `pre-clean` e `clean` (poiché la fase `pre-clean` precede la fase `clean`).

Il plug-in `maven-clean-plugin`, abilitato di default, definisce il goal `clean` associato alla fase `clean` del ciclo di vita `clean`.

Pertanto, se lanciamo il comando `mvn clean`, allora verrà eseguito il goal `maven-clean-plugin:clean` durante la fase `clean` (per impostazione predefinita, nessun altro plug-in ha goal associati al ciclo di vita `clean`).

Di default, il goal `clean` del plug-in `maven-clean` rimuove la cartella `target` della build.

N.B.: possiamo configurare il goal `clean` del plug-in `maven-clean` per rimuovere anche altre cartelle oppure per escludere specifiche cartelle durante l'operazione di pulizia.



ES (compile):

Il plug-in `maven-compiler-plugin`, abilitato per impostazione predefinita, definisce il goal `compile` associato alla fase `compile` del ciclo di vita `default`.

Pertanto, quando lanciamo il comando `mvn compile`, il goal `maven-compiler-plugin:compile` verrà eseguito durante la fase `compile`. Questo goal compila i file sorgenti Java presenti in `src/main/java`, utilizzando il compilatore del JDK e generando i file `.class` nella directory `target/classes`.

La fase `compile` nel ciclo di vita `default` sarà eseguita solo dopo aver completato tutte le fasi precedenti. Tra queste fasi, troviamo `process-resources`, associata per impostazione predefinita al goal `resources` del plug-in `maven-resources-plugin`, il quale copierà il contenuto di `src/main/resources` in `target/classes`.



ES (clean compile):

Se lanciamo il comando `mvn clean compile`, verranno eseguiti i seguenti goal:

- Il goal `clean` del plug-in `maven-clean-plugin` (associato alla fase `clean` del ciclo di vita `clean`).
- Il goal `resources` del plug-in `maven-resources` (associato alla fase `process-resources` del ciclo di vita `default`).
- Il goal `compile` del plug-in `maven-compiler` (associato alla fase `compile` del ciclo di vita `default`).

Il risultato di questo comando sarà una completa ricompilazione dei file sorgenti Java.



ES (test): Se lanciamo il comando `mvn test` verranno completati i seguenti task:

- I file sorgenti principali di Java vengono compilati in `target/classes`.
- Le risorse principali vengono copiate in `target/classes`.
- I file sorgenti dei test Java vengono compilati in `target/test-classes`.
- Le risorse dei test vengono copiate in `target/test-classes`.
- Vengono eseguiti gli Unit Test utilizzando il goal `test` del plug-in `maven-surefire-plugin`, il quale è associato per impostazione predefinita alla fase `test` del ciclo di vita `default`. I risultati degli Unit Test vengono generati nella cartella `target/surefire-reports`.

N.B: per impostazione predefinita, gli Unit Test eseguiti corrispondono alle classi non astratte che corrispondono ai pattern `**/*Test*.java`, `**/*Test.java`, `**/*Tests.java` e `**/*TestCase.java`. Tuttavia, il plug-in può essere configurato per includere o escludere specifiche classi di test.

N.B: l'ordine delle fasi è importante. Infatti, ha perfettamente senso compilare il codice di test soltanto dopo aver compilato il codice principale (poiché il codice di test necessita del codice di produzione per funzionare). Inoltre, se il codice principale non può essere compilato, allora la build viene interrotta e di conseguenza non viene compilato neanche il codice di test. Infatti, non ha senso compilare il codice di test se il codice principale contiene errori di compilazione.



Come abbiamo detto, è possibile eseguire un singolo goal utilizzando una sintassi specifica `<groupId>:<artifactId>:<goal>`.

Nel caso in cui si tratti di un plug-in standard di Maven, possiamo utilizzare la forma contratta `<artifactId>:<goal>` (cioè, possiamo usare la forma abbreviata quando un plug-in è del tipo `maven-<artifactId>-plugin` oppure `<artifactId>-maven-plugin`). Quando un plug-in prevede diversi goal con lo stesso nome associati alla stessa fase, possiamo specificare l'id di una singola esecuzione per quel goal con la seguente sintassi `<groupId>:<artifactId>:<goal>@<id>`.

N.B: quando lanciamo soltanto goal specifici, dobbiamo assicurarci che la loro esecuzione abbia senso. Ad esempio, se eseguiamo il comando `mvn compiler:testCompile`,



verrà compilato soltanto il codice di test. Tuttavia, se il codice principale non è stato compilato in precedenza, il codice di test non può essere compilato correttamente.

Se siamo certi di avere tutte le dipendenze memorizzate nella cache locale, è possibile eseguire Maven in modalità offline. Ciò può essere fatto da riga di comando utilizzando l'opzione `-o`, oppure da Eclipse selezionando l'apposita casella di controllo. In questo modo, in modalità offline, Maven eviterà di contattare i repository remoti.

N.B.: prima di passare alla modalità offline, è consigliabile assicurarsi di avere tutte le dipendenze nella cache eseguendo il comando `mvn dependency:go-offline`.

Per lanciare Maven da Eclipse, dobbiamo seguire questi passaggi:

1. Fare clic con il tasto destro del mouse sul file `pom.xml`.
2. Selezionare l'opzione "Run As" e poi "Maven Build..."

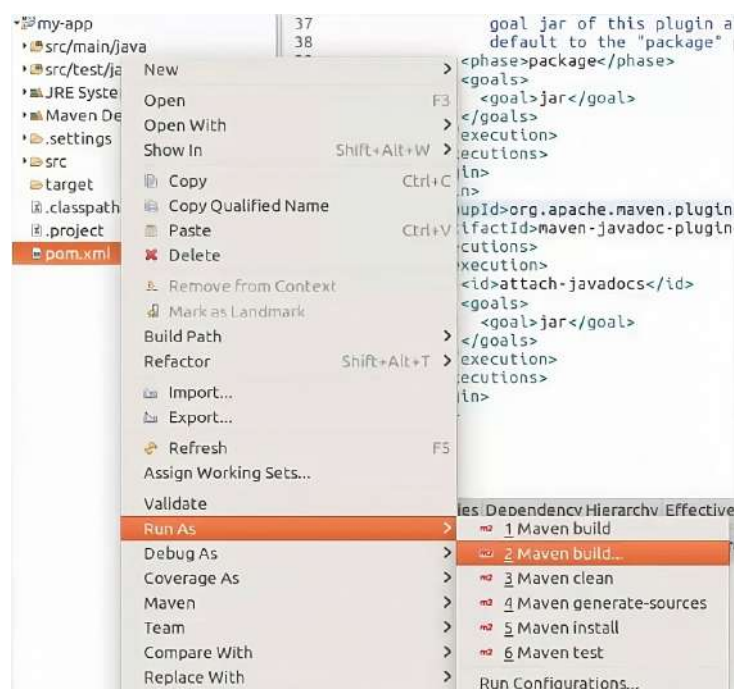


Figura 6.9: Esecuzione di Maven da Eclipse.

3. Specificare le fasi o i goal desiderati.
4. Fare clic su "Run".

N.B.: l'output dell'esecuzione verrà mostrato nell'editor "Console".

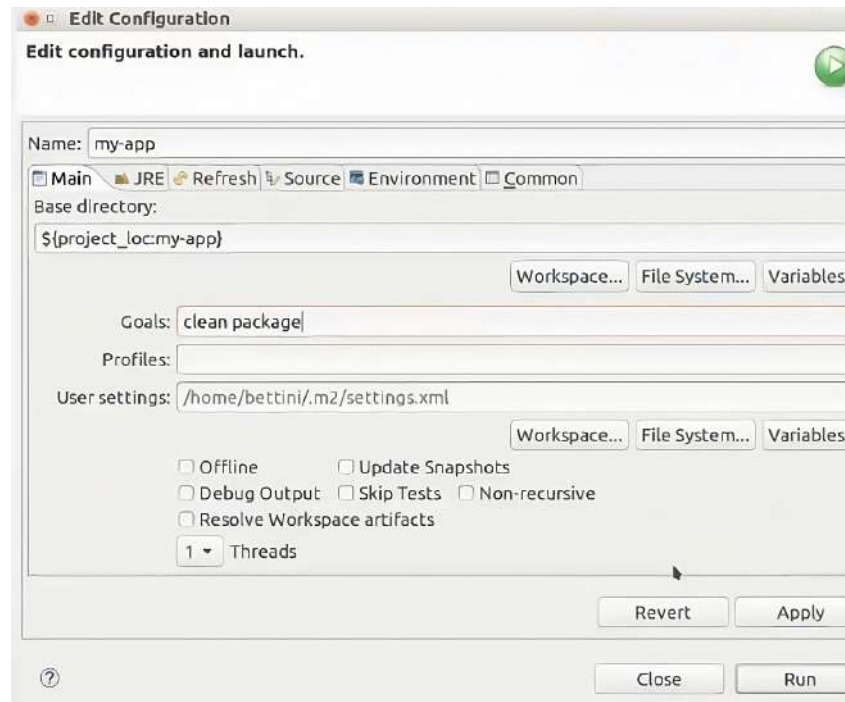


Figura 6.10: Configurazione dell'esecuzione.



6.4 Packaging con Maven

Ogni progetto Maven specifica uno ed un solo tipo di **packaging**, che identifica l'obiettivo principale della compilazione di quel progetto ed il tipo di artefatti generati (come JAR, WAR, EAR, ecc...). Il packaging di default è JAR.

Il packaging POM non genera alcun artefatto. Un progetto di tipo POM può essere utilizzato per vari scopi:

- Dichiarare dipendenze e configurazioni comuni presenti in più progetti.
- Costruire più progetti Maven, chiamati **moduli Maven**.
- Coordinare e gestire le dipendenze e la build tra vari moduli (cioè, la combinazione delle due precedenti).

Ogni progetto Maven eredita implicitamente dal POM principale (noto come **Super POM**), che fa parte dell'installazione di Maven. Questo Super POM fornisce alcune proprietà e configurazioni di base per i plug-in.

Un POM può fare riferimento ad un POM genitore (**Parent POM**) per ereditare le sue configurazioni, specificandone le coordinate GAV nella sezione `<parent>`.

N.B.: il Super POM è sempre considerato il POM genitore di ogni progetto Maven.

N.B.: se nel POM non vengono specificati `groupId` e `version`, questi vengono ereditati dal POM genitore.

N.B.: per impostazione predefinita, il file del POM genitore viene cercato nella directory padre. In caso contrario, è possibile specificarlo esplicitamente in `relativePath`.



**ES:**

Definiamo il POM genitore per ereditare le sue configurazioni:

```

1 <parent>
2   <groupId>com.mycompany.myproject</groupId>
3   <artifactId>my-parent-pom</artifactId>
4   <version>1.0-SNAPSHOT</version>
5   <relativePath>../my.parent/pom.xml</relativePath>
6 </parent>

```

Il **Maven Reactor** rappresenta il meccanismo per effettuare la build di progetti che consistono in più moduli, detti **multi-modulo**. Esso opera nel seguente modo:

- Raccoglie tutti i moduli da compilare, i quali sono specificati nella sezione `<modules>` dei file `pom.xml`.
- Ordina i moduli in base alle relazioni di dipendenza (cioè, compila prima i moduli che sono necessari ad altri moduli).

N.B.: l'ordine specificato nella sezione `<modules>` non è rilevante, poiché Maven esegue un riordinamento automatico.

N.B.: i cicli di dipendenze non sono ammessi.

- Proceda con la compilazione di un modulo alla volta.



In sostanza, il Maven Reactor si occupa di gestire la costruzione dei moduli in modo efficiente, risolvendo le dipendenze in modo sequenziale.

Per importare un progetto multi-modulo in Eclipse i cui moduli sono presenti in sotto-directory, dobbiamo assicurarci di selezionare l'opzione "Search for nested projects".

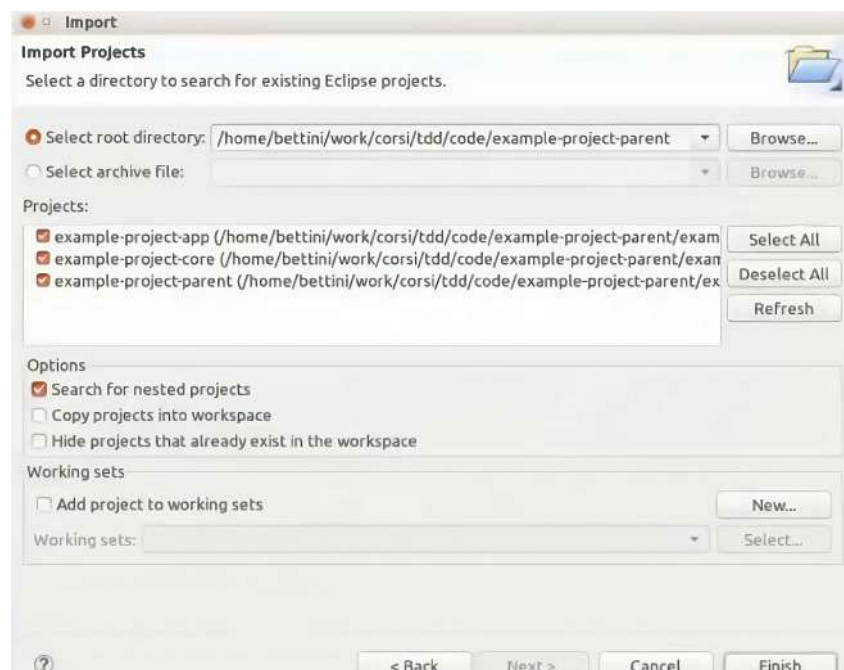


Figura 6.11: Impostazioni per importare un progetto multi-modulo.

Un POM può svolgere il ruolo sia di *genitore*, noto come **Parent POM**, che di *aggregatore*, denominato **Aggregator POM**.

Il Parent POM contiene dipendenze e configurazioni comuni, ma non include la sezione `<modules>`. Queste configurazioni saranno ereditate dai sottomoduli collegati al POM genitore.

Invece, l'Aggregator POM contiene soltanto la sezione `<modules>` e non richiede ulteriori relazioni di ereditarietà.

N.B.: un POM può svolgere entrambi i ruoli. Tuttavia, in generale, questi due concetti sono distinti.

N.B.: ogni modulo può avere un solo POM genitore. Tuttavia, un POM genitore può ereditare da un altro POM genitore, formando una catena di ereditarietà (cioè, la relazione di ereditarietà è lineare e singola).

N.B.: è possibile avere diversi POM aggregatori (ad esempio, per compilare solo un sottoinsieme di moduli di un progetto multimodulo), a condizione che siano rispettate tutte le dipendenze tra i moduli.

Quando viene dichiarata una dipendenza nel POM genitore, essa viene ereditata da tutti i moduli del progetto.

Tuttavia, potremmo trovarci in una situazione in cui desideriamo che solo alcuni moduli ereditino tale dipendenza. Per gestire questa situazione, possiamo utilizzare la sezione `<dependencyManagement>` nel POM genitore. In questa sezione, possiamo specificare le coordinate GAV e altre configurazioni per la dipendenza.

Tuttavia, la dipendenza non verrà attivata automaticamente per tutti i moduli. Per abilitare la dipendenza solo nei moduli che ne hanno bisogno, possiamo dichiarare la dipendenza nei POM dei singoli moduli, specificando il `groupId` e l'`artifactId`.

N.B.: non è necessario ripetere la versione, in quanto questa è già definita nella sezione `<dependencyManagement>` del POM genitore (in questo modo abbiamo coerenza tra le dipendenze utilizzate dai moduli).



ES:

Vediamo un esempio per illustrare la gestione delle dipendenze in un progetto multi-modulo. Innanzitutto, specifichiamo le dipendenze condivise nel POM genitore:

```
1 | <!-- Parent POM -->
2 | <project>
3 |   ...
4 |   <dependencyManagement>
5 |     <dependencies>
6 |       <dependency>
7 |         <groupId>com.example</groupId>
8 |         <artifactId>example-dependency</artifactId>
9 |         <version>1.0.0</version>
10 |         <!-- Other configurations as needed -->
11 |       </dependency>
12 |     </dependencies>
13 |   </dependencyManagement>
14 |   ...
15 | </project>
```

Successivamente, specifichiamo la dipendenza nella sezione `<dependencies>` dei moduli che ne hanno bisogno:

```

1 <!-- Module POM -->
2 <project>
3   ...
4   <dependencies>
5     <dependency>
6       <groupId>com.example</groupId>
7       <artifactId>example-dependency</artifactId>
8       <!-- No need to specify the version, inherited from the
          ↳ parent POM -->
9     </dependency>
10    <!-- Other dependencies specific to the module -->
11  </dependencies>
12  ...
13 </project>

```

Un **BOM (Bill of Materials)** rappresenta uno speciale tipo di POM, utilizzato per gestire le versioni delle dipendenze di un progetto. In particolare, il BOM consente di definire ed aggiornare le versioni delle dipendenze all'interno di un progetto multi-modulo (o tra diversi progetti) in modo *centralizzato*. In questo modo, il BOM consente di aggiungere le dipendenze ad un modulo senza doverne specificare la versione.

Infatti, quando due dipendenze fanno riferimento a versioni diverse dello stesso artefatto, possono sorgere dei conflitti (Maven risolverà il conflitto utilizzando la versione più vicina nell'albero delle dipendenze). Tuttavia, possiamo evitare questi conflitti utilizzando la sezione `<dependencyManagement>` per centralizzare le informazioni riguardanti le versioni degli artefatti. In questo modo non si ha la necessità di ripetere queste informazioni in ogni modulo. Quando un modulo richiede una dipendenza inclusa nel BOM, può specificarla senza dover dichiarare esplicitamente la versione, poiché questa viene ereditata dal BOM.



Quando diversi progetti ereditano da un genitore comune, il BOM può essere utilizzato per inserire tutte le informazioni sulle dipendenze in un file condiviso. Pertanto, il BOM è un file POM con una sezione `<dependencyManagement>` in cui vengono specificate le informazioni e le versioni degli artefatti.

N.B.: possiamo anche importare il BOM di un altro progetto Maven nel proprio progetto. Infatti, nel caso di progetti complessi, l'approccio dell'ereditarietà potrebbe non essere efficiente. Quindi, l'importazione del BOM da altri progetti offre una maggiore flessibilità, consentendo di integrare le versioni delle dipendenze necessarie in modo più dinamico.

6.5 Plug-In

I **plug-in** devono essere abilitati e configurati nella sezione `<plugins>` all'interno della sezione `<build>` del file `pom.xml`. Ogni plug-in richiede una configurazione specifica, definita nella sua sezione `<plugin>`. In particolare dobbiamo specificare le seguenti informazioni:

- Le coordinate GAV (GroupId, ArtifactId, Version) del plug-in, che consentono di identificare in modo univoco quel particolare plug-in.
- Uno o più goal del plug-in che desideriamo eseguire, per definire le operazioni specifiche che il plug-in deve compiere.
- La fase associata al goal, se questa non viene già associata di default dal plug-in.
- Alcune configurazioni aggiuntive, le quali variano a seconda delle specifiche del plug-in.

Ogni tipo di packaging in Maven ha alcuni plug-in abilitati per impostazione predefinita. Per determinare quali plug-in sono già attivati e configurati di default, è possibile consultare il file `pom.xml` effettivo del progetto. Se un plug-in è già abilitato di default, è comunque possibile specificare alcune configurazioni desiderate (che verranno combinate con le configurazioni predefinite). In questo modo possiamo personalizzare il comportamento del plug-in e adattarlo alle esigenze del progetto.

Per configurare un plug-in, è necessario specificare uno o più goal da eseguire e la fase a cui deve essere associato (se il plug-in non associa automaticamente il goal ad una fase predefinita). Questa configurazione avviene nella sezione `<executions>` di quello specifico plug-in nel file `pom.xml` del progetto.

All'interno di questa sezione `<executions>` possiamo definire diversi elementi `<execution>`, ognuno dei quali specifica il goal da eseguire in una determinata fase.

Inoltre, ogni elemento `<execution>` comprende una sezione `configuration` che contiene le specifiche per quella determinata esecuzione.

N.B.: lo stesso goal può essere eseguito più volte, anche nella stessa fase, con configurazioni diverse.

N.B.: all'esterno della sezione `<executions>` è possibile specificare un blocco `<configuration>` contenente le configurazioni comuni a tutte le esecuzioni.

```

1 <plugin>
2   <groupId>...</groupId>
3   <artifactId>...</artifactId>
4   <version>...</version>
5   <executions>
6     <execution>
7       <id>...</id>
8       <phase>phase to bind the goal to</phase>
9       <goals>
10        <goal>plugin goal</goal>
11      </goals>
12      <configuration>
13        <!-- configuration for this execution only -->
14        ...
15      </configuration>
16    </execution>
17    <execution>
18      <!-- ... as above ... -->
19    </execution>
20  </executions>
21  <configuration>
22    <!-- configuration common to all executions -->
23    ...
24  </configuration>
25 </plugin>

```

Come possiamo osservare dal precedente frammento di codice, la sezione `<executions>` contiene uno o più elementi `<execution>`, ognuno dei quali specifica uno o più goal da eseguire in una fase specifica. All'interno di ciascun elemento `<execution>`, è possibile definire una sezione `<configuration>` per specificare delle configurazioni per quell'esecuzione.

La configurazione comune a tutte le esecuzioni può essere definita anche all'esterno della sezione `<executions>`.

**ES:**

Consideriamo il seguente esempio di configurazione ereditata dal Super POM per il plugin Maven Compiler. Il plugin è configurato per eseguire il goal `compile` durante la fase `compile` ed il goal `testCompile` durante la fase `test-compile`:

```
1 <plugin>
2   <artifactId>maven-compiler-plugin</artifactId>
3   <version>3.1</version>
4   <executions>
5     <execution>
6       <id>default-compile</id>
7       <phase>compile</phase>
8       <goals>
9         <goal>compile</goal>
10      </goals>
11    </execution>
12    <execution>
13      <id>default-testcompile</id>
14      <phase>test-compile</phase>
15      <goals>
16        <goal>testCompile</goal>
17      </goals>
18    </execution>
19  </executions>
20 </plugin>
```

Tuttavia, possiamo modificare la configurazione del plug-in Maven Compiler per specificare la versione del compilatore Java che vogliamo utilizzare in Maven. A tale scopo, dobbiamo includere la seguente configurazione nel blocco `<build>` del file `pom.xml`:

```
1 <build>
2   <plugins>
3     <plugin>
4       <artifactId>maven-compiler-plugin</artifactId>
5       <version>3.1</version>
6       <configuration>
7         <source>1.8</source>
8         <target>1.8</target>
9       </configuration>
10    </plugin>
11  </plugins>
12 </build>
```


In questo esempio, stiamo configurando le opzioni **source** e **target** del plugin **maven-compiler-plugin** (versione 3.1) per specificare la versione di Java desiderata. Questa configurazione assicura che il compilatore utilizzi la versione 1.8 sia per la compilazione che per il target del bytecode generato.

Il POM effettivo risultante dalla configurazione sarà il seguente (la configurazione di default del plugin Maven Compiler viene combinata con la nostra configurazione personalizzata):

```

1 | <plugin>
2 |   <artifactId>maven-compiler-plugin</artifactId>
3 |   <version>3.1</version>
4 |   <executions>
5 |     <execution>
6 |       <id>default-compile</id>
7 |       <phase>compile</phase>
8 |       <goals>
9 |         <goal>compile</goal>
10 |      </goals>
11 |      <configuration>
12 |        <source>1.8</source>
13 |        <target>1.8</target>
14 |      </configuration>
15 |    </execution>
16 |    <execution>
17 |      <id>default-testcompile</id>
18 |      <phase>test-compile</phase>
19 |      <goals>
20 |        <goal>testCompile</goal>
21 |      </goals>
22 |      <configuration>
23 |        <source>1.8</source>
24 |        <target>1.8</target>
25 |      </configuration>
26 |    </execution>
27 |  </executions>
28 |  <configuration>
29 |    <source>1.8</source>
30 |    <target>1.8</target>
31 |  </configuration>
32 |</plugin>

```



ES:

Esempio di configurazione Maven per generare un JAR sorgente utilizzando il plug-in Maven Source. In questo esempio, stiamo configurando il plug-in Maven Source per eseguire il goal **jar** durante la fase di **package**. Il JAR sorgente risultante verrà incluso tra gli artefatti della build quando si esegue il comando Maven per creare il pacchetto (ad esempio, **mvn package**):


```

1|<plugin>
2|  <groupId>org.apache.maven.plugins</groupId>
3|  <artifactId>maven-source-plugin</artifactId>
4|  <version>3.0.1</version>
5|  <executions>
6|    <execution>
7|      <id>attach-sources</id>
8|      <!-- the phase would not be required in this case.
9|           Goal jar of this plug-in already binds by default
10|          to the "package" phase -->
11|      <phase>package</phase>
12|      <goals>
13|        <goal>jar</goal>
14|      </goals>
15|    </execution>
16|  </executions>
17|</plugin>

```



ES:

Esempio di configurazione Maven per generare un JAR di Javadoc utilizzando il plug-in Maven Javadoc (crea un JAR contenente la documentazione JavaDoc del progetto). In questo esempio, stiamo configurando il plug-in Maven Javadoc per eseguire il goal `jar` durante la fase di `package`. Il JAR sorgente risultante verrà incluso tra gli artefatti della build quando si esegue il comando Maven per creare il pacchetto (ad esempio, `mvn package`):

```

1|<plugin>
2|  <groupId>org.apache.maven.plugins</groupId>
3|  <artifactId>maven-javadoc-plugin</artifactId>
4|  <version>3.0.1</version>
5|  <executions>
6|    <execution>
7|      <id>attach-javadocs</id>
8|      <goals>
9|        <goal>jar</goal>
10|      </goals>
11|    </execution>
12|  </executions>
13|</plugin>

```

Dopo l'esecuzione del comando `mvn package`, saranno generati i seguenti file nella directory `target` (rappresentano gli artefatti generati come output del processo di build Maven):

- `my-app-0.0.1-SNAPSHOT.jar` (JAR principale dell'applicazione).
- `my-app-0.0.1-SNAPSHOT-javadoc.jar` (JAR contenente la documentazione JavaDoc).



- `my-app-0.0.1-SNAPSHOT-sources.jar` (JAR contenente il codice sorgente).

Un plug-in specificato nel POM genitore viene abilitato in tutti i moduli dei progetti. Tuttavia, se desideriamo abilitare un plug-in soltanto in alcuni moduli, possiamo utilizzare la sezione `<pluginManagement>` del POM genitore. Questa sezione si trova all'interno della sezione `<build>`, ma al di fuori della sezione `<plugins>`.

In questo modo, possiamo configurare il plug-in una volta sola. Successivamente, possiamo abilitare il plug-in soltanto nei moduli che ne hanno bisogno, specificando la configurazione desiderata all'interno della sezione `<build><plugins>` del file `pom.xml` relativo al modulo in questione. Per abilitare il plug-in è sufficiente specificare il `groupId` e l'`artifactId` del plug-in. Maven si occuperà di unire le configurazioni ereditate dal POM genitore, quelle del POM corrente e quelle della sezione `<pluginManagement>` (del POM genitore e del POM corrente).



ES:

Esempio di configurazione dei plug-in Maven Javadoc e Java Source in un progetto multi-modulo. In questo esempio, il plug-in Maven Source viene configurato nel POM genitore e abilitato in tutti i moduli. Invece, il plug-in Maven Javadoc viene configurato nel POM padre e abilitato soltanto nei moduli che lo richiedono.

```

1 <build>
2   <pluginManagement>
3     <!-- NOT enabled in all modules -->
4     <plugins>
5       <plugin>
6         <groupId>org.apache.maven.plugins</groupId>
7         <artifactId>maven-javadoc-plugin</artifactId>
8         <version>3.0.1</version>
9         <!-- executions as we saw in previous example -->
10        </plugin>
11      </plugins>
12    </pluginManagement>
13    <!-- enabled in all modules -->
14    <plugins>
15      <plugin>
16        <groupId>org.apache.maven.plugins</groupId>
17        <artifactId>maven-source-plugin</artifactId>
18        <version>3.0.1</version>
19        <!-- executions as we saw in previous example -->
20      </plugin>
21    </plugins>
22  </build>

```

Nel modulo figlio, la configurazione viene ereditata dal POM genitore tramite la sezione `<pluginManagement>`. In questo caso, il modulo abilita sia il plug-in Maven Source (abilitato in tutti i moduli) che il plug-in Maven Ja-

vadoc (richiesto esplicitamente):

```

1| <parent>
2|   <groupId>com.mycompany.example</groupId>
3|   <artifactId>example-project-parent</artifactId>
4|   <version>0.0.1-SNAPSHOT</version>
5| </parent>
6| <artifactId>example-project-core</artifactId>
7| <build>
8|   <plugins>
9|     <plugin>
10|       <!-- Inherited by parent's <pluginManagement> -->
11|       <groupId>org.apache.maven.plugins</groupId>
12|       <artifactId>maven-javadoc-plugin</artifactId>
13|     </plugin>
14|   </plugins>
15| </build>

```

Forzare le versioni dei plug-in è una pratica utile per garantire la coerenza nelle build di Maven. Invece di affidarsi alle versioni predefinite basate sull'installazione corrente di Maven, è consigliabile specificare esplicitamente le versioni dei plug-in desiderati nella sezione `<pluginManagement>`.

Quando questi plugin vengono utilizzati nei vari moduli del progetto, Maven li applicherà con le versioni specificate nel `<pluginManagement>`.



ES:

Specifichiamo le versioni dei plug-in nel `<pluginManagement>`. In questo modo, saranno utilizzate le versioni specificate quando i rispettivi plug-in vengono eseguiti nei moduli del progetto:

```

1| <build>
2|   <pluginManagement>
3|     <plugins>
4|       <plugin>
5|         <artifactId>maven-compiler-plugin</artifactId>
6|         <version>3.8.0</version>
7|       </plugin>
8|       <plugin>
9|         <artifactId>maven-surefire-plugin</artifactId>
10|        <version>2.22.1</version>
11|      </plugin>
12|    </plugins>
13|  </pluginManagement>
14| </build>

```

Nella sezione `<pluginManagement>` possiamo anche specificare delle configurazioni per i plug-in che possono essere eseguite direttamente dalla riga di comando, senza essere vincolati alla compilazione.

**ES (PIT):**

Esempio di configurazione di un plug-in dalla sezione `<pluginManagement>`. In particolare, viene illustrato come eseguire uno specifico goal di un plug-in dalla riga di comando senza essere vincolati alla fase di compilazione. In questo caso, utilizziamo il plug-in `pitest-maven` per il Mutation Testing:

```

1 <build>
2   <pluginManagement>
3     <plugins>
4       <plugin>
5         <groupId>org.pitest</groupId>
6         <artifactId>pitest-maven</artifactId>
7         <version>1.4.10</version>
8         <configuration>
9           <targetClasses>
10            <!-- excludes testing.example.bank.app.Main -->
11            <param>testing.example.bank.Bank *</param>
12          </targetClasses>
13          <targetTests>
14            <param>testing.example.bank.*</param>
15          </targetTests>
16          <mutators>
17            <mutator>DEFAULTS</mutator>
18            <mutator>NON_VOID_METHOD_CALLS</mutator>
19          </mutators>
20          <mutationThreshold>90</mutationThreshold>
21        </configuration>
22      </plugin>
23    </plugins>
24  </pluginManagement>
25 </build>

```

In questo esempio, la configurazione del plug-in `pitest-maven` viene inserita nella sezione `<pluginManagement>`. In particolare, non vengono definite esplicitamente esecuzioni né vincoli ad una fase specifica del ciclo di vita. Quando desideriamo eseguire il goal dalla riga di comando, è possibile utilizzare il comando `mvn org.pitest:pitest-maven:mutationCoverage`. Questo comando attiverà l'esecuzione del goal `mutationCoverage` del plug-in `pitest-maven`, utilizzando la configurazione specificata nella sezione `<pluginManagement>`.

**ES (JaCoCo):**

Per definire una proprietà `argLine` che possa essere passata come argomento all'applicazione da testare, è possibile utilizzare il plug-in Maven JaCoCo. Nel seguente esempio, definiamo una configurazione per di generare automaticamente una proprietà `argLine` che sarà letta dal plug-in Maven Surefire durante l'esecuzione dei test. In questo modo, i test verranno eseguiti attraverso JaCoCo:

```

1 | <build>
2 |   <pluginManagement>
3 |     <plugins>
4 |       <plugin>
5 |         <groupId>org.jacoco</groupId>
6 |         <artifactId>jacoco-maven-plugin</artifactId>
7 |         <version>0.8.5</version>
8 |         <executions>
9 |           <execution>
10 |            <goals>
11 |              <!-- binds by default to "initialize" phase
12 |                ↪ -->
13 |              <goal>prepare-agent</goal>
14 |            </goals>
15 |          </execution>
16 |        </executions>
17 |      </plugin>
18 |    </plugins>
19 |  </pluginManagement>
20 </build>

```

Successivamente, sarà possibile generare un report XML e HTML sulla Code Coverage con il comando `mvn jacoco:report`. I risultati del report saranno disponibili nella directory `target/site/jacoco`.

6.6 Profili Maven

I **profili Maven (Maven Profiles)** offrono la possibilità di attivare proprietà, plugin, dipendenze o moduli **soltanto in circostanze specifiche**. Ogni profilo viene definito all'interno dell'elemento `profile` contenuto nella sezione `<profiles>` del file `pom.xml`. Ad ogni profilo viene associato un identificatore (`id`). In questo modo, i profili **possono essere attivati dalla linea di comando mediante l'opzione `-P<id>`**. Inoltre, i profili possono avere attivazioni condizionali.



ES:

Esempio per attivare versioni specifiche del compilatore Java (dalla versione 6 alla versione 8 esclusa):

```

1 | <activation>
2 |   <jdk>[1.6,1.8)</jdk>
3 | </activation>

```

Esempio per attivare Sistemi Operativi specifici (MacOS):

```

1 | <activation>
2 |   <os>
3 |     <family>mac</family>
4 |   </os>
5 | </activation>

```

**ES:**

JaCoCo, essendo un strumento di Code Coverage, può rallentare l'esecuzione dei test. Per evitare l'overhead in situazioni in cui la Code Coverage non è necessaria, possiamo configurare il plug-in Jacoco come parte di uno specifico profilo Maven (ad esempio con l'identificatore "jacoco") che non viene attivato di default.

Quindi, possiamo attivare questo profilo soltanto quando ne abbiamo bisogno, utilizzando l'opzione della riga di comando `-Pjacoco`.

I profili dovrebbero essere impiegati per estendere la compilazione in circostanze specifiche e non per fornire configurazioni essenziali alla compilazione stessa. Ad esempio, se la compilazione ha successo con il comando `mvn clean verify -Pmyprofile`, ma fallisce senza l'attivazione del profilo, allora si potrebbe avere un problema. Pertanto, dobbiamo garantire che la compilazione abbia successo anche quando nessun profilo è attivato.

N.B: i profili non dovrebbero essere un requisito indispensabile per superare la compilazione.



6.7 Maven e JUnit 5

Con Maven possiamo configurare l'utilizzo di JUnit 5 insieme a JUnit 4 (ad esempio, per prevedere il supporto di vecchi progetti). In particolare, dobbiamo aggiungere le seguenti dipendenze al file `pom.xml`:

```

1 <dependency>
2   <groupId>org.junit.jupiter</groupId>
3   <artifactId>junit-jupiter</artifactId>
4   <version>5.7.0</version>
5   <scope>test</scope>
6 </dependency>
7 <dependency>
8   <groupId>org.junit.vintage</groupId>
9   <artifactId>junit-vintage-engine</artifactId>
10  <version>5.7.0</version>
11  <scope>test</scope>
12 </dependency>
```

Inoltre, dobbiamo assicurarci di utilizzare una versione recente del plug-in Surefire. Questo può essere configurato nella sezione `<plugins>` del file `pom.xml`:

```

1 <plugin>
2   <artifactId>maven-surefire-plugin</artifactId>
3   <version>2.22.1</version>
4 </plugin>
```

Se si sta utilizzando il plugin PIT per il Mutation Testing e si desidera integrare il supporto per JUnit 5, è necessario aggiungere la dipendenza specifica nella sezione `<pluginManagement>`:

```

1 <plugin>
2   <groupId>org.pitest</groupId>
3   <artifactId>pitest-maven</artifactId>
```

```
4      <version>1.5.2</version>
5      <dependencies>
6          <dependency>
7              <groupId>org.pitest</groupId>
8              <artifactId>pitest-junit5-plugin</artifactId>
9              <version>0.12</version>
10         </dependency>
11     </dependencies>
12 </plugin>
```

Con queste configurazioni, Maven sarà pronto per eseguire i test con JUnit 5 e integrare PIT per il Mutation Testing.

Capitolo 7

MOCKING

Quando sviluppiamo gli Unit Test per verificare il comportamento isolato di un singolo componente, dobbiamo gestire le dipendenze a cui la classe in esame fa riferimento.

Questi **collaboratori** possono essere componenti interni all'applicazione o librerie di terze parti (ad esempio, un database, un servizio remoto o altri moduli interni).

La gestione delle dipendenze nei test può diventare problematica, specialmente quando dobbiamo interagire con componenti reali (come un database o un servizio remoto). Questo può portare a test complessi e lenti, violando il principio fondamentale degli Unit Test: testare una singola unità in modo isolato senza coinvolgere dipendenze esterne.

Inoltre, potremmo trovarci a dover affrontare situazioni in cui le implementazioni effettive delle dipendenze non sono ancora disponibili (ad esempio, nel caso in cui i collaboratori devono essere sviluppati da altri componenti del team e non sono ancora pronti). Questo può causare blocchi nel processo di testing.

Prima di scrivere gli Unit Test, dobbiamo alcune premesse chiave riguardanti le dipendenze del SUT:

- Si presume che le dipendenze della SUT siano implementate correttamente, anche se non necessariamente da chi sta scrivendo i test. Questo vale sia per le dipendenze interne che per quelle esterne.
- Le dipendenze devono svolgere correttamente il loro compito. Quando si tratta di dipendenze interne, si presume che siano già state testate o che verranno testate nel corso dello sviluppo. Per quanto riguarda le dipendenze esterne, si confida nella loro implementazione e si considera la loro funzionalità come un dato di fatto.

In generale, durante la scrittura degli Unit Test, si assume che l'effettiva implementazione delle dipendenze non sia di nostra competenza.

Quando sviluppiamo gli Unit Test, dobbiamo rimuovere le attuali implementazioni delle dipendenze (assumendo che funzionino correttamente). Questa separazione può essere realizzata attraverso l'utilizzo di **mock**, ovvero implementazioni semplificate delle dipendenze. Invece di utilizzare le dipendenze reali durante i test, le sostituiamo con delle simulazioni (cioè, i mock rappresentano false implementazioni dei collaboratori dell'unità che vogliamo testare).

Questa pratica ci consente di concentrarci esclusivamente sul SUT, senza preoccuparci delle eventuali complessità o del funzionamento effettivo delle dipendenze. Pertanto, possiamo continuare a lavorare sull'implementazione del codice anche nel caso in cui le dipendenze non sono ancora state completamente implementate oppure sono sviluppate

da membri diversi del team. Inoltre, non siamo vincolati da dipendenze reali durante l'esecuzione dei test (come database o servizi remoti).

Poiché dobbiamo assumere che le dipendenze svolgano correttamente il loro compito, allora configuriamo i mock in modo tale che agiscano secondo le nostre esigenze specifiche per testare un particolare scenario del nostro SUT (cioè, ricreiamo lo scenario di test).

Questo approccio ci consente di mantenere l'efficienza richiesta dagli Unit Test. Infatti, con questa soluzione siamo in grado di scrivere i test in modo semplice e rapido (ad esempio, possiamo sostituire un database reale o un servizio che richiede una connessione ad Internet con un mock).

L'approccio del **Mocking** funziona in modo efficace solo quando i nostri componenti seguono buone pratiche di modularità del codice. Questo implica che i nostri componenti:

- Utilizzino le dipendenze attraverso astrazioni come interfacce o classi astratte.
- Non istanzino direttamente le dipendenze, ma le ricevano nel costruttore (tramite *dependency injection*).

N.B: la metodologia TDD ci costringe a scrivere codice modulare e l'utilizzo del Mocking ci consente di isolare le dipendenze, concentrandoci sul SUT.

Uno **stub** rappresenta una risposta *predefinita* che viene fornita da una dipendenza ed utilizzata in uno specifico contesto di test. Una volta creato il mock della dipendenza, viene effettuato lo **Stubbing** dell'implementazione di un metodo fornito da tale dipendenza. Questo metodo verrà chiamato internamente dal SUT. In questo modo, vengono creati dei metodi di un mock per fornire risposte predefinite durante l'esecuzione dei test.



ES:

Ipotizziamo che il SUT interroghi un database utilizzando il suo metodo `getAllElements()`. Vogliamo testare il comportamento del SUT quando il database contiene un singolo elemento. A tale scopo, effettuiamo lo Stubbing del metodo `getAllElements()` fornito dall'API del database, in modo tale che restituisca una lista contenente un singolo elemento. Successivamente, verifichiamo che il nostro SUT si comporti come ci aspettiamo (seguendo i principi del TDD).

```
1 // Abstract interface of dependency.
2 public interface Database {
3     List<String> getAllElements();
4 }
5
6 public class MyComponent {
7     private Database database;
8
9     // Pass the dependency to constructor (dependency injection).
10    public MyComponent(Database database) {
11        this.database = database;
12    }
```

```

13 | public Object myMethod() {
14 |     Object result = null;
15 |     List<String> list = database.getAllElements();
16 |     // Do something with list and prepare the result.
17 |     return result;
18 | }
19 |

```

Nella classe di test eseguiamo manualmente il Mocking e lo Stubbing dell'interfaccia `Database` per il metodo `getAllElements()`.

N.B.: sebbene sia possibile asserire manualmente il risultato, questo approccio può richiedere molto tempo e può comportare la scrittura di test meno leggibili (successivamente introdurremo degli strumenti per fare queste operazioni in modo semplice e pulito).

```

1 | public class MyComponentTest {
2 |     @Test
3 |     public void testMyMethodWhenDBReturnsOneElement() {
4 |         // Manual Mocking and Stubbing.
5 |         Database database = new Database() {
6 |             @Override
7 |             public List<String> getAllElements() {
8 |                 List<String> list = new ArrayList<>();
9 |                 list.add("pippo");
10 |                 return list;
11 |             }
12 |         };
13 |
14 |         MyComponent myComponent = new MyComponent(database);
15 |         Object result = myComponent.myMethod();
16 |
17 |         // Do assertions on the result
18 |     }
19 | }

```

Finora, abbiamo focalizzato la scrittura dei test sul codice di produzione che restituisce un valore (ad esempio, un metodo con un tipo di ritorno) oppure che modifica lo stato delle istanze del SUT. Questi test si concentrano sulla verifica dello **stato** del codice, assicurandoci che il valore restituito da un metodo in fase di test sia quello previsto oppure che, dopo aver richiamato un metodo, lo stato di un oggetto sia cambiato secondo le aspettative (oppure una combinazione di entrambi).

Tuttavia, quando si scrivono test per un SUT che dipende da collaboratori esterni, diventa necessario verificare anche l'*interazione* del nostro codice con tali collaboratori. In questi casi, i test sono progettati per verificare che la nostra SUT chiami i metodi specifici dei collaboratori un numero prestabilito di volte e che gli vengano passati determinati argomenti durante queste chiamate.

Verificare le interazioni con i mock è necessario quando vogliamo testare come interagisce il nostro SUT con le sue dipendenze.

N.B.: la verifica manuale delle interazioni è possibile, ma risulta notevolmente complessa. I test risultano meno leggibili e questa pratica è in contrasto con l'approccio del TDD (successivamente introdurremo degli strumenti per fare queste operazioni).

**ES:**

Immaginiamo che il nostro componente processi gli elementi restituiti da un database e successivamente chiami un altro metodo sul database, come `update(args)`.

```

1 | public interface Database {
2 |     List<String> getAllElements();
3 |     void update(String arg );
4 | }

```

In questo contesto, vogliamo verificare che il metodo `update(args)` del collaboratore `Database` venga chiamato esattamente una volta con specifici argomenti (oppure, che non venga mai chiamato in circostanze specifiche):

```

1 | public class MyComponentTest {
2 |     private static final class MockedDatabase implements Database {
3 |         String arg = null;
4 |
5 |         @Override
6 |         public List<String> getAllElements() {
7 |             ArrayList<String> list = new ArrayList<String>();
8 |             list.add("foo");
9 |             return list;
10 |        }
11 |
12 |        @Override
13 |        public void update(String arg) {
14 |            // Keep track of the passed argument.
15 |            this.arg = arg;
16 |        }
17 |    }
18 |
19 |    @Test
20 |    public void testMyMethodCallsUpdateWithExpectedArg() {
21 |        MockedDatabase database = new MockedDatabase();
22 |        MyComponent myComponent = new MyComponent(database);
23 |        myComponent.myMethod();
24 |
25 |        // Check that the method update has been called.
26 |        assertEquals("bar", database.arg);
27 |
28 |        // Or that it has never been called.
29 |        assertNull(database.arg);
30 |    }
31 | }

```

Come abbiamo osservato dai precedenti esempi, effettuare manualmente le operazioni di Mocking, Stubbing e verifica delle interazioni risulta essere molto complesso e rende i test difficili da leggere.

Per semplificare questo processo e garantire test più chiari, possiamo utilizzare **Mockito**, uno dei framework più utilizzati per il mocking in Java.

**ES:**

Riscriviamo i precedenti esempi con l'utilizzo di Mockito. In particolare, possiamo utilizzare Mockito per creare un mock e definire un stub in modo semplice e leggibile:

```

1 | import static org.mockito.Mockito.*;
2 |
3 | @Test
4 | public void testMyMethodWhenDBReturnsOneElement() {
5 |     // Creating a mock for the Database interface.
6 |     Database database = mock(Database.class);
7 |
8 |     // Creating the MyComponent instance using the mock.
9 |     MyComponent myComponent = new MyComponent(database);
10 |
11 |    // Stubbing the getAllElements() method to make it return a list
12 |    //    ↪ with an element when called.
13 |    List<String> list = new ArrayList<String>();
14 |    list.add("pippo");
15 |    when(database.getAllElements()).thenReturn(list);
16 |
17 |    // Call to the method to be tested.
18 |    Object result = myComponent.myMethod();
19 |
20 |    // Do assertions on the result.
21 | }
```

Analogamente, possiamo utilizzare Mockito per verificare le interazioni tra il SUT ed i suoi collaboratori:

```

1 | @Test
2 | public void testMyMethodWhenDBReturnsOneElementWithMock() {
3 |     Database database = mock(Database.class);
4 |     MyComponent mycomponent = new MyComponent(database);
5 |
6 |     ArrayList<String> list = new ArrayList<String>();
7 |     list.add("foo");
8 |     when(database.getAllElements()).thenReturn(list);
9 |
10 |    Object result = mycomponent.myMethod();
11 |
12 |    // Verify that database.update("foo") is called once.
13 |    verify(database, times(1)).update("foo");
14 |
15 |    // Do assertions on the result.
16 | }
```

Notiamo che non è possibile effettuare lo Stubbing di metodi void utilizzando la sintassi classica di Mockito, ovvero:

```
1 | when(o.m()).thenThrow( ... exception ... )
```

Al contrario, è necessario adottare la seguente sintassi alternativa:

```
1 | doThrow( ... exception ... ).when(o).m()
```

La sintassi alternativa può essere utilizzata anche per i metodi non `void`, ma in questo caso è preferibile evitarla poiché comporta la perdita del controllo statico dei tipi.

```
1|doReturn( ... ).when(o).m()
```

Dobbiamo evitare di creare mock per dipendenze che non si controllano direttamente (come le librerie di terze parti). In alternativa, dobbiamo preferire la creazione di *wrapper* o *facade* per le librerie di terze parti. In particolare, dobbiamo creare un'interfaccia per il wrapper ed un'implementazione concreta. Quindi, il codice dovrebbe utilizzare solo il wrapper, senza chiamare direttamente il codice delle librerie di terze parti. Questa soluzione semplifica il processo di Mocking per i test e facilita la sostituzione con un'implementazione diversa delle librerie di terze parti (se necessario).

Inoltre, non dobbiamo effettuare il Mocking dei tipi che non si controllano direttamente. Ad esempio, se il SUT richiede una lista, allora nei test dovremmo utilizzare effettivamente una lista, evitando di simulare questo comportamento.

Fino a questo momento, abbiamo sempre effettuato lo Stubbing di un metodo su un mock restituendo un valore oppure lanciando un'eccezione. Se desideriamo restituire valori diversi per diverse invocazioni, possiamo ricorrere al concetto di **Subsequent Stubbing** o agli **Argument Matchers**.

Tuttavia, se abbiamo bisogno che il metodo stub restituisca qualcosa calcolato a runtime, possiamo adottare il concetto di **Stubbing with Answers**.

Come abbiamo già osservato, nel contesto degli Unit Test, dobbiamo eliminare le implementazioni reali o le dipendenze sostituendole con implementazioni che ne simulano il comportamento. Questi surrogati sono comunemente noti come **test doubles** (ad esempio, il mock è un test double).

In alternativa al Mocking, è possibile *spiare* l'oggetto di una classe effettiva. Un oggetto **spia** (**spy**) si comporta come l'originale, ma i metodi possono essere stubbati esplicitamente. Se disponiamo già di un oggetto reale, anziché creare un mock (cioè, una simulazione), possiamo creare un'istanza reale e successivamente “spiare” tale oggetto. Questo approccio risulta utile per verificare le interazioni del SUT con l'oggetto spia.

I **fakes** rappresentano un altro tipo di test double. Si trattano di implementazioni effettive dei collaboratori, ma semplificate. Essi sono progettati per essere molto più semplici delle implementazioni reali e possono essere utilizzati per test in ambienti isolati (poiché consentono di sostituire una dipendenza o un collaboratore con un'implementazione più leggera). Ad esempio, un database in-memory può essere considerato un fake.

N.B: anche i fakes devono essere sottoposti a test per garantirne la correttezza.



ES (PayRoll):

Immaginiamo di dover implementare un sistema PayRoll in cui la classe `EmployeeManager` definisce due dipendenze:

- **EmployeeRepository**: rappresenta un'interfaccia astratta dal database per salvare e recuperare i record dei dipendenti.
- **BankService**: rappresenta un'interfaccia astratta per l'accesso ad un servizio bancario remoto.

L'implementazione degli Unit Test per il sistema di PayRoll richiederebbe l'utilizzo di un vero database e l'accesso ad un vero servizio Internet della banca. Tuttavia, queste operazioni richiedono un tempo significativo per essere eseguite. Inoltre, richiedono transazioni reali su un vero conto bancario, creando delle complicazioni per la fase di test.

Per affrontare queste sfide, ricorriamo all'uso di test doubles per simulare le dipendenze con i mock ed isolare il sistema durante i test.

Il **pattern Repository** è un modo efficace per implementare il **Data Access Layer (DAL)** in un'architettura a strati. Questo approccio mira a migliorare la modularità del codice base. Nel contesto di questo esempio, il Data Access Layer implementa la logica di accesso al database. Pertanto, fornisce dei metodi per accedere e modificare i record del database.

Quando affrontiamo la creazione della classe **EmployeeManager**, abbiamo diverse opzioni per la definizione delle interfacce Java coinvolte. Potremmo decidere di creare le interfacce da zero quando necessario oppure di fare affidamento sulle interfacce Java già fornite da altri membri del team. Dobbiamo però ricordare che non abbiamo bisogno delle implementazioni reali delle dipendenze per gli Unit Test.

Tuttavia, è fondamentale sottolineare che durante la scrittura dei test dobbiamo assumere che le dipendenze fornite siano corrette.

Capitolo 8

VERSION CONTROL SYSTEM

Un **Version Control System (VCS)** è uno strumento che consente di tenere traccia della *storia* di un insieme di file. Ogni elemento della storia rappresenta uno “snapshot” dei file (cioè, uno stato specifico del progetto). Grazie a questo strumento, gli utenti possono spostarsi avanti e indietro nella storia, consentendogli di esplorare tale storia e ripristinare facilmente ad una versione precedente.

La storia completa viene archiviata in un **repository**, ovvero un deposito che funge da cronologia del progetto. Un VCS offre un modo efficace per tenere traccia di tutti gli esperimenti e le modifiche apportate al software nel corso del tempo. Questa funzionalità diventa preziosa quando qualcosa va storto, consentendo agli utenti di tornare rapidamente ad una versione funzionante del progetto.

Un approccio comune alla gestione delle versioni consiste nel creare manualmente una copia dell'intera directory del progetto con un nuovo nome ogni volta che vengono apportate modifiche significative. Tuttavia, questo metodo presenta diversi svantaggi:

- Creare copie complete della directory ogni volta può portare a uno spreco significativo di spazio su disco, specialmente quando il progetto cresce in dimensioni.
- Non è facile tornare indietro nel tempo o esplorare versioni precedenti del progetto. La gestione manuale delle versioni rende complesso il recupero di una specifica versione precedente.
- In una copia manuale, non è immediatamente evidente quali file sono stati cambiati o quali modifiche sono state apportate in una versione specifica. Questo rende difficile comprendere le differenze tra le diverse versioni e può rallentare il processo di debugging o di verifica delle modifiche.
- In un contesto di sviluppo collaborativo, la gestione manuale delle versioni può causare confusione tra i membri del team.

Per superare queste limitazioni, è necessario adottare uno strumento di VCS dedicato. Tra i più comuni troviamo CVS, SVN, Mercurial e **Git**.

La storia di un repository in un VCS è costituita da **commit**. Ogni commit rappresenta una modifica salvata, che può coinvolgere diversi file. Essenzialmente, un commit è uno snapshot dell'intero progetto in un determinato momento della storia. Quindi, un commit consente di tracciare l'evoluzione del codice e mantenere una cronologia delle modifiche effettuate nel tempo.

N.B: un commit deve essere effettuato ogni qualvolta riteniamo che le modifiche apportate siano significative e degne di essere salvate nella storia del progetto.

Un commit può essere etichettato mediante l'uso di un **tag**. L'etichettatura dei commit con tag specifici è un modo efficace per marcare punti di rilascio significativi nella cronologia del repository. Questa pratica semplifica il processo di ritorno ad una versione specifica del software (ad esempio, quando rilasciamo la nuova versione 1.3.2 del software, possiamo etichettare il commit associato alla release con il tag `rel_1.3.2`).

N.B.: l'assegnazione di un tag ad un commit viene tipicamente effettuato ogni volta che si rilascia una nuova versione del software. Perciò, un tag rappresenta uno snapshot particolarmente rilevante nella storia del software.

Un **branch** rappresenta un flusso di sviluppo separato all'interno del repository. Questo concetto consente di lavorare su diverse funzionalità o miglioramenti in un contesto isolato, prima di integrarle con il flusso principale (cioè, i branch consentono di sviluppare nuove funzionalità o risolvere problemi in un ambiente separato, evitando potenziali interferenze con il flusso principale del progetto).

La capacità di *unire* (**merge**) i branch consente di integrare le modifiche apportate in uno specifico branch con il branch principale (o con altri branch), gestendo eventuali conflitti durante questo processo di unione.

La gestione dei branch in Git è considerata una delle caratteristiche più potenti e distintive rispetto ad altri VCS. La facilità ed efficienza con cui è possibile creare, spostarsi tra i branch e unire le modifiche contribuisce in modo significativo alla flessibilità e alla gestione dello sviluppo del software.

N.B.: il branch principale di default è chiamato **master** (o **main**).

Nei **VCS centralizzati**, esiste un singolo repository centralizzato ospitato su un server. Ogni commit effettuato localmente viene immediatamente inviato al repository remoto. Tuttavia, questa architettura presenta alcune limitazioni significative. Innanzitutto, dobbiamo considerare che ogni sviluppatore può accedere soltanto alla versione locale corrente del software. Pertanto, se lo sviluppatore necessita di accedere ad uno specifico commit del software, deve necessariamente contattare il server. Inoltre, in caso di indisponibilità del server o mancanza di connessione, l'accesso alla cronologia diventa impossibile. Oltre a questo, l'assenza di connessione impedisce di eseguire commit o aggiornamenti delle versioni.

Quindi, questa struttura centralizzata presenta una notevole dipendenza dal server e dalla connessione.

Al contrario, i **VCS distribuiti** offrono soluzioni più resilienti e flessibili. Infatti, in un VCS distribuito può esistere un repository centrale su un server, ma la caratteristica distintiva è la presenza di numerosi repository clonati su server diversi. Di conseguenza, ogni sviluppatore possiede un clone completo dell'intero repository.

In questo contesto, ogni commit viene effettuato sulla copia locale del repository, consentendo agli sviluppatori di accedere a qualsiasi versione della cronologia attraverso il proprio repository locale.

Uno degli aspetti distintivi dei VCS distribuiti è la possibilità di lavorare offline. Infatti, ogni sviluppatore ha accesso alla storia completa del progetto sul proprio computer locale, riducendo significativamente il rischio di perdere la cronologia. Di conseguenza, la storia diventa implicitamente ridondante.

Inoltre, i VCS distribuiti consentono la connessione tra il proprio repository locale e qualsiasi altro clone dello stesso repository, noto come repository **remoto**. Questo assicura che la storia sia identica in ogni clone del repository, definendo così i VCS distribuiti come una forma di VCS peer-to-peer.

L'aggiornamento della copia locale con le modifiche presenti nel repository remoto avviene tramite l'operazione di **fetch**, mentre l'invio dei commit locali al repository remoto è gestito attraverso il **push**.

La decentralizzazione rappresenta un approccio vantaggioso nei VCS. Infatti, consente di effettuare commit frequenti, senza timore di compromettere l'integrità del progetto. La facilità con cui è possibile tornare indietro nel tempo offre una flessibilità notevole agli sviluppatori. Inoltre, in un VCS distribuito non esiste un singolo punto di guasto, semplificando il processo di backup.

Questo rappresenta un netto contrasto con i VCS centralizzati, dove l'approccio è esattamente opposto.

Osserviamo che i VCS distribuiti non sono ottimizzati per la gestione dei file binari, specialmente se questi cambiano frequentemente. Infatti, la cronologia dei file binari richiede un notevole spazio di archiviazione per mantenere tutte le versioni. Pertanto, la gestione dei file binari in un contesto distribuito può diventare più complessa a causa della copia completa del repository in ogni clone locale.

Al contrario, Nei VCS centralizzati questo può non essere un problema significativo, poiché sul computer locale è presente soltanto l'ultima versione e solo il server richiede una grande capacità di archiviazione.

Tipicamente, In un VCS centralizzato si adotta una numerazione crescente per versionare i commit, poiché esiste un unico repository centrale. Tuttavia, questa pratica non è applicabile in un VCS distribuito, dato che possono esistere diversi cloni di un repository e ogni sviluppatore può lavorare offline. Di conseguenza, la denominazione dei commit deve seguire un approccio diverso. Ad esempio, in Git i commit vengono identificati in modo univoco utilizzando l'hash (SHA-1) del contenuto del commit. Questo approccio consente di identificare in modo univoco ogni commit in ogni clone del repository, garantendo coerenza nella gestione della cronologia.

8.1 Git

Git rappresenta il VCS distribuito più ampiamente utilizzato. Creato nel 2005 da Linus Torvalds per facilitare lo sviluppo collaborativo del kernel Linux, Git ha conquistato popolarità non solo tra gli sviluppatori open source, ma anche in numerosi contesti aziendali. Infatti, Git offre una solida base per la collaborazione, consentendo a team di sviluppatori di lavorare in modo efficace su progetti complessi.

Git può essere impiegato sia attraverso la riga di comando che integrato direttamente negli ambienti di sviluppo mediante strumenti specifici. Nell'IDE Eclipse è disponibile il plug-in EGit, tipicamente già preinstallato nelle distribuzioni di Eclipse.

Per iniziare ad esplorare Git, procediamo con la creazione di un nuovo repository locale. Seguiamo questi passaggi:

1. Creare una nuova directory:

```
1|mkdir myrepo
```

2. Accedere alla nuova directory appena creata:

```
1|cd myrepo
```

3. Inizializzare il repository Git nella directory:

```
1|git init
```

Con questi comandi, viene creata la directory **myrepo**, nota come **Working Directory** (o **Working Tree**), che contiene la versione attuale dei file. Al suo interno, viene creata la sottodirectory **.git** in cui viene memorizzato il repository.

Per configurare Git, è importante impostare globalmente alcune proprietà relative al proprio utente. Ecco come farlo:

```
1|git config --global user.name "your name"
2|git config --global user.email "your email"
```

In questo modo, ogni commit effettuato con Git includerà queste informazioni.

N.B.: le configurazioni globali vengono memorizzate nel file **.gitconfig** localizzato nella home directory.

N.B.: è possibile anche definire alcune configurazioni specifiche per un singolo repository locale. Tali configurazioni locali sono memorizzate nel file **config** localizzato all'interno della sottodirectory **.git** del repository stesso (cioè, al percorso **.git/config**).

La gestione dell'*end-of-line* in Git è fondamentale, soprattutto quando si lavora su Sistemi Operativi diversi (potrebbero verificarsi problemi di inconsistenza quando si lavora in un team con membri che utilizzano Sistemi Operativi diversi). Per per garantire la coerenza nel versionamento dei file in Git, è necessario gestire correttamente l'*end-of-line* utilizzando i seguenti comandi:

- Su sistemi Windows:

```
1|git config core.autocrlf true
```

- Su sistemi Unix (Linux e Mac):

```
1|git config core.autocrlf input
```

La **Working Directory** (o **Working Tree**) in Git rappresenta una versione specifica appartenente alla storia del repository. In particolare, rappresenta la versione su cui attualmente si sta lavorando e che non è stata ancora sottoposta a commit.

Ogni file presente nella Working Directory può trovarsi in uno dei seguenti stati:

- **Untracked:** il file non è presente nel repository.
- **Tracked:** il file è presente nel repository e non è stato modificato.
- **Staged:** il file è stato modificato ed è pronto per essere incluso nel prossimo commit.
- **Dirty:** il file è stato modificato, ma le modifiche non sono ancora state incluse nell'**area di staging** (nota anche come **index**).

Questa architettura consente di poter annullare le modifiche non ancora sottoposte a commit in ogni momento. Il comando **git status** fornisce informazioni dettagliate sullo stato attuale dei file nella Working Directory. Questo comando è uno strumento utile per monitorare le modifiche apportate e per preparare i file da includere nel prossimo commit.

N.B.: possiamo eseguire il **checkout** di qualsiasi versione e la Working Directory verrà aggiornata di conseguenza.

Per effettuare un commit in Git, dobbiamo seguire un processo a due fasi:

1. Aggiungere file Dirty o Untracked all'area di staging, utilizzando uno dei seguenti comandi:

```
1|git add file_name # adds one or more specific files.
2|git add . # adds all Dirty or Untracked files.
```

Questo comando prepara i file per essere inclusi nel prossimo commit.

2. Eseguire il commit dell'area di staging utilizzando il seguente comando:

```
1|git commit -m "message describing the commit"
```

Questo comando crea un nuovo commit (snapshot) nella cronologia del repository, registrando le modifiche preparate nell'area di staging.

N.B.: non utilizzare mai e poi mai un messaggio vuoto.

N.B.: i comandi possono essere eseguiti da qualsiasi livello della Working Directory, poiché Git opera a livello di repository.

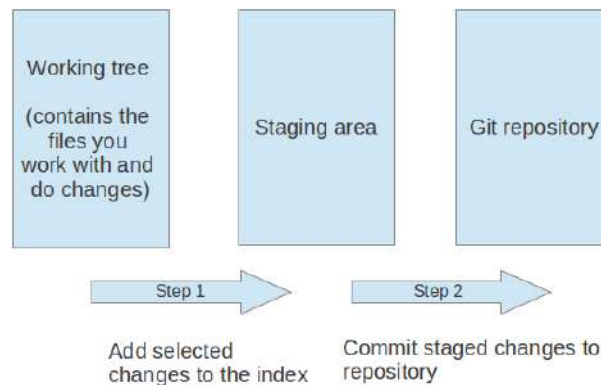


Figura 8.1: Fasi di un commit.

Git consente di fornire un messaggio per ogni commit. Dopo la prima riga del messaggio, è possibile aggiungere qualsiasi informazione aggiuntiva relativa alle modifiche apportate nel commit (se un commit è molto importante, è meglio scrivere un messaggio più dettagliato).

N.B.: nel caso in cui non si specifichi un messaggio tramite l'opzione `-m`, verrà aperto l'editor di sistema, consentendo di scrivere un messaggio di commit più dettagliato su quante righe si ritenga necessario.

N.B.: possiamo fare riferimento ad un commit usando un prefisso dell'hash, a condizione che tale prefisso sia univoco e non ambiguo.

Il file `.gitignore` può essere utilizzato per specificare i file e le directory che Git deve ignorare durante il versionamento. Ogni riga di questo file contiene un'espressione regolare per identificare i file da ignorare (i commenti iniziano con il simbolo `#`). Se i file ignorati vengono modificati o rimossi, Git non li considererà nelle operazioni di commit.



ES:

Esempio di file `.gitignore` per ignorare alcuni file tipici dei progetti Java che non devono essere considerati nel versionamento:

```

1 | # Backup files created by editors.
2 | *~
3 |
4 | # Target folder for Maven projects.
5 | /target

```

Il file `.gitignore` specifica che Git dovrebbe ignorare i file di backup generati dagli editor (cioè, quelli con suffisso `~`) e la cartella `target` tipica dei progetti Maven.

Git offre due interfacce grafiche per semplificare l'interazione con esso:

- **Git GUI:** consente di effettuare commit, clonare repository e altre operazioni attraverso un'interfaccia grafica intuitiva. Per avviare Git GUI, basta eseguire seguente il comando:

```
1 | git gui
```

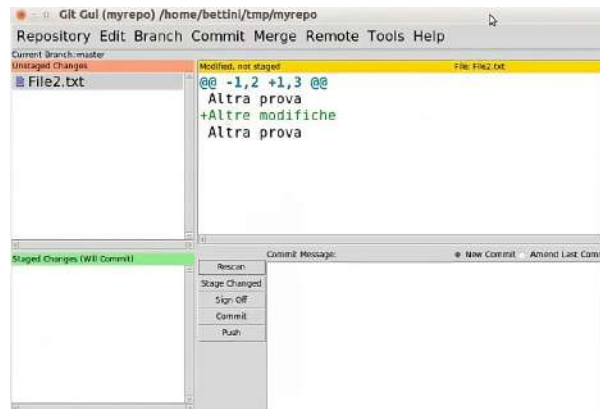


Figura 8.2: Interfaccia grafica Git GUI.

- **Gitk:** offre la possibilità di esaminare la cronologia del repository in modo grafico. Inoltre, consente di visualizzare le differenze tra commit. Per avviare Gitk, basta eseguire seguente il comando:

```
1 | gitk --all
```

N.B.: l'opzione `--all` consente di visualizzare tutti i branch del repository.

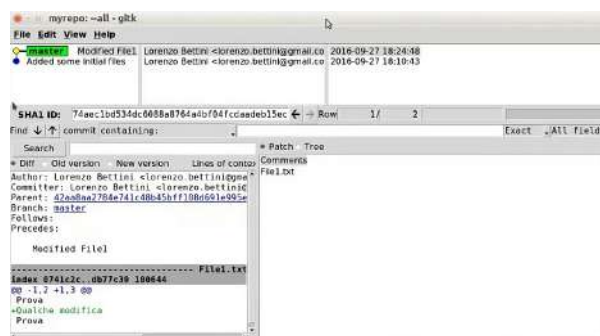


Figura 8.3: Interfaccia grafica Gitk.

La gestione dei branch in Git offre molta flessibilità. Ecco alcuni comandi utili:

- Creare un nuovo branch:
1|git branch <branch_name>
- Passare al branch creato:
1|git checkout <branch_name>
- Creare un nuovo branch e passare ad esso in un solo passaggio:
1|git checkout -b <branch_name>
- Mostrare tutti i branch:
1|git branch
- Rimuovere un branch:
1|git branch -d <branch_name>

Se il branch non è stato ancora unito al flusso principale, l'esecuzione di questo comando comporterà un errore. In tal caso, si può forzare la rimozione usando il seguente comando:

```
1|git branch -D <branch_name>
```

Il branching consente di sviluppare funzionalità in modo isolato e di integrare le modifiche in modo flessibile.



Figura 8.4: Visualizzazione grafica della divergenza tra il branch `experiments` ed il flusso principale `master`. Le funzionalità implementate sul branch non influenzeranno il flusso principale, finché non verranno uniti.

Per integrare le modifiche effettuate in un branch (ad esempio, dal branch `experiments`) in un altro branch (ad esempio, nel flusso principale `master`), possiamo utilizzare il comando di `merge` in Git. Per fare ciò, ci posizioniamo sul branch in cui vogliamo eseguire il merge (ad esempio, `master`) e lanciamo il seguente comando specificando il nome del branch da unire (ad esempio, `experiments`):

```
1|git merge <branch_name>
```

Se non ci sono conflitti tra i due branch, il merge avrà successo e verrà creato un nuovo commit per registrare l'unione.



Figura 8.5: Visualizzazione grafica per l'unione del branch `experiments` nel flusso principale `master`.

Nel caso in cui i due branch non abbiano *divergenze*, il merge verrà eseguito in modalità **Fast Forward**. Ciò significa che non verrà creato un commit separato per il merge, ma la storia dei commit del branch di destinazione (ad esempio, `master`) verrà semplicemente aggiornata con i commit del branch sorgente (ad esempio, `experiments`) senza creare un nuovo commit di merge.

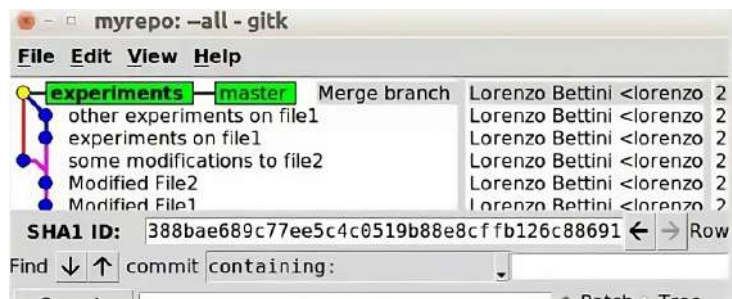


Figura 8.6: Visualizzazione grafica per l'unione in modalità Fast Forward del branch `experiments` nel flusso principale `master` (i due branch non divergono, perciò non viene creato un nuovo commit separato).

Se si desidera forzare la creazione di un commit separato per il merge anche in caso di Fast Forward, è possibile utilizzare l'opzione `--no-ff`:

```
1| git merge --no-ff <branch_name>
```

Se lo stesso file risulta essere modificato in commit appartenenti a due branch differenti e tali modifiche coinvolgono la stessa porzione di file, allora durante il processo di unione si verificherà un **conflitto**. In questo caso, il merge viene momentaneamente interrotto ed i file coinvolti nel conflitto vengono *annotati* con informazioni utili a risolvere tale conflitto, indicando cosa è presente nel branch corrente e cosa è presente nell'altro branch che si sta cercando di unire.

La risoluzione di un conflitto richiede un intervento manuale, rimuovendo le annotazioni e decidendo quale versione del codice mantenere. Successivamente, è necessario aggiungere all'area di staging i file che sono stati risolti ed infine eseguire un commit. Quest'ultimo passaggio finalizzerà l'unione precedentemente sospesa.

N.B: se nel commit di risoluzione del conflitto non viene specificato un messaggio, verrà proposto un messaggio predefinito che contiene informazioni sui file coinvolti nel conflitto, fornendo dettagli utili nella cronologia dei commit.

N.B: poiché Git inserisce annotazioni nelle porzioni in conflitto per agevolare la risoluzione, se i file in conflitto contengono codice sorgente (ad esempio, file Java), il codice potrebbe non compilarsi correttamente a causa delle annotazioni aggiunte da Git.

Il comando **reset** in Git consente di riportare un branch da un commit X ad un commit Y nella cronologia (possiamo anche specificare un commit da un altro branch come destinazione del reset). Il reset può essere eseguito in due modalità:

- **Soft** (o **Mixed**): questa opzione permette di mantenere le modifiche che sono state apportate fino al commit di destinazione specificato (Y), conservando i cambiamenti nell'area di staging.
- **Hard**: questa opzione consente di eseguire un reset completo, eliminando tutte le modifiche che sono state apportate fino al commit di destinazione specificato (Y). In altre parole, riporta l'area di staging e la copia di lavoro alle condizioni esatte del commit Y.

N.B.: se il commit X è in un punto successivo rispetto ad Y, tutti i commit da Y ad X verranno irrimediabilmente persi (a meno che non esista un altro branch che contenga il commit X).



ES:

Consideriamo il seguente esempio di reset per riportare il branch **master** a due commit precedenti in modalità **Hard**. In questo modo cancelleremo tutte le modifiche apportate nei commit successivi a quello selezionato.

N.B.: in questo caso non perdiamo la cronologia delle modifiche successive perché è presente un branch **remoto**. In questo modo, la cronologia viene conservata anche se effettuiamo un reset locale e possiamo recuperare eventuali modifiche perse tramite il branch remoto.

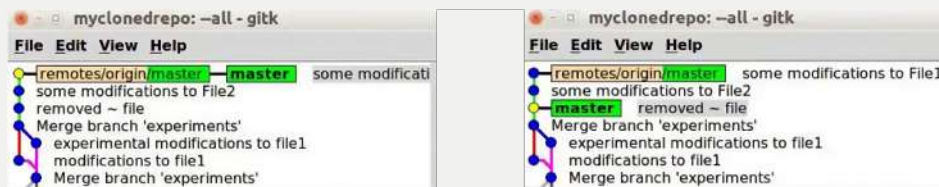


Figura 8.7: Hard reset del branch **master** a due commit precedenti (verranno perse tutte le modifiche che erano state fatte dal punto di ripristino in poi). La cronologia non viene persa grazie al branch remoto.

L'operazione di **Cherry Pick** in Git consente di selezionare un singolo commit da uno specifico branch ed applicarlo al branch corrente. Un esempio tipico di utilizzo è quando un commit in un altro branch corregge un bug e si desidera applicare quella correzione al branch attuale.

N.B.: se in futuro si effettua un merge con il branch specificato, il commit Cherry Pick non genererà conflitti.

L'operazione di **Stash** in Git è utile quando stiamo lavorando su un branch e dobbiamo spostarci temporaneamente su un altro branch senza perdere le modifiche che non sono ancora state memorizzate nella cronologia con un commit (ad esempio, perché non vogliamo fare un commit delle modifiche parziali). In pratica, le modifiche vengono salvate in un'area apposita chiamata **stash**. In questo modo, possiamo passare ad un altro branch e successivamente recuperare le modifiche messe nell'area stash per continuare il lavoro.

N.B.: possiamo gestire più stash contemporaneamente, ognuno con un nome diverso, per organizzare e recuperare le modifiche di branch differenti.

Un **bare repository** rappresenta un repository costituito soltanto dal repository Git stesso, escludendo la Working Directory. Questo tipo di repository è progettato per essere condiviso (ad esempio, su un server) e viene utilizzato per la collaborazione tra sviluppatori. Per creare un bare repository, eseguiamo i seguenti comandi:

```
1|mkdir myremoterepo
2|cd myremoterepo
3|git init --bare
```

In Git, è possibile collegare un repository locale a diversi repository **remoti**, consentendo la sincronizzazione delle modifiche tra il repository locale e quelli remoti. Questa connessione è essenziale per evitare la perdita di modifiche e facilitare la collaborazione con altri sviluppatori.

Ogni repository remoto viene identificato da un **nome** ed un **URI** (Uniform Resource Identifier). L'URI può assumere diverse forme, come un indirizzo HTTPS (ad esempio `https://github.com/...`), un indirizzo SSH (ad esempio, `git@github.com...`) oppure un percorso locale.

N.B.: un repository remoto non necessita fisicamente di trovarsi su un computer remoto, ma può anche essere un repository locale.

Per connettere un repository locale con un repository remoto, utilizziamo il seguente comando:

```
1|git remote add <name> <uri>
```

Per inizializzare un repository remoto, considerando che esso è attualmente vuoto (cioè, senza cronologia), possiamo effettuare un **push** dal repository locale al repository remoto. In particolare, il comando **push** invierà tutti i commit dal repository locale al repository remoto. Il seguente comando consente di effettuare il push dei commit di un singolo branch:

```
1|git push <remote_name> <branch_name>
```

Invece, se vogliamo effettuare il push di tutti i branch possiamo utilizzare il seguente comando con l'opzione `--all`:

```
1|git push <remote_name> --all
```

In questo modo, tutti i rami presenti nel nostro repository locale saranno inviati al repository remoto.

N.B.: nel caso in cui si verifichino conflitti con il repository remoto durante il processo di push, è possibile utilizzare l'opzione `--force` per eseguire il push dei propri commit, anche in presenza di conflitti con il repository remoto. Tuttavia, è fondamentale essere consapevoli che questa operazione comporta la riscrittura della cronologia nel repository remoto, generando possibili incoerenze nella cronologia condivisa (alcuni repository remoti potrebbero non consentire l'utilizzo del *push forzato*).

Per clonare un repository remoto ed ottenere tutta la sua cronologia, creando così un nuovo repository locale con inclusa la Working Directory, possiamo utilizzare il seguente comando:

```
1|git clone <uri> [<local_destination_path>]
```


Automaticamente, il comando assegnerà **remote** come URI del repository remoto ed **origin** come il suo nome. La cartella di destinazione (se specificata) conterrà il nuovo repository clonato (con URI **remote** e nome **origin**).

N.B.: opzionalmente, è possibile limitare la profondità della cronologia clonata utilizzando l'opzione **--depth**.

Per recuperare le nuove modifiche da un repository remoto, è possibile utilizzare il comando **fetch**:

```
1|git fetch <remote_name>
```

Questo comando recupera le informazioni e la cronologia dal repository remoto, ma non aggiorna direttamente i branch locali. In questo modo, possiamo decidere se e come integrare queste modifiche nei branch locali (utilizzando comandi come **git merge** o **git rebase**).

Nel caso in cui si verifichi un fallimento durante il push (ad esempio, quando si tenta di fare il push di un branch locale **X** su un corrispondente branch remoto **X** e nel frattempo il branch remoto ha subito delle deviazioni), è necessario seguire una procedura specifica:

1. Recuperare le informazioni dal branch remoto **X**. Prima di effettuare il push, è importante assicurarsi di avere le ultime modifiche dal branch remoto. Ciò è possibile farlo eseguendo un **fetch**:

```
1|git fetch origin X
```

2. Unire localmente il branch **X**. Dopo aver recuperato le modifiche dal repository remoto, dobbiamo unire il proprio branch locale **X** con il branch remoto **X**. Questo può essere fatto tramite il comando **merge**:

```
1|git merge origin/X
```

Durante questo processo, potrebbero sorgere conflitti che devono essere risolti manualmente.

3. Risolvere eventuali conflitti. Nel caso in cui si verifichino conflitti durante il merge, è necessario risolverli manualmente.
4. Effettuare il nuovamente push. Dopo aver risolto eventuali conflitti, è possibile effettuare nuovamente il push del branch locale **X** sul branch remoto **X**:

```
1|git push origin X
```

Questo comando invierà le modifiche al branch remoto dopo averlo sincronizzato localmente.

Come abbiamo osservato, il comando **fetch** di Git consente di recuperare le informazioni e la cronologia da un branch remoto senza apportare modifiche al branch locale. Pertanto, dopo il **fetch** dobbiamo unire manualmente il branch locale con il branch remoto. Per semplificare questo processo, Git fornisce il comando **pull**, che combina le seguenti due operazioni:

1. Fetch dal branch remoto: questa operazione consente di recuperare tutte le informazioni e la cronologia dal repository remoto specificato.

```
1|git fetch <remote_name>
```

2. Unione con il branch remoto: il branch locale viene unito con il branch remoto. Durante questa fase, possono sorgere conflitti nel caso in cui siano presenti modifiche sia nel branch locale che in quello remoto. Dobbiamo risolvere questi conflitti manualmente.

```
1|git merge <remote_name>/<branch_name>
```

Queste due operazioni possono essere combinate in un'unica fase utilizzando il comando **pull** in questo modo:

```
1|git pull <remote_name> <branch_name>
```

L'utilizzo del comando **pull** risulta essere utile quando si desidera recuperare le modifiche dal repository remoto e unire automaticamente il branch locale con il corrispondente branch remoto, semplificando il processo di aggiornamento del repository locale.

Il **rebase** è un'operazione che consente di gestire la divergenza tra due branch, ad esempio il branch corrente (R1) ed un altro branch (R2). Al contrario del tradizionale merge, il comando **rebase** consente di riscrivere la cronologia del branch corrente (R1) sopra al branch con cui sta divergendo (R2). Questo processo presenta alcune caratteristiche importanti:

- Tutti i commit nel branch corrente (R1) a partire dal punto di divergenza saranno spostati sopra il branch con cui sta divergendo (R2).
- La storia risultante sarà più lineare ed ordinata rispetto al tradizionale merge, evitando la creazione di molti branch.

Durante il processo di rebase, abbiamo la possibilità di apportare diverse modifiche ai commit esistenti:

- Possiamo aggiornare o modificare i messaggi associati ai singoli commit.
- Possiamo eliminare un commit specifico dalla cronologia del branch corrente.
- Possiamo unire più commit consecutivi in un singolo commit più significativo per semplificare la cronologia (noto come **squash** dei commit.)

N.B.: il rebase riscrive la cronologia del repository e non dovrebbe essere utilizzato su branch di cui è già stato effettuato il push sul repository remoto, poiché potrebbe causare problemi nei successivi push.

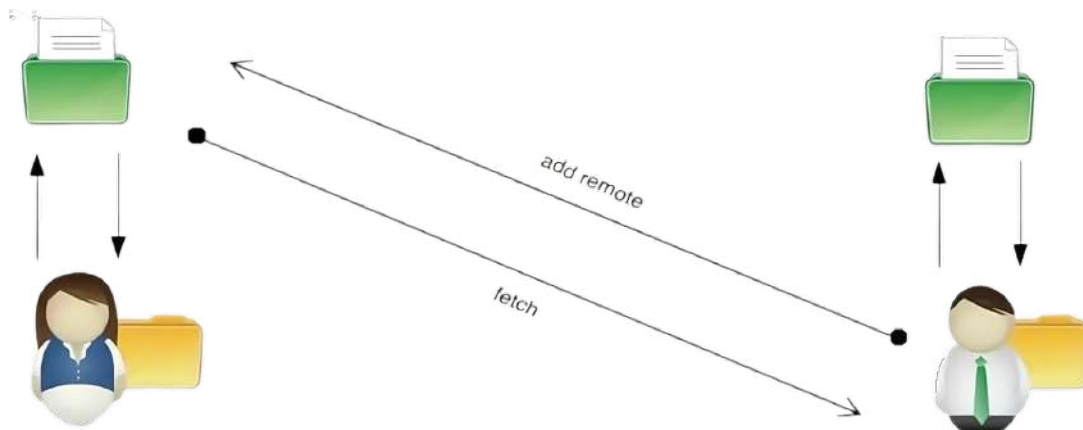
N.B.: durante il rebase possono verificarsi conflitti simili a quelli riscontrati durante il merge. In caso di conflitti, il processo di rebase viene sospeso ed è necessario gestire manualmente i conflitti prima di poter continuare con il rebase.

Il concetto di **Pull Request** costituisce uno degli schemi di lavoro più diffusi nell'ambito dello sviluppo collaborativo, consentendo a diversi membri di un team di contribuire in modo efficace ad un repository Git. Piattaforme come GitHub, GitLab e Bitbucket offrono servizi di hosting e collaborazione che facilitano l'implementazione di questo workflow.

Consideriamo il caso di due sviluppatori, Alice e Bob, entrambi impegnati nello sviluppo di un progetto su cloni locali dello stesso repository. In questo scenario, Alice ha creato un nuovo branch denominato **feature1** nel suo repository locale ed ha effettuato il push di tale branch sul repository remoto. Successivamente, Alice comunica a Bob la necessità di revisionare e integrare il branch **feature1** nel suo repository locale, avviando il processo di Pull Request.



Bob inizia aggiungendo il repository di Alice come remoto, esegue il fetch del branch **feature1** e verifica l'integrità delle modifiche. Una volta accertato che tutto sia corretto, Bob procede con l'unione del branch **feature1** nel suo branch di lavoro all'interno del repository locale.



8.2 GitHub Flow

Il modello di sviluppo noto come **GitHub Flow** costituisce un approccio leggero e basato sull'utilizzo di branch, adatto sia a team che a progetti di piccole dimensioni.

La creazione di branch durante lo sviluppo consente di gestire idee e funzionalità in modo flessibile. Alcune modifiche possono essere pronte per essere integrate nel branch principale (**master**), mentre altre potrebbero richiedere ulteriori sviluppi. L'utilizzo dei branch offre un ambiente isolato per esplorare nuove idee senza influenzare il flusso principale, consentendo di sperimentare senza il timore di compromettere il codice. Quando le modifiche sono complete, è possibile richiedere la revisione da parte degli altri sviluppatori e, se opportuno, integrare tali modifiche nel branch principale. In ogni momento, si presume che il contenuto del branch **master** sia stabile e pronto per la distribuzione.

N.B.: è consigliato assegnare un nome significativo al branch su cui si sta lavorando, come il nome della funzionalità in fase di sviluppo oppure l'identificativo di un bug.

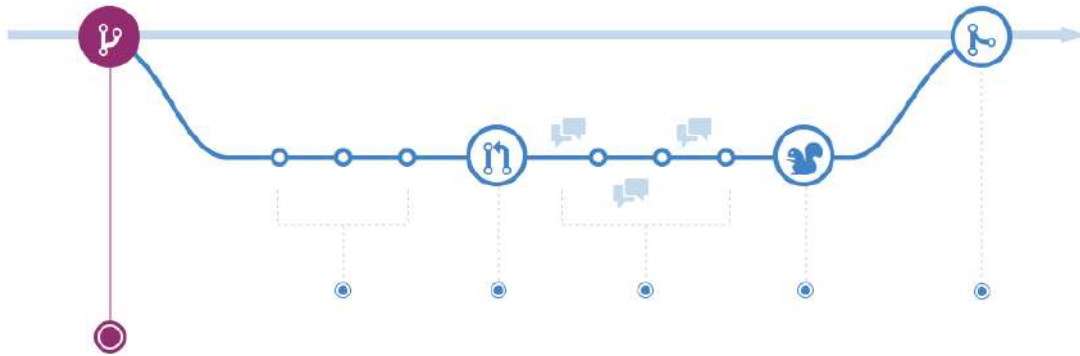


Figura 8.8: Fase di creazione di un branch per testare una nuova funzionalità o risolvere un bug.

Una volta creato il branch, dobbiamo aggiungere commit con frequenza per tenere traccia dei progressi. Ogni commit dovrebbe rappresentare una “unità di cambiamento separata”, in modo da facilitare la revisione. I messaggi dei commit devono essere chiari e informativi, preferibilmente utilizzando diverse righe per separare la descrizione principale da eventuali dettagli aggiuntivi. Questa pratica aiuta a spiegare in modo esaustivo le modifiche apportate da quel commit.

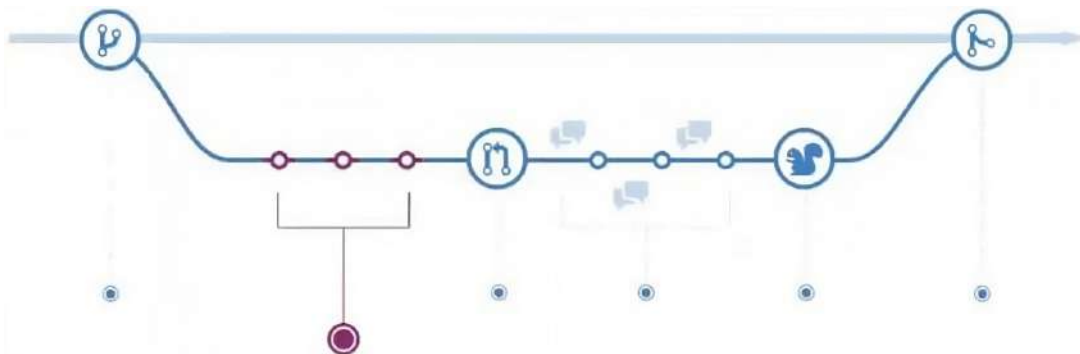


Figura 8.9: Fase di commit del branch per tenere traccia della cronologia dell'implementazione.

Dopo aver eseguito il push del branch desiderato, procediamo con la creazione della Pull Request. Questa richiesta avvia una discussione sulle modifiche apportate nel branch, documentando chiaramente tali proposte di modifica nel repository GitHub. Tale approccio consente a tutti di visualizzare le modifiche che intendiamo unire. Questa pratica può essere utile per iniziare una discussione, chiedere aiuto o raccogliere opinioni sulle direzioni future del branch.

N.B.: la creazione di una Pull Request può avvenire in qualsiasi momento, anche se il lavoro sul branch non è ancora completo.

N.B.: è possibile menzionare specifici utenti utilizzando il tag @<github_id>. Tale menzione attira l'attenzione della persona indicata sulla Pull Request. Una volta creata la Pull Request, si avvia una conversazione con gli altri membri del team su un repository condiviso.

N.B.: quando si contribuisce ad un progetto di un altro sviluppatore (cioè, si fa un fork di qualche repository), si diventa parte integrante di quel progetto. I proprietari

del repository originale possono esaminare e valutare le modifiche proposte, rendendo trasparente il processo di contribuzione.

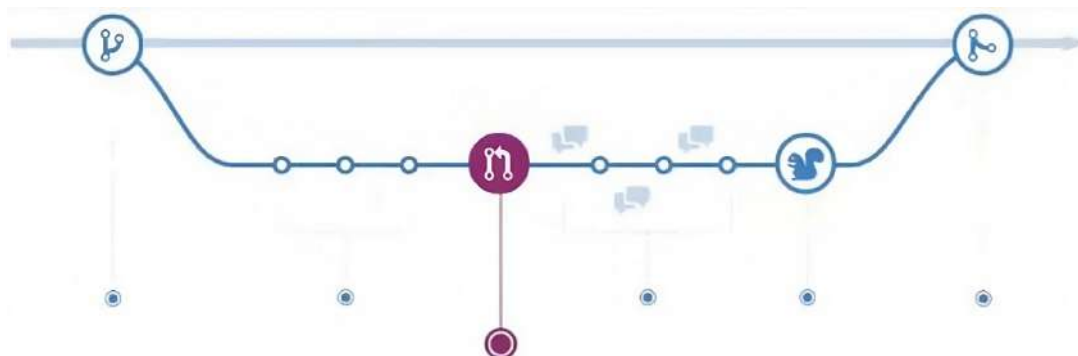


Figura 8.10: Fase di Pull Request per discutere sulle modifiche di un branch con gli altri membri del team.

Dopo aver avviato una Pull Request, inizia la fase di **Code Review** (*revisione del codice*) in cui si avvia una conversazione riguardo alle modifiche proposte sul branch. Se stiamo lavorando in modo indipendente, abbiamo l'opportunità di esaminare attentamente tutte le modifiche incluse nella Pull Request. I commit vengono presentati in modo chiaro, consentendo la visualizzazione di tutte le modifiche apportate a ciascun file coinvolto.

Al contrario, se siamo parte di un team, gli altri sviluppatori hanno la possibilità di commentare ogni riga di modifica, esprimere suggerimenti e richiedere eventuali modifiche o correzioni.

Nel caso in cui vengano effettuati nuovi commit nel branch della Pull Request, questa si aggiornerà automaticamente. La conversazione durante la fase di revisione del codice avviene attraverso il linguaggio **Markdown**, garantendo una formattazione chiara e leggibile.

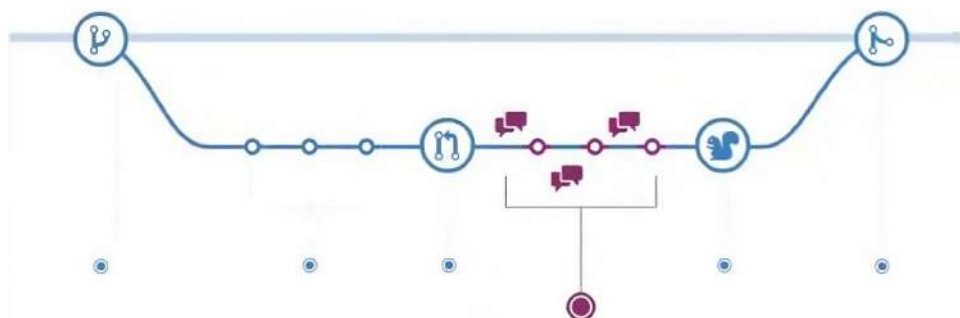


Figura 8.11: Fase di Code Review per esaminare le modifiche incluse nella Pull Request.

Una volta che la Pull Request è stata approvata, procediamo con il merge nel branch di destinazione, che può essere il branch principale (**master**) o un altro branch specificato nella Pull Request. Successivamente, è possibile rimuovere il branch remoto direttamente da GitHub utilizzando l'apposito pulsante presente nella pagina della Pull Request.

Se la Pull Request è collegata a specifiche issues su GitHub, è possibile indicare nel commento di commit "Close #<issue_number>". Quando la Pull Request viene unita nel branch principale (**master**), l'issue correlata viene automaticamente chiusa (cioè,

marcata come risolta). All'interno di tale issue viene tracciata la Pull Request che l'ha chiusa, fornendo così un collegamento tra le modifiche apportate e la risoluzione delle problematiche correlate.

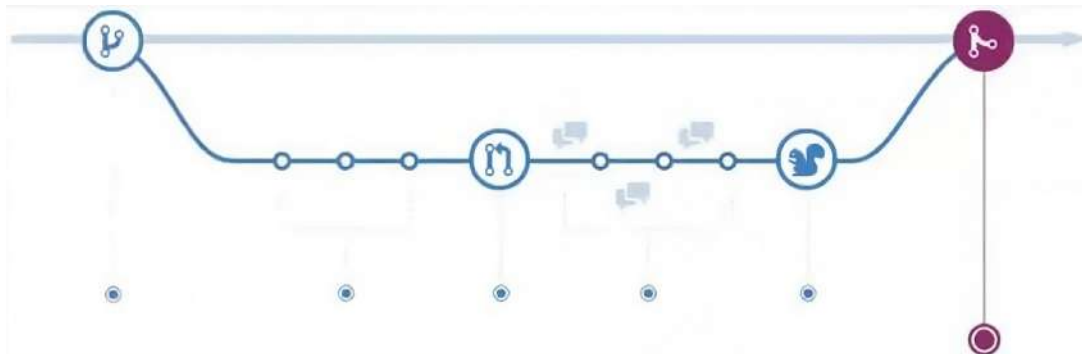


Figura 8.12: Fase di merge delle modifiche nel branch di destinazione.

8.3 Comandi Git

8.3.1 Configurazione

Impostare il nome e l'email da associare ai commit ed ai tag:

```
1|git config --global user.name "your name"
2|git config --global user.email "your email"
```

8.3.2 Inizializzazione di un Progetto con Git

Creare un repository locale all'interno della cartella specificata in <directory>:

```
1|git init <directory>
```

Scaricare e clonare un repository remoto localizzato in <url>:

```
1|git clone <url>
```

8.3.3 Commit

Aggiungere un file all'area di staging:

```
1|git add <file>
```

Aggiungere tutti i file all'area di staging:

```
1|git add .
```

Effettuare il commit di tutti i file presenti nell'area di staging, fornendo un messaggio descrittivo:

```
1|git commit -m "message describing the commit"
```

Aggiungere tutte le modifiche apportate ai file tracciati e fare il commit:

```
1|git commit -am "message describing the commit"
```

Aggiungere un tag al commit corrente (spesso usato per i rilasci di nuove versioni):

```
1|git tag <tag_name>
```

8.3.4 Principi fondamentali di Git

- `main` o `master`: il branch di sviluppo predefinito in Git.
- `origin`: il repository remoto predefinito da cui è stato clonato il repository locale.
- `HEAD`: il riferimento al branch corrente.
- `HEAD^`: il riferimento al genitore del branch corrente.
- `HEAD~4`: il riferimento al trisavolo del branch corrente.

8.3.5 Branch

Elencare tutti i branch locali. Aggiungere il flag `-r` per mostrare tutti i branch remoti:

```
1|git branch
```

Creare un nuovo branch:

```
1|git branch <branch_name>
```

Passare ad un branch ed aggiornare la Work Directory:

```
1|git checkout <branch_name>
```

Creare un nuovo branch e passare ad esso:

```
1|git checkout -b <branch_name>
```

Eliminare un branch unito:

```
1|git checkout -d <branch_name>
```

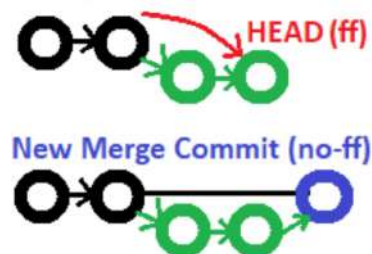
Eliminare un branch, che sia unito o meno:

```
1|git checkout -D <branch_name>
```

8.3.6 Merge

Unire il branch X al branch Y. Aggiungere l'opzione `--no-ff` per una merge senza Fast Forward:

```
1|git checkout Y
2|git merge X
```



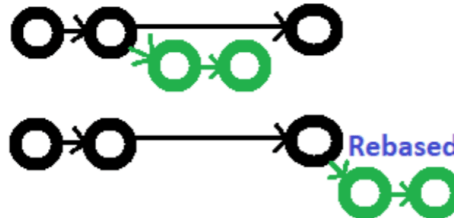
Unire un branch e schiacciare (squash) tutti i commit in un nuovo commit:

```
1|git merge --squash a
```

8.3.7 Rebasing

Fare un rebase di un branch **feature** sul branch principale **master** per incorporare le nuove modifiche (previene i commit di merge non necessari, mantenendo la cronologia pulita):

```
1|git checkout feature
2|git rebase master
```



Pulire in modo interattivo i commit di un branch prima di eseguire il rebase sul branch principale:

```
1|git rebase -i master
```

Pulire in modo interattivo gli ultimi 3 commit di un branch prima di eseguire il rebase sul branch principale:

```
1|git rebase -i Head~3
```

8.3.8 Annullare

Spostare e/o rinominare un file:

```
1|git mv <existing_path> <new_path>
```

Rimuovere un file dalla Working Directory e dall'area di staging:

```
1|git rm <file>
```

Rimuovere un file solo dall'area di staging:

```
1|git rm --cached <file>
```

Visualizzare un commit precedente (solo in lettura):

```
1|git checkout <commit_ID>
```

Creare un nuovo commit, ripristinando le modifiche di un commit specificato:

```
1|git revert <commit_ID>
```

Tornare ad un commit precedente cancellando tutti gli altri commit (il revert è più sicuro). Aggiungere il flag **--hard** per cancellare anche le modifiche allo spazio di lavoro.

```
1|git reset <commit_ID>
```

8.3.9 Esaminare il repository

Elencare i nuovi file o i file modificati di cui non è ancora stato fatto un commit:

```
1|git status
```

Elencare la cronologia dei commit, con i rispettivi ID:

```
1|git log --oneline
```


Mostrare le modifiche ai file che non sono nell'area di stage. Aggiungere l'opzione `--cached` per mostrare le modifiche ai file presenti nell'area di stage:

```
1|git diff
```

Mostrare le modifiche tra due commit:

```
1|git diff <commit1_ID> <commit2_ID>
```

8.3.10 Stash

Memorizzare le modifiche ed i cambiamenti dei file presenti nell'area di stage. Aggiungere il flag `-u` per includere i file non tracciati. Aggiungere il flag `-a` per includere i file non tracciati ed i file ignorati:

```
1|git stash
```

Come sopra, ma aggiungendo un commento:

```
1|git stash save "comment"
```

Stash parziale. Conserva solo un singolo file, un insieme di file o singole modifiche all'interno dei file:

```
1|git stash -p
```

Elencare tutti gli stash:

```
1|git stash list
```

Riapplicare lo stash senza cancellarlo:

```
1|git stash apply
```

Riapplicare lo stash all'indice 2, quindi eliminarlo dall'elenco degli stash. Omettere `stash@{n}` per eliminare lo stash più recente:

```
1|git stash pop stash@{2}
```

Mostrare il riepilogo della differenza dello stash 1. Passare il flag `-p` per vedere la differenza completa:

```
1|git stash show stash@{1}
```

Eliminare lo stash all'indice 1. Omettere `stash@{n}` per eliminare l'ultimo stash creato:

```
1|git stash drop stash@{1}
```

Cancellare tutti gli stash:

```
1|git stash clear
```

8.3.11 Sincronizzazione

Aggiungere un repository remoto:

```
1|git remote add <name> <uri>
```

Visualizzare tutte le connessioni remote. Aggiungere il flag `-v` per visualizzare gli url:

```
1|git remote
```

Rimuovere una connessione:

```
1|git remote remove <name>
```

Rinominare una connessione:

```
1|git remote rename <old_name> <new_name>
```

Recuperare tutti i branch dal repo remoto (senza merge):

```
1|git fetch <name>
```

Recuperare un branch specifico:

```
1|git fetch <name> <branch_name>
```

Recuperare la copia del branch corrente dal repository remoto (con merge):

```
1|git pull
```

Spostare (rebase) le modifiche locali in cima alle nuove modifiche apportate al repository remoto (per una cronologia pulita e lineare):

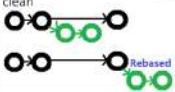
```
1|git pull --rebase <name>
```

Caricare il contenuto locale nel repository remoto:

```
1|git push <name>
```

Caricare il contenuto locale in un branch del repository remoto:

```
1|git push <name> <branch_name>
```

Git Cheat Sheet			
Setup Set the name and email that will be attached to your commits and tags <pre>\$ git config --global user.name "Danny Adams"</pre> <pre>\$ git config --global user.email "my-email@gmail.com"</pre>	Branches List all local branches. Add -r flag to show all remote branches. -a flag for all branches. <pre>\$ git branch</pre> Create a new branch <pre>\$ git branch <new-branch></pre> Switch to a branch & update the working directory <pre>\$ git checkout <branch></pre> Create a new branch and switch to it <pre>\$ git checkout -b <new-branch></pre> Delete a merged branch <pre>\$ git branch -d <branch></pre> Delete a branch, whether merged or not <pre>\$ git branch -D <branch></pre> Add a tag to current commit (often used for new version releases) <pre>\$ git tag <tag-name></pre>	Rebasing Rebase feature branch onto main (to incorporate new changes made to main). Prevents unnecessary merge commits into feature, keeping history clean  <pre>\$ git checkout feature</pre> <pre>\$ git rebase main</pre> Iteratively clean up a branches commits before rebasing onto main <pre>\$ git rebase -i main</pre> Iteratively rebase the last 3 commits on current branch <pre>\$ git rebase -i HEAD~3</pre>	Review your Repo List new or modified files not yet committed <pre>\$ git status</pre> List commit history, with respective IDs <pre>\$ git log --oneline</pre> Show changes to unstaged files. For changes to staged files, add --cached option <pre>\$ git diff</pre> Show changes between two commits <pre>\$ git diff commit1_ID commit2_ID</pre>
			Delete stash at index 1. Omit stash@{n} to delete last stash made <pre>\$ git stash drop stash@{1}</pre> Delete all stashes <pre>\$ git stash clear</pre>
Start a Project Create a local repo (omit <directory> to initialise the current directory as a git repo) <pre>\$ git init <directory></pre> Download a remote repo <pre>\$ git clone <url></pre>	Merging Merge branch a into branch b. Add --no-ff option for no-fast-forward merge  <pre>\$ git checkout b</pre> <pre>\$ git merge a</pre> Merge & squash all commits into one new commit <pre>\$ git merge --squash a</pre>	Undoing Things Move (&/or rename) a file & stage move <pre>\$ git mv <existing_path> <new_path></pre> Remove a file from working directory & staging area, then stage the removal <pre>\$ git rm <file></pre> Remove from staging area only <pre>\$ git rm --cached <file></pre> View a previous commit (READ only) <pre>\$ git checkout <commit_ID></pre> Create a new commit, reverting the changes from a specified commit <pre>\$ git revert <commit_ID></pre> Go back to a previous commit & delete all commits ahead of it (revert is safer). Add --hard flag to also delete workspace changes (BE VERY CAREFUL) <pre>\$ git reset <commit_ID></pre>	Stashing Store modified & staged changes. To include untracked files, add -u flag. For untracked & ignored files, add -a flag. <pre>\$ git stash</pre> As above, but add a comment. <pre>\$ git stash save "comment"</pre> Partial stash. Stash just a single file, a collection of files, or individual changes from within files <pre>\$ git stash -p</pre> List all stashes <pre>\$ git stash list</pre> Re-apply the stash without deleting it <pre>\$ git stash apply</pre> Re-apply the stash at index 2, then delete it from the stash list. Omit stash@{n} to pop the most recent stash. <pre>\$ git stash pop stash@{2}</pre> Show the diff summary of stash 1. Pass the -p flag to see the full diff. <pre>\$ git stash show stash@{1}</pre>
Make a Change Add a file to staging <pre>\$ git add <file></pre> Stage all files <pre>\$ git add .</pre> Commit all staged files to git <pre>\$ git commit -m "commit message"</pre> Add all changes made to tracked files & commit <pre>\$ git commit -am "commit message"</pre>	Synchronizing Add a remote repo <pre>\$ git remote add <alias> <url></pre> View all remote connections. Add -v flag to view urls. <pre>\$ git remote</pre> Remove a connection <pre>\$ git remote remove <alias></pre> Rename a connection <pre>\$ git remote rename <old> <new></pre> Fetch all branches from remote repo (no merge) <pre>\$ git fetch <alias></pre> Fetch a specific branch <pre>\$ git fetch <alias> <branch></pre> Fetch the remote repo's copy of the current branch, then merge <pre>\$ git pull</pre> Move (rebase) your local changes onto the top of new changes made to the remote repo (for clean, linear history) <pre>\$ git pull --rebase <alias></pre> Upload local content to remote repo <pre>\$ git push <alias></pre> Upload to a branch (can then pull request) <pre>\$ git push <alias> <branch></pre>	Basic Concepts main: default development branch origin: default upstream repo HEAD: current branch HEAD*: parent of HEAD HEAD~4: great-great-grandparent of HEAD By @DoobleDanny	

Capitolo 9

CONTINUOUS INTEGRATION

Il **Continuous Integration** (*integrazione continua*) è un processo applicato in contesti in cui lo sviluppo del software fa uso di un sistema di versionamento (VCS). Questo processo implica un allineamento frequente degli ambienti di sviluppo dei programmatori con il codice principale (mainline). In particolare, risulta essere efficace in contesti che impiegano un meccanismo di compilazione automatizzato, soprattutto quando vengono inclusi test automatici.

Un **Continuous Integration Server (CI Server)** è un server dedicato all'automazione del processo di compilazione. Ogni sviluppatore contribuisce a specifiche parti del software, eseguendo Unit Test sul proprio computer. Il compito del CI Server è quello di compilare l'intero software e di eseguire tutti i test, inclusi gli Integration Test e gli End-to-End test che richiedono un tempo più considerevole per essere completati. Inoltre, il CI Server può generare report sulla Code Coverage e sulla Code Quality. Nel caso in cui si verificano problemi, lo sviluppatore riceverà un feedback immediato per segnalare che le sue modifiche hanno compromesso la compilazione.

Il flusso di lavoro del processo di Continuous Integration prevede i seguenti passaggi:

1. Gli sviluppatori effettuano il push delle loro modifiche nel repository remoto.
2. Il CI Server si attiva automaticamente in risposta al push.
3. Il CI Server avvia il processo di compilazione, esegue i test e fornisce un feedback immediato allo sviluppatore.

N.B.: l'intero processo è automatico (l'unica azione manuale è il push delle modifiche).

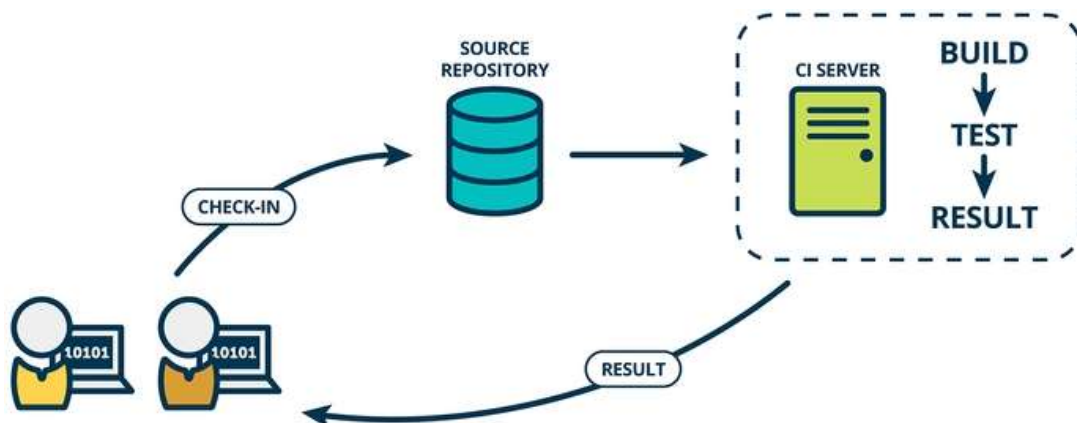


Figura 9.1: Workflow del processo di Continuous Integration.

Di seguito sono elencati gli strumenti che utilizzeremo nel processo di Continuous Integration:

- **GitHub**: utilizzato come host del repository Git per il nostro progetto.
- **Maven**: strumento impiegato per la compilazione e il testing del progetto.
- **GitHub Actions**: utilizzato come CI Server per consentire l'automazione del processo di Continuous Integration. Fornisce ambienti virtuali per i Sistemi Operativi Linux, Windows e macOS.
- **Coveralls**: utilizzato per registrare la cronologia e le statistiche di Code Coverage, offrendo una visione dettagliata.
- **SonarCloud**: utilizzato per condurre l'analisi della Code Quality.

Tutti questi strumenti sono progettati per interoperare tra loro. Richiedono solo la creazione di account gratuiti e il collegamento tra di loro per implementare il processo di Continuous Integration.

N.B.: tutti questi strumenti sono gratuiti per progetti open source pubblici.

Il nostro processo di Continuous Integration si sviluppa nel seguente modo:

1. Inseriamo il nostro repository Git sulla piattaforma GitHub.
2. Effettuiamo il push di un commit su GitHub.
3. Queste azioni innescano automaticamente il processo di build sul CI Server GitHub Actions.
4. Il progetto viene compilato, comprendendo il download automatico di tutte le dipendenze e la compilazione del codice.
5. I test vengono eseguiti, includendo la misurazione della Code Coverage.
6. I risultati dettagliati sulla Code Coverage vengono trasmessi a Coveralls.
7. Viene eseguita un'analisi approfondita della Code Quality utilizzando SonarCloud.
8. Tutti i risultati ottenuti da questi servizi remoti (cioè GitHub Actions, Coveralls, SonarCloud) vengono riportati su GitHub. Questi risultati vengono utilizzati per annotare il commit e la Pull Request con alcuni dettagli (come eventuali successi o fallimenti).

GitHub Actions ci offre la possibilità di implementare **workflows** (*flussi di lavoro*) per l'automazione della compilazione, costruendo e testando il codice archiviato nei repository GitHub. Un workflow viene attivato automaticamente quando si effettua il push di un branch su GitHub oppure quando si crea una Pull Request.

Questi workflows vengono eseguiti di default su macchine virtuali ospitate da GitHub, ma è anche possibile eseguirli su macchine virtuali gestite da noi stessi.

N.B.: gli workflows di GitHub Actions possono essere configurati per essere eseguiti anche in risposta ad eventi specifici (ad esempio, quando viene avviata una nuova issue su GitHub) oppure secondo una pianificazione predefinita (ad esempio, ogni notte).

N.B: nel nostro processo di Continuous Integration, ci limiteremo ad utilizzare le macchine virtuali ospitate da GitHub.

La definizione di un workflow di GitHub Actions avviene attraverso un file con estensione `.yaml` o `.yml` e segue la sintassi **YAML**, cioè un linguaggio di specificazione tipicamente utilizzato per le configurazioni. Questo file deve essere posizionato nella directory `.github/workflows`, situata nella root del repository Git (non è essenziale il nome del file, purché abbia estensione `.yaml` o `.yml` e risieda in questa specifica path nella cartella del progetto).

Ogni file di workflow rappresenta la configurazione di un flusso di lavoro distinto. Possiamo avere quanti file di workflow desideriamo in questa directory, poiché saranno tutti esaminati ed eventualmente eseguiti da GitHub Actions.

Un file di workflow è composto da uno o più **job**. Ogni job specifica il **runner** per quel particolare job, ovvero la macchina virtuale su cui desideriamo eseguire quel job. In particolare, viene fornito il nome di uno degli ambienti virtuali offerti da GitHub (come Ubuntu, Windows o macOS). Ogni job in un workflow viene eseguito in un ambiente virtuale separato.

Ogni volta che viene eseguito un job, si avvia una nuova macchina virtuale partendo da uno spazio pulito e perdendo tutti i dati generati nelle esecuzioni precedenti del workflow. Tuttavia, è possibile utilizzare le cache per memorizzare le dipendenze e pubblicare eventuali artefatti (come file JAR o risultati di test). In questo modo possiamo preservare le informazioni tra le diverse esecuzioni di un job.

In GitHub Actions, un job rappresenta un elenco ordinato di **step** eseguiti sullo stesso runner del job. Ogni step condivide dati di input e di output con gli altri step dello stesso job.

Uno step può assumere diverse forme:

- Può essere un comando di shell, definito con la chiave **run** (ad esempio, uno step potrebbe eseguire la compilazione di Maven). Il comando dipende dalla shell del Sistema Operativo virtuale.
- Può utilizzare un'**action**, definita con la chiave **uses**. Il concetto di action è simile a quello di plug-in (consente di incorporare azioni predefinite o personalizzate negli step del workflow). Le action possono essere reperite nel marketplace di GitHub Actions.

N.B: possiamo assegnare un nome specifico ad ogni step. Tale nome verrà mostrato nel log dell'esecuzione del workflow, migliorando la tracciabilità e la comprensione del processo in corso.

Un file YAML è composto da coppie key-value, in cui ogni chiave può definire un sottoinsieme di valori annidati anch'essi come coppie key-value. Gli spazi sono fondamentali per delineare questi sottoinsiemi annidati. Pertanto, YAML è un linguaggio space-sensitive, ma non supporta l'uso di tabulazioni.



ES:

Per implementare un workflow, creiamo un file `.yaml` all'interno della directory `.github/workflows` nella cartella principale del repository Git. Di se-

guito è riportato un esempio di configurazione per un workflow:

```

1 name: Java CI with Maven in Linux
2 on:
3   push:
4   pull_request:
5 jobs:
6   build:
7     runs-on: ubuntu-latest
8     steps:
9       # Step to clone repo on VM.
10      - uses: actions/checkout@v2
11      # Step to set Java version.
12      - name: Set up JDK 8
13        uses: actions/setup-java@v1
14        with:
15          java-version: 1.8
16      # Step to enable caching.
17      - name: Cache Maven packages
18        uses: actions/cache@v2
19        with:
20          path: ~/.m2
21          key: ${ runner.os }-m2-${ hashFiles('**/pom.xml') }
22          restore-keys: ${ runner.os }-m2-
23      # Step to execute Maven command.
24      - name: Build with Maven
25        run: mvn -f com.examples.myproject/pom.xml clean verify

```

La chiave `on` definisce gli eventi per i quali si desidera eseguire il workflow (in questo caso, il workflow viene eseguito ad ogni push e ad ogni Pull Request, ma è possibile configurare il workflow per essere eseguito solo quando il push avviene su un particolare branch, ecc...).

La chiave `jobs` definisce un job che vogliamo eseguire (è possibile definire più job contemporaneamente). Un job è identificato da un nome e dal parametro `runs-on` che specifica il runner in cui eseguire il job `build` (in questo caso, un Sistema Operativo Ubuntu di ultima versione). Ogni job definisce un elenco di steps.

Poiché ogni esecuzione di un job avviene su una nuova macchina virtuale, Maven scarica e reinstalla le dipendenze ad ogni avvio. Per risolvere questo problema, possiamo sfruttare la cache utilizzando un'apposita action, specificando il nome della cartella che desideriamo conservare e la chiave per l'archiviazione nella cache (è necessario utilizzare cache diverse per l'esecuzione del workflow su Sistemi Operativi differenti). Inoltre, calcoliamo l'hash del file `pom.xml`. In questo modo, ogni volta che il file `pom.xml` cambia, la cache viene invalidata e creata una nuova cache con le nuove dipendenze.

Le actions offerte GitHub prevedono anche delle **build metrics**, che consentono di definire delle configurazioni diverse per uno stesso job. In altre parole, anziché creare diversi workflow con differenti job per testare varie configurazioni, possiamo specificare diverse build metrics per lo stesso workflow. In questo caso, desideriamo verificare che la build funzioni con diverse

versioni di Java (ad esempio, Java 8 e Java 11) su diversi Sistemi Operativi (ad esempio, Ubuntu, Windows e macOS). Pertanto, definiamo un solo job con diverse configurazioni passando una lista come argomento e richiamando questi argomenti nel resto della configurazione.

Possiamo anche effettuare configurazioni combinate per testare comportamenti diversi su differenti architetture. Tuttavia, essendo una matrice, il numero di combinazioni cresce in modo quadratico e GitHub Actions potrebbe imporre limiti se ne facciamo un uso eccessivo. D'altro canto, non siamo sempre interessati a tutte le combinazioni e a volte possiamo escluderne alcune.

```

1 | name: Java CI with Maven in different OS
2 | on:
3 |   push:
4 |   pull_request:
5 | jobs:
6 |   build:
7 |     runs-on: ubuntu-latest
8 |     strategy:
9 |       matrix:
10 |         #test against several OS.
11 |         os: [ubuntu-latest, windows-latest, macos-latest]
12 |         # Test against several Java version.
13 |         java: [8, 11]
14 |         # Excludes JDK 11 on Windows and macOS.
15 |         exclude:
16 |           - os: macos-latest
17 |             java: 11
18 |           - os: windows-latest
19 |             java: 11
20 |     name: Build with Java ${ matrix.java } on ${ matrix.os }
21 |     steps:
22 |       # Step to clone repo on VM.
23 |       - uses: actions/checkout@v2
24 |       # Step to set Java version.
25 |       - name: Set up JDK ${ matrix.java }
26 |         uses: actions/setup-java@v1
27 |         with:
28 |           java-version: ${ matrix.java }
29 |       # Step to execute Maven command from shell.
30 |       - name: Build with Maven
31 |         run: >
32 |           mvn -f com.examples.myproject/pom.xml
33 |             clean verify
34 |             surefire-report:report-only site:site -DgenerateReports=
35 |               ↪ false
36 |       # Step to store generated artifacts.
37 |       - name: Archive JUnit Report
38 |         uses: actions/upload-artifact@v2
39 |         with:
40 |           name: surefire-report-jdk-${ matrix.java }
41 |           path: '**/target/site'
```

N.B.: possiamo accedere alle variabili con la sintassi `${{ variable }}`.

N.B.: analogamente alla chiave `exclude`, possiamo utilizzare la chiave `include` e specificare soltanto le configurazioni che vogliamo combinare. Un'altra possibile strategia è quella di definire più workflows (ognuno definito in un file `.yaml` distinto).

N.B.: se necessario possiamo conservare gli artefatti generati durante la build (come JAR, reports, documentazioni, ecc...).

N.B.: ogni volta che eseguiamo un push di un commit sul repository GitHub, la build verrà avviata automaticamente su GitHub Actions.

9.1 Coveralls

Coveralls è uno strumento online che consente di monitorare la Code Coverage di tutti i progetti su GitHub. Per integrare questo servizio, è necessario inviare il report generato con JaCoCo. Tuttavia, per farlo, è fondamentale connettersi all'account GitHub su Coveralls. A tale scopo, dobbiamo evitare di inserire la password direttamente nel file `pom.xml` poiché si tratta di un'informazione sensibile. Invece, possiamo crittografare la password, ed utilizzarla da JaCoCo per inviare il report a Coveralls.

Per raggiungere questo scopo, Coveralls fornisce un token associato ad un progetto GitHub specifico. Questo token deve essere inserito nel repository GitHub (va notato che anche questo token è considerato una password e non dovrebbe essere salvato in chiaro nel progetto). Nelle impostazioni del progetto su GitHub, possiamo creare un nuovo "Secret", assegnargli un nome ed inserire il token nel campo "Value". GitHub crittograferà queste informazioni e ci sarà possibile fare riferimento ad esse con il nome assegnato.

N.B.: se cloniamo il repository con un fork, questo "Secret" non sarà disponibile nel clone e sarà accessibile solo al proprietario del repository.

N.B.: possiamo configurare la comunicazione tra GitHub e Coveralls per ottenere feedback. Ad esempio, è possibile impostare la build in modo che fallisca se Coveralls rileva una percentuale di copertura al di sotto di una soglia specificata. Questo contrassegnerà la build come fallita su GitHub e Coveralls commenterà la Pull Request con informazioni rilevanti.

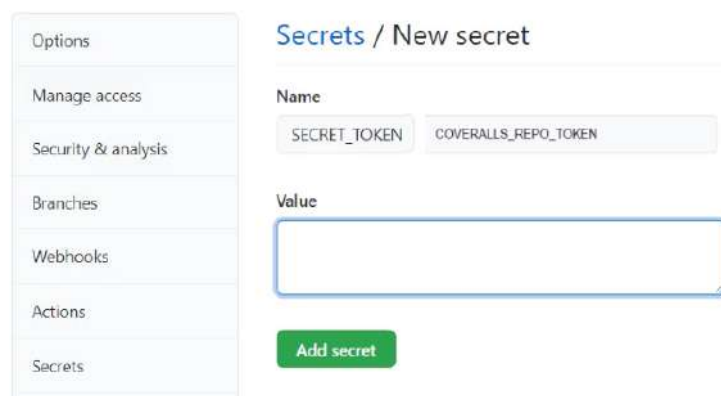


Figura 9.2: Aggiunta di un nuovo "Secret" su GitHub per crittografare un'informazione sensibile.

Di conseguenza possiamo configurare il workflow nel seguente modo:

```

1 name: Java CI with Maven and Coveralls report
2 on:
3   push:
4   pull_request:
5 jobs:
6   build:
7     runs-on: ubuntu-latest
8     strategy:
9       matrix:
10        # Test against several Java versions.
11        include:
12          - java: 8
13            additional-maven-args: "-Pjacoco -DrepoToken=${COVERALLS_REPO_TOKEN}
14              ↪ coveralls:report$"
15          - java: 11
16 name: Build with Java ${ matrix.java } on Linux
17 steps:
18   # Step to clone repo on VM.
19   - uses: actions/checkout@v2
20   # Step to set Java version.
21   - name: Set up JDK ${ matrix.java }
22     uses: actions/setup-java@v1
23     with:
24       java-version: ${ matrix.java }
25   # Step to enable caching.
26   - name: Cache Maven packages
27     uses: actions/cache@v2
28     with:
29       path: ~/.m2
30       key: ${ runner.os }-m2-${ hashFiles('**/pom.xml') }
31       restore-keys: ${ runner.os }-m2-
32   # Step to execute Maven command from shell.
33   - name: Build with Maven
34     run: >
35       mvn -f com.examples.myproject/pom.xml
36       clean verify ${ matrix.additional-maven-args }
37     env:
38       COVERALLS_REPO_TOKEN: ${ secrets.COVERALLS_TOKEN }
39   # Step to execute JaCoCo report command.
40   - name: Generate JUnit Report
41     run: >
42       mvn -f com.examples.myproject/pom.xml
43       surefire-report:report-only site:site -DgenerateReports=false
44     if: ${ always() }
45   # Step to store generated artifacts.
46   - name: Archive JUnit Report
47     uses: actions/upload-artifact@v2
48     if: ${ always() }
49     with:
50       name: surefire-report-jdk-${ matrix.java }
51       path: '**/target/site'
```

Capitolo 10

DOCKER

I **container** Linux rappresentano un meccanismo di virtualizzazione che opera a livello di Sistema Operativo. Questa tecnologia consente l'esecuzione di Sistemi Operativi Linux multipli ed isolati sulla stessa macchina, senza la necessità di virtualizzare il kernel della macchina ospite. Tale approccio offre agli sviluppatori la possibilità di creare un ambiente di sviluppo che è riproducibile e identico all'ambiente di produzione.

L'uso di una macchina virtuale (come Virtualbox) comporta la virtualizzazione, l'installazione e l'esecuzione di un intero Sistema Operativo, compreso il kernel. Questo approccio richiede una considerevole quantità di spazio sull'host ed un notevole tempo per avviare la macchina virtuale, poiché deve essere avviato il Sistema Operativo completo all'interno dell'host. Inoltre, è necessario allocare specifiche risorse dell'host (come memoria e processore) alla macchina virtuale.

Invece, i container presentano un approccio più leggero sia in termini di risorse che di tempo. L'esecuzione dei container è molto simile all'avvio di un processo locale, in quanto utilizzano il kernel dell'host e condividono le risorse con quest'ultimo (ad esempio, se l'host possiede un Sistema Operativo Ubuntu, allora container può eseguire RedHat sfruttando il kernel Ubuntu dell'host).

I vantaggi chiave nell'uso dei container includono:

- Gli ambienti di sviluppo e produzione possono essere facilmente replicati. Ciò significa che tutti, sia gli sviluppatori che gli utenti finali, lavoreranno con le stesse versioni dei server (come MySQL, Apache, ecc...) e con le stesse configurazioni (ad esempio, l'inizializzazione del database sarà uniforme per tutti).
- Il package dell'applicazione finale conterrà tutte le dipendenze ed i servizi necessari per la sua esecuzione. Ciò semplifica la distribuzione, riducendo le possibili incompatibilità e semplificando la gestione delle dipendenze.

Docker si basa su container Linux, offrendo un livello di astrazione più elevato per nascondere le complessità sottostanti. L'architettura di Docker comprende principalmente due componenti fondamentali:

- **Docker Daemon** (o **Host**): rappresenta la macchina (fisica o virtuale) in esecuzione che ospita il "motore di Docker". Questo motore è responsabile della gestione dei container e dell'esecuzione delle istanze di applicazioni all'interno di essi.
- **Docker Client**: rappresenta un programma di configurazione che consente la comunicazione con il motore di Docker e offre un'interfaccia per utilizzare le funzionalità fornite da Docker.

Durante lo sviluppo, l'Host ed il Client risiedono sulla stessa macchina, ma in generale possono essere distribuiti su macchine diverse.

Un **container Docker** rappresenta un'istanza in esecuzione di un'**immagine Docker**. Quando un Client avvia un container, crea un'istanza di questo container eseguendo l'immagine specificata.

N.B.: un'immagine Docker può essere eseguita in più Container Docker.

N.B.: possiamo pensare alla relazione tra un'immagine Docker ed un container Docker come la relazione tra programma e processo. In questo contesto, l'immagine Docker può essere vista come il programma, mentre il container Docker rappresenta un processo in esecuzione.

N.B.: i container vengono eseguiti sul Client, fornendo un ambiente isolato e portatile per l'esecuzione delle applicazioni.

Quando un Client desidera eseguire un container, deve prima recuperare (**pull**) un'immagine pre-costruita da uno dei **registri** disponibili. In questo contesto, un registro è la nomenclatura Docker per indicare un repository. Il registro predefinito è **Docker Hub**. Una volta estratta l'immagine, il Client può procedere a costruire una nuova immagine utilizzando il comando **build** oppure eseguire direttamente l'immagine su un container con il comando **run**.

Quando il Client richiede l'esecuzione di un'immagine, l'Host verifica se l'immagine è già presente nella cache locale della macchina. Nel caso in cui non sia presente, l'Host scarica l'immagine dal registro centrale (ad esempio Docker Hub) e la memorizza localmente.

N.B.: possiamo estrarre molteplici immagini con versioni diverse sullo stesso Host.

N.B.: ogni immagine rappresenta un componente software distinto, consentendo l'esecuzione di ambienti isolati e indipendenti sullo stesso Host.

N.B.: un'immagine Docker è un modello di sola lettura (**read-only**). un container Docker è un modello di lettura e scrittura (read-write) che esegue un'immagine Docker.

L'architettura di un container Docker consiste in diversi livelli. Questo sistema prevede un livello di base di sola lettura (read-only), al quale possono essere aggiunti livelli successivi. L'unico livello che può essere modificato è quello più alto. Questa struttura permette di condividere i livelli sottostanti tra più immagini, consentendo di risparmiare spazio.

Il registro di Docker conserva varie versioni di un'immagine, identificate mediante il formato **image-name:tag**. Il tag predefinito è **latest** ed indica la versione più recente dell'immagine.

Un container dovrebbe eseguire un singolo comando o servizio. Tuttavia, spesso un'applicazione è costituita da più container (ad esempio, un server web, un server di applicazioni ed un database). In questi casi, può essere utile utilizzare **Docker Compose**, ovvero uno strumento progettato per definire e gestire applicazioni basate su più container. Docker Compose si occupa dell'*orchestrazione* di questi container, facilitando la comunicazione tra i servizi necessari per il funzionamento dell'applicazione. In particolare, tutti i container vengono configurati ed eseguiti con un solo comando, interconnettendo tutti i servizi.

N.B.: Docker funziona in modo ottimale e nativo su Linux. Tuttavia, è possibile utilizzarlo anche su sistemi Windows e Mac. Su Windows, è addirittura possibile eseguire container in modo "nativo" grazie al sottosistema Linux. Le istruzioni dettagliate per

l'installazione possono essere trovate sul sito web ufficiale di Docker, che fornisce indicazioni specifiche in base al Sistema Operativo e alla distribuzione Linux utilizzati. In alcuni casi, potrebbe essere necessario installare anche Docker Compose, a seconda del Sistema Operativo.

I comandi di Docker vengono lanciati specificando l'eseguibile **docker** seguito da un comando Docker.

Di norma, Docker richiede i privilegi di superutente, pertanto è comune eseguire i comandi Docker con **sudo**. Un suggerimento per evitare l'uso frequente di **sudo** è aquello giungere il proprio utente al gruppo **docker**. Ciò può essere fatto con il comando:

```
1|sudo usermod -aG docker ${USER}
```

Successivamente, è necessario effettuare il logout e rifare il login affinché le modifiche abbiano effetto. Dopo questa procedura, non sarà più necessario utilizzare **docker** per eseguire i comandi Docker.

Il comando **docker run** consente di eseguire un comando in un nuovo container, estraendo l'immagine e avviando il container. Durante la prima esecuzione, potrebbe essere necessario attendere il download completo dell'immagine, inclusi tutti i suoi layer. Le principali opzioni disponibili per questo comando sono:

- **-i**: mantiene STDIN aperto anche se non collegato. Cioè, consente l'input interattivo, anche quando il terminale non è effettivamente in modalità interattiva (può essere utile in situazioni in cui si desidera fornire un input ad un'applicazione all'interno di un container, anche se il terminale utente non è direttamente coinvolto nell'interazione).
- **-t**: alloca uno pseudo-terminale all'interno del container.
- **-d**: esegue il container in background e stampa il suo ID.
- **--name**: assegna un nome al contenitore. Quando viene creato un contenitore senza specificare un nome tramite l'opzione **--name**, Docker assegna automaticamente un nome casuale
- **--rm**: rimuove automaticamente il container quando termina.
- **-e**: imposta una variabile d'ambiente.
- **-P**: pubblica tutte le porte esposte su porte casuali dell'Host.
- **-p <host_port>:<container_port>**: pubblica la porta (o le porte) di un container sulla porta specificata dell'Host.
- **-m**: limita l'utilizzo della memoria del container.

N.B. per visualizzare l'elenco di tutti i container in esecuzione, è possibile utilizzare il comando **docker ps**. L'opzione **-a** consente di elencare anche i container non in esecuzione.

I container sono *effimeri*. Ovvero, ogni volta che eseguiamo **docker run**, anche con un'immagine già utilizzata in precedenza, viene avviato un nuovo container. Ad esempio, se creiamo un file all'interno di un container e successivamente usciamo, tale file

non sarà presente nel nuovo container generato tramite un'altra esecuzione di `docker run`.

Per garantire la persistenza dei dati tra le esecuzioni dei container, è fondamentale fare uso dei **Data Volumes**. Un Data Volume rappresenta una directory speciale utilizzata da uno o più container, completamente separata dal file system di Docker. Tali Data Volumes vengono inizializzati durante la creazione di un container e possono essere condivisi e riutilizzati da più container, mantenendo la loro integrità anche dopo la cancellazione di singoli container.

Poiché la gestione della persistenza dei dati associati ai container avviene attraverso l'uso di Data Volumes, i container dovrebbero essere considerati effimeri, poiché persistono nel file system anche dopo la loro terminazione.

N.B: Docker non cancella mai automaticamente i Data Volumes. Pertanto, se si desidera rimuoverli, è necessario eseguire questa operazione manualmente.

N.B: si consiglia di avviare i container con l'argomento `--rm`, il quale provvede automaticamente alla rimozione del container al termine della sua esecuzione.

Possiamo montare una directory locale come Data Volume utilizzando l'opzione `-v <host_dir>:<container_dir>`. In questo modo, la directory `<host_dir>` verrà montata nel container nella posizione `<container_dir>`.



ES:

Nel seguente esempio montiamo la cartella locale `mydir` nella posizione `adirectory` del container. Pertanto, ciò che viene scritto nella directory `adirectory` del container finirà nella directory `mydir` del file system locale, e viceversa.

```
1|docker run --rm -it -v "$PWD/mydir:/adirectory" ubuntu /bin/bash
```

N.B: la variabile `$PWD` rappresenta la directory corrente.

N.B: si consiglia di creare la directory locale in anticipo, altrimenti verrà creata con l'utente `root` come proprietario, rendendo difficile la scrittura in essa.

Per accedere ad Internet da un container, è possibile aprire una porta nel container e mapparla su una porta locale del Sistema Operativo.

A tale scopo, utilizziamo l'opzione `-p <local_port>:<container_port>`. In questo modo, la porta esposta dal container (`<container_port>`) sarà accessibile dall'Host attraverso la porta locale specificata (`<local_port>`).

Ad esempio, utilizzando `-p 8080:80`, la porta 80 del container sarà mappata sulla porta 8080 del computer Host. Quindi, se un servizio nel container è accessibile dalla porta 80, allora possiamo accederci dall'Host attraverso la porta 8080. In altre parole, se si accede alla porta 8080 del computer Host, si sta effettivamente accedendo alla porta 80 del container.

**ES** (Apache):

Per eseguire un container che ospita un server Apache, utilizziamo il seguente comando:

```
1| docker run -dit --name my-apache-app \
2|   -v "$PWD"/myweb:/usr/local/apache2/htdocs/ \
3|   -p 8080:80 httpd
```

Questo comando avvia un container come un processo Daemon (`-dit`) con il nome `my-apache-app`. Inoltre, monta la directory locale `myweb` sulla directory `/usr/local/apache2/htdocs/` all'interno del container (rappresenta la directory predefinita di Apache per servire le pagine web). Di conseguenza, il contenuto presente nella directory `myweb` sarà servito dal server Apache in esecuzione nel container.

In particolare, il server Apache risulta essere in esecuzione sulla porta 80 del container ed è accessibile dall'Host alla porta 8080.

Come abbiamo già osservato, ogni container deve eseguire un singolo servizio. Tuttavia, dobbiamo consentire l'interazione tra i servizi eseguiti su container distinti.

Consideriamo un'applicazione Java che necessita di un database MongoDB. In questo scenario, dobbiamo avviare un container per l'applicazione Java ed un altro per il server MongoDB.

Il problema principale è dovuto dal fatto che il container Java non può accedere alla porta aperta dal container MongoDB, poiché operano su reti differenti rispetto a quella locale.

N:B: un servizio in esecuzione su un container può dividersi in più processi. Tuttavia, non deve eseguire un altro servizio.

N:B:

MongoDB è un database NoSQL orientato ai documenti, diverso dai database relazionali che si basano su tabelle e relazioni.

Per risolvere il problema dell'interazione tra container che ospitano servizi diversi, utilizziamo i **Docker Networks** attraverso i seguenti passaggi:

1. Creare una rete Docker:

```
1| docker network create mynetwork
```

2. Eseguire i container all'interno della stessa rete, assegnandogli un nome. Il nome del container in esecuzione verrà automaticamente mappato all'indirizzo IP corrispondente nella rete. Ad esempio:

```
1| docker run --net=mynetwork --rm --name=mongodb mongo:4.2.3
```

Il nome del container (`mongodb`) viene automaticamente mappato al suo indirizzo IP nella rete.

3. Modificare il Dockerfile dell'applicazione Java per includere il nome `mongodb` nella connessione al server MongoDB.
4. Eseguire il container dell'applicazione Java nella stessa rete (`mynetwork`).

Cheatsheet for Docker CLI			
Run a new Container	Manage Containers	Manage Images	Info & Stats
Start a new Container from an Image <code>docker run IMAGE</code> <code>docker run nginx</code> ...and assign it a name <code>docker run --name CONTAINER IMAGE</code> <code>docker run --name web nginx</code> ...and map a port <code>docker run -p HOSTPORT:CONTAINERPORT IMAGE</code> <code>docker run -p 8080:80 nginx</code> ...and map all ports <code>docker run -P IMAGE</code> <code>docker run -P nginx</code> ...and start container in background <code>docker run -d IMAGE</code> <code>docker run -d nginx</code> ...and assign it a hostname <code>docker run --hostname HOSTNAME IMAGE</code> <code>docker run --hostname srv nginx</code> ...and add a dns entry <code>docker run --add-host HOSTNAME:IP IMAGE</code> ...and map a local directory into the container <code>docker run -v HOSTDIR:TARGETDIR IMAGE</code> <code>docker run -v ~/.:/usr/share/nginx/html nginx</code> ...but change the entrypoint <code>docker run -it --entrypoint EXECUTABLE IMAGE</code> <code>docker run -it --entrypoint bash nginx</code>	Show a list of running containers <code>docker ps</code> Show a list of all containers <code>docker ps -a</code> Delete a container <code>docker rm CONTAINER</code> <code>docker rm web</code> Delete a running container <code>docker rm -f CONTAINER</code> <code>docker rm -f web</code> Delete stopped containers <code>docker container prune</code> Stop a running container <code>docker stop CONTAINER</code> <code>docker stop web</code> Start a stopped container <code>docker start CONTAINER</code> <code>docker start web</code> Copy a file from a container to the host <code>docker cp CONTAINER:SOURCE TARGET</code> <code>docker cp web:/index.html index.html</code> Copy a file from the host to a container <code>docker cp TARGET CONTAINER:SOURCE</code> <code>docker cp index.html web:/index.html</code> Start a shell inside a running container <code>docker exec -it CONTAINER EXECUTABLE</code> <code>docker exec -it web bash</code> Rename a container <code>docker rename OLD_NAME NEW_NAME</code> <code>docker rename 096 web</code> Create an image out of container <code>docker commit CONTAINER</code> <code>docker commit web</code>	Download an image <code>docker pull IMAGE[:TAG]</code> <code>docker pull nginx</code> Upload an image to a repository <code>docker push IMAGE</code> <code>docker push myimage:1.0</code> Delete an image <code>docker rmi IMAGE</code> Show a list of all images <code>docker images</code> Delete dangling images <code>docker image prune</code> Delete all unused images <code>docker image prune -a</code> Build an image from a Dockerfile <code>docker build DIRECTORY</code> <code>docker build .</code> Tag an image <code>docker tag IMAGE NEWIMAGE</code> <code>docker tag ubuntu ubuntu:18.04</code> Build and tag an image from a Dockerfile <code>docker build -t IMAGE DIRECTORY</code> <code>docker build -t myimage .</code> Save an image to .tar file <code>docker save IMAGE > FILE</code> <code>docker save nginx > nginx.tar</code> Load an image from a .tar file <code>docker load -i TARFILE</code> <code>docker load -i nginx.tar</code>	Show the logs of a container <code>docker logs CONTAINER</code> <code>docker logs web</code> Show stats of running containers <code>docker stats</code> Show processes of container <code>docker top CONTAINER</code> <code>docker top web</code> Show installed docker version <code>docker version</code> Get detailed info about an object <code>docker inspect NAME</code> <code>docker inspect nginx</code> Show all modified files in container <code>docker diff CONTAINER</code> <code>docker diff web</code> Show mapped ports of a container <code>docker port CONTAINER</code> <code>docker port web</code>

10.1 Dockerfile

Dockerfile è un file di testo contenente istruzioni per la creazione e la costruzione di immagini Docker. Questo file segue la sintassi specifica di Dockerfile ed include comandi eseguibili dalla riga di comando all'interno di un container.

Il comando `build` utilizza il Dockerfile per eseguire le istruzioni e creare un'immagine. Durante la compilazione, viene fornito un **context**, che rappresenta il percorso sul filesystem locale oppure un URL di un repository Git. Tale context viene utilizzato per costruire l'immagine secondo le specifiche definite nel Dockerfile.

Nel Dockerfile, si ha accesso solo alle directory del context al momento della compilazione. La directory del context viene elaborata ricorsivamente, e tutto il suo contenuto viene inviato all'Host Docker durante la compilazione.

N.B.: è consigliabile utilizzare un file `.dockerignore` simile a `.gitignore` per escludere i percorsi non necessari e migliorare le prestazioni della compilazione.

N.B.: i percorsi nel file Dockerfile sono intesi come relativi al context, dove `/` rappresenta la radice del contesto. Non dobbiamo mai specificare `/` come context per la compilazione, altrimenti tutto il contenuto del disco rigido verrebbe inviato all'Host Docker.

Vediamo i principali comandi di Dockerfile:

- **FROM:** rappresenta la prima istruzione nel Dockerfile. Viene utilizzata per specificare l'immagine di base. Ad esempio:

```
1|FROM ubuntu
```

- **COPY:** copia più file sorgente dal context al filesystem del container nel percorso specificato. Ad esempio:


```
1|COPY .bash_profile /home
```

- ENV: imposta una variabile d'ambiente. Ad esempio:

```
1|ENV HOSTNAME=test
```

- RUN: esegue un comando specificato durante il processo di build. Ad esempio:

```
1|RUN apt-get update
```

- CMD: specifica il comando predefinito eseguito da un container quando viene avviato. Ad esempio:

```
1|CMD ["/bin/echo", "hello world"]
```

- EXPOSE: espone le porte di rete del container, ma non le rende automaticamente visibili. Ad esempio:

```
1|EXPOSE 8093
```



ES:

Esempio di creazione di un'immagine Docker per un'applicazione Java. In particolare, viene utilizzata un'immagine base di Ubuntu, si installa Java, si copia l'archivio JAR dell'applicazione nel container e si definisce il comando predefinito per eseguire l'applicazione Java all'avvio del container:

```
1|FROM ubuntu:bionic
2|
3|RUN apt-get update && \
4|    apt-get install -y openjdk-8-jk &&
5|    apt-get clean
6|
7|# Just to make sure java can now be run in the container.
8|RUN java --version
9|
10|COPY /target/hellodocker-0.0.1-SNAPSHOT.jar /app/app.jar
11|
12|CMD ["java", "-cp", "/app/app.jar", "com.examples.hellodocker.App"]
```

N.B.: il termine **Dockerizing** viene utilizzato per indicare il processo di creazione dell'immagine.

10.2 Docker Compose

Docker Compose è uno strumento di orchestrazione per i container Docker. L'approccio di Docker Compose consiste nel definire un singolo file di configurazione per i container che devono comunicare tra loro e specificare i parametri necessari per avviarli. Questo file Docker Compose, semplifica il processo di avvio e configurazione di tutti i container in modo automatico per favorire la comunicazione tra di essi. La configurazione viene definita utilizzando la sintassi YAML.

N.B: per avviare il file Docker Compose, dobbiamo posizionarci all'interno della cartella in cui si trova il file ed eseguire il comando `docker-compose up`.



ES:

Esempio di file `docker-compose.yaml` per la configurazione di due container, uno per l'applicazione Java e l'altro per il database MongoDB. Si definisce una Docker Network per far comunicare tra loro i due servizi:

```
1|version: '2'
2|
3|services:
4|  app: # Name of Java application's container.
5|    image: java-mongo-hello
6|    networks:
7|      - my_network
8|    depends_on:
9|      - mongodb
10|  mongodb: # Name of MongoDB server's container.
11|    image: mongo:4.4.3
12|    networks:
13|      - my-network
14|
15|networks:
16|  my-network:
17|    driver: bridge
```

Questo file Docker Compose definisce due servizi (`app` e `mongodb`), ciascuno con la propria immagine Docker. La sezione `networks` definisce una rete personalizzata chiamata `my_network`. Entrambi i servizi sono collegati a questa rete per consentire la comunicazione tra di essi. La direttiva `depends_on` specifica che il servizio `app` dipende dal servizio `mongodb`, garantendo che il database MongoDB venga avviato prima di eseguire l'applicazione Java.

10.3 Docker Hub

Per pubblicare un'immagine Docker su Docker Hub, è necessario creare un account e effettuare il login con l'username e la password associati all'account. Una volta creato l'account su Docker Hub, è possibile eseguire il push dell'immagine utilizzando i seguenti passaggi:

1. Effettuare il login su Docker Hub:

```
1|docker login
```

2. Creare la build dell'immagine Docker.

3. Eseguire il push dell'immagine specificando il nome utente e il tag dell'immagine:

```
1|docker push <user_id>/<image_name>
```

N.B: dobbiamo assicurarci che l'immagine sia stata costruita localmente sul computer prima di eseguire il push. L'uso di `<user_id>` ed `<image_name>` è fondamentale per associare l'immagine al proprio account su Docker Hub.

10.4 Docker su GitHub Actions

L'uso dei container Docker su GitHub Actions richiede alcune considerazioni in base al Sistema Operativo virtuale che intendiamo utilizzare:

- Nell'ambiente virtuale Linux fornito da GitHub Actions, l'utilizzo dei container è rapido e diretto, in quanto i container sono nativi di Linux.
- Nell'ambiente virtuale Windows fornito da GitHub Actions, è possibile utilizzare solo immagini Docker Windows. Potrebbe essere necessario configurare alcune impostazioni aggiuntive per adattare un workflow alle esigenze specifiche del progetto.
- Nell'ambiente virtuale macOS fornito da GitHub Actions, Docker non è previsto di default e deve essere installato ogni volta che viene eseguito un workflow. Pertanto, l'installazione di Docker è necessaria come parte del workflow di GitHub Actions. **N.B.**: l'esecuzione su macOS potrebbe richiedere più tempo rispetto ad altri sistemi operativi.

N.B.: le immagini Docker non vengono memorizzate nella cache su GitHub Actions. Perciò, ad ogni esecuzione di un container dobbiamo scaricare e installare tutte le immagini richieste.

 Create README.md Java CI with Maven and Docker in Linux #55: Commit f64f117 pushed by LorenzoBettini	master		23 minutes ago ... 52s
 Create README.md Java CI with Maven and Docker in Windows #47: Commit f64f117 pushed by LorenzoBettini	master		23 minutes ago ... 2m 54s
 Create README.md Java CI with Maven and Docker in macOS #7: Commit f64f117 pushed by LorenzoBettini	master		23 minutes ago ... 13m 41s

Figura 10.1: Differenti tempi di esecuzione di un container sui diversi ambienti virtuali offerti da GitHub Actions.

Capitolo 11

INTEGRATION TEST

Gli **Integration Test** rappresentano una componente importante nello sviluppo del software, poiché consentono di verificare l'integrazione tra i singoli componenti (già testati in modo isolato con gli Unit Test).

Ricapitolando, gli Unit Test si concentrano sulla verifica di un singolo SUT (System Under Test) isolato, simulando le dipendenze interne al componente. Il fallimento di uno Unit Test indica un potenziale problema nel singolo componente sotto test (se il test è scritto correttamente).

Gli Unit Test sono progettati per essere eseguiti in modo rapido.

Invece, gli Integration Test verificano l'integrazione tra almeno due componenti reali, coinvolgendo una delle seguenti situazioni:

- Un nostro componente (o più componenti) con almeno una dipendenza esterna reale.
- Due nostri componenti (o più componenti) che interagiscono insieme.

In genere, gli Integration Test verificano il corretto comportamento dei componenti quando interagiscono con le dipendenze reali, che erano state simulate durante gli Unit Test (attraverso il Mocking e lo Stubbing).

Gli Integration Test tendono ad essere più lenti rispetto agli Unit Test, poiché potrebbero richiedere l'attivazione e il corretto funzionamento di servizi esterni reali, aggiungendo un certo overhead al processo di testing.

La granularità dell'integrazione (cioè, quanti componenti devono essere testati insieme) dipende strettamente dall'applicazione in questione. Non è necessario testare tutte le possibili combinazioni dei componenti (soprattutto considerando che dovranno essere implementati anche gli End-to-End Test che verificano l'intera applicazione).

Inoltre, è importante ricordare la forma della Test Pyramid:

- Alla base, dobbiamo disporre di numerosi Unit Test (piccoli e veloci). Gli Unit Test devono coprire approfonditamente tutti i possibili percorsi (*path*) del flusso, con tempi di esecuzione nell'ordine dei millisecondi.
Per verificare tutti i possibili *path*, è necessario testare anche le condizioni al confine e le eccezioni. Tale obiettivo può essere facilmente raggiunto grazie all'uso del Mocking.
Adottando la metodologia TDD, si inizia scrivendo un test che ci aspettiamo fallisca.
- Successivamente, dobbiamo disporre di alcuni Integration Test con una maggiore granularità. Si prevede che gli Integration Test abbiano tempi di esecuzione mag-

giori rispetto agli Unit Test, poiché devono verificare l'Interazione tra due o più componenti reali.

Questi Integration Test si concentrano sui casi più interessanti, escludendo le condizioni al confine e le eccezioni (poiché sono già state verificate dagli Unit Test).

Inoltre, ci aspettiamo che gli Integration Test abbiano successo immediatamente, considerato che i singoli componenti sono già stati verificati in modo isolato attraverso gli Unit Test.

N.B: negli Integration Test è ancora consentito l'utilizzo del Mocking per simulare alcune dipendenze, ma almeno due componenti devono essere reali.

- In cima alla piramide, dobbiamo avere solo pochi End-to-End Test, che possono richiedere tempi di esecuzione più lunghi e possono essere più complessi da implementare.

Gli Unit Test e gli Integration Test dovrebbero essere organizzati separatamente (ad esempio, in due cartelle distinte) al fine di poterli eseguire in modo isolato.

Infatti, tipicamente vogliamo eseguire gli Unit Test in modo continuo durante lo sviluppo del software. Mentre, vogliamo eseguire gli Integration Test solamente quando gli Unit Test hanno esito positivo.

Di solito, gli Integration Test vengono eseguiti come parte del processo di Continuous Integration per verificare che le singole unità di codice interagiscano correttamente.

Nel ciclo di vita `default` di Maven, sono previste quattro fasi specifiche per gli Integration Test:

1. `pre-integration-test`: prepara la configurazione degli Integration Test (ad esempio, avvia i servizi necessari).
2. `integration-test`: esegue gli Integration Test.
3. `post-integration-test`: ripulisce la configurazione effettuata nella fase `pre-integration-test` (ad esempio, interrompe i servizi).
4. `verify`: verifica che gli Integration Test siano andati a buon fine.

Queste fasi vengono eseguite dopo la fase `package`, poiché gli Integration Test potrebbero dipendere dagli artefatti (ad esempio, potrebbero richiedere un file JAR o un'immagine Docker). Infatti, gli Integration Test possono richiedere l'avvio di container Docker basati su un'immagine costruita durante il processo di `package`. Inoltre, gli Integration Test sono spesso eseguiti dopo il deploy dell'applicazione su un server applicativo (ad esempio, Tomcat).

Come abbiamo visto, Maven utilizza il plug-in **Surefire** per gli Unit Test. Tale plug-in è associato alla fase `test` del ciclo di vita `default`. In questa fase, dovrebbero essere eseguiti unicamente gli Unit Test.

Invece, per gli Integration Test Maven dispone di un apposito plug-in chiamato **Failsafe**. Questo plug-in non è configurato né abilitato per impostazione predefinita, poiché Maven gestisce gli Integration Test in fasi specifiche (può essere attivato in una delle quattro fasi previste da Maven per l'Integration Test).

Il nome Failsafe deriva dal fatto che la compilazione non è destinata a fallire immediatamente se uno o più Integration Test non vanno a buon fine (a differenza di Surefire).

Infatti, se la compilazione dovesse interrompersi non appena un Integration Test fallisce durante la fase `integration-test`, i servizi avviati nella fase `pre-integration-test` rimarrebbero in esecuzione. Invece, la compilazione effettiva fallisce solo nella fase `verify`, che viene eseguita dopo `post-integration-test`. In questo modo, nel `post-integration-test`, è possibile fermare e pulire le risorse allocate durante la fase `pre-integration-test`.

A differenza di Surefire, il plug-in Failsafe non è abilitato di default e richiede una configurazione esplicita. Ecco un esempio di configurazione del plugin Failsafe nel file `pom.xml`:

```
1 <plugin>
2   <groupId>org.apache.maven.plugins</groupId>
3   <artifactId>maven-failsafe-plugin</artifactId>
4   <version>2.22.1</version>
5   <executions>
6     <execution>
7       <goals>
8         <goal>integration-test</goal>
9         <goal>verify</goal>
10      </goals>
11    </execution>
12  </executions>
13</plugin>
```

Il goal `integration-test` è associato di default alla fase `integration-test`. Invece, il goal `verify` è associato alla fase `verify`.

Le fasi `pre-integration-test` e `post-integration-test` sono spesso utilizzate da altri plug-in (ad esempio, il plug-in Maven Docker) per avviare ed arrestare i servizi necessari agli Integration Test.

N.B: durante l'esecuzione del goal `verify`, il plug-in Maven Failsafe verifica i report generati durante la fase di `integration-test` per determinare se alcuni test sono falliti. In caso di fallimento di alcuni test, questo goal arresterà la build.

Il plug-in Maven Failsafe identifica automaticamente le classi da considerare come Integration Test utilizzando un pattern definito (ad esempio, `**/IT*.java`, `**/*IT.java` oppure `**/*ITCase.java`) e le esegue di conseguenza. Questo ci consente di eseguire in modo isolato sia gli Unit Test che gli Integration Test, poiché Failsafe esclude implicitamente le classi eseguite da Surefire e viceversa.



ES:

Implementiamo un'architettura **Model-View-Controller (MVC)**, tipicamente utilizzata per lo sviluppo di interfacce utente. Tale architettura suddivide il programma in tre componenti interconnessi, con l'obiettivo di separare le rappresentazioni interne dell'informazione dai modi in cui l'informazione viene presentata all'utente. I tre elementi chiave sono:

- **Model:** gestisce i dati raccolti, spesso rappresentati attraverso un Repository.
- **View:** responsabile della presentazione dell'informazione all'utente.

- **Controller**: svolge il ruolo di mediatore tra il Model e la View, gestendo la logica di controllo e coordinando le interazioni tra i due.

Questa suddivisione consente una progettazione più modulare e una maggiore facilità di manutenzione nel processo di sviluppo del software. Inoltre, questa suddivisione ci consente di testare ogni singolo componente in modo isolato, utilizzando la metodologia Test-Driven Development (TDD) e gli Unit Test. Successivamente, sarà possibile verificare la loro integrazione attraverso gli Integration Test.

In realtà, implementiamo una variante del Model-View-Controller (MVC), conosciuta come **Model-View-Presenter (MVP)**. In questa architettura, il **Presenter** assume il ruolo di intermediario tra il Model e la View, occupandosi della logica di presentazione. In particolare, il Presenter recupera i dati dal Model e li elabora per la visualizzazione nella View.

N.B.: per semplificare, faremo riferimento al Presenter come Controller.

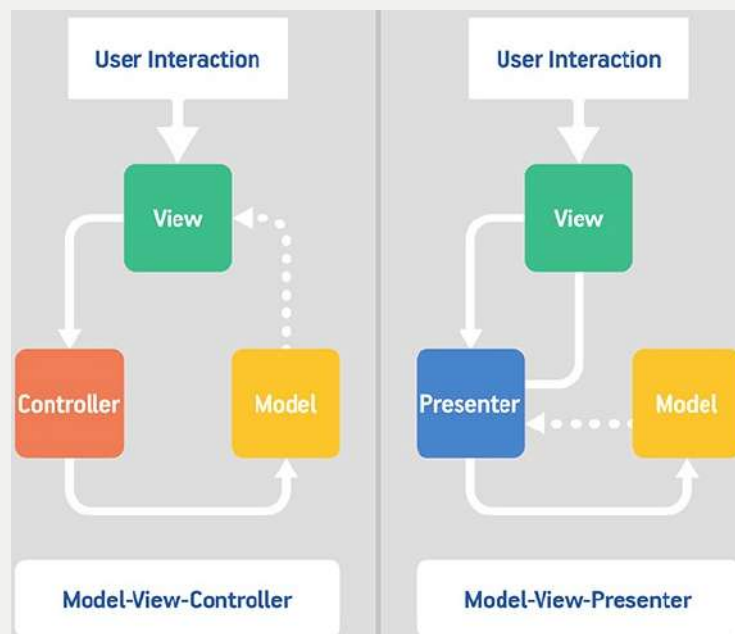


Figura 11.1: Confronto tra l'architettura Model-View-Controller (MVC) e l'architettura Model-View-Presenter (MVP). Nel design pattern MVC, il Controller può essere condiviso tra diverse View e la comunicazione avviene direttamente tra le View ed il Model. Invece, nel design pattern MVP, la View è separata dal Model ed il Presenter funge da intermediario tra i due. Questo design pattern semplifica la scrittura di Unit Test e stabilisce una mappatura diretta tra View e Presenter, permettendo l'uso di Presenter multipli per gestire View complesse. I vantaggi dell'architettura MVP emergono soprattutto nella possibilità di implementare in modo autonomo il Presenter, facilitando la manutenzione e agevolando la verifica delle logiche di interazione.

Nel nostro esempio, gli oggetti Model sono rappresentati da istanze della classe **Student**. Questi oggetti possono essere memorizzati e recuperati in un database MongoDB.

Gli oggetti Model sono presentati all'utente attraverso una View, dalla quale possono essere manipolati, inseriti e rimossi dall'utente.

Il Controller, elabora le richieste provenienti dall'utente attraverso la View e delega le operazioni di lettura e scrittura nel database al Model (implementato mediante il pattern Repository). Quindi, i risultati ottenuti vengono presentati all'utente attraverso la View.

N.B.: il Repository (cioè il Model) e la View vengono implementati come interfacce astratte.

N.B.: il Repository e la View sono iniettati nel Controller (cioè il Controller ha riferimenti sia al Model che alla View poiché sta nel mezzo). Questa struttura permette una migliore separazione delle responsabilità e facilita il testing. In particolare, possiamo sviluppare il Controller utilizzando la metodologia TDD, in cui simuliamo l'implementazione del Repository e della View attraverso il Mocking. In questo modo, possiamo implementare il Controller anche senza conoscere le implementazioni concrete delle interfacce del Repository e della View.

La View rappresenta un'interfaccia astratta che interagisce con il Controller (pertanto deve avere un riferimento ad esso). L'implementazione della View chiamerà i metodi del Controller precedentemente iniettato.

Il Repository è un'interfaccia astratta che viene implementata utilizzando l'API Java di MongoDB per le operazioni di inserimento e recupero dei dati. In particolare, la connessione con il server MongoDB viene stabilita utilizzando il seguente codice:

```
1 MongoClient mongoClient = new MongoClient(mongoHost);
2 MongoDBDatabase db = mongoClient.getDatabase("school");
3 MongoCollection<Document> collection = db.getCollection("students");
```

Quindi, l'implementazione per l'inserimento ed il recupero dei dati dal database avviene utilizzando il seguente codice:

```
1 Document doc = new Document("id", studentId)
2               .append("name", "A student");
3 collection.insertOne(doc);
4
5 String retrievedId = collection.find().first().get("id");
```

In questo contesto, il Repository si basa sull'utilizzo di un MongoClient per gestire le operazioni di inserimento e recupero nel database MongoDB.

Per testare l'implementazione del Repository, è possibile fare il Mocking del MongoClient. Tuttavia questa soluzione risulta essere molto inefficiente e complessa da implementare.

Osserviamo che il Repository non ha una logica reale, poiché deve eseguire operazioni di inserimento e recupero di dati dal database. Pertanto, il Mocking del database renderebbe il test dell'implementazione del Repository inutile, poiché non verificherebbe la corretta persistenza ed il recupero dei dati o la correttezza delle query.

Invece, è fondamentale utilizzare un vero database nei test del Repository per verificare che gli oggetti vengano inseriti e recuperati correttamente e

che le query siano formulate correttamente. Ricordiamo che non possiamo fare il mock di tipi che non possediamo.

La nostra test fixture è costituita dalla SUT (cioè, la classe `StudentMongoRepository`) e dallo stato del database. Prima di eseguire ogni test, è fondamentale che il database si trovi in uno stato ben definito e conosciuto. Nel nostro esempio, il database deve essere vuoto prima di ogni test.

Se desideriamo evitare l'interazione con un vero database negli Unit Test, possiamo adottare un'implementazione simile al Mocking utilizzando una soluzione **in-memory** di MongoDB (ad esempio, MongoDB Java Server). Per utilizzare questa soluzione, avviamo un server MongoDB in-memory prima di eseguire i test. Quindi, prima di ogni singolo test, creiamo un `MongoClient` connesso al server in-memory e lo passiamo ad uno `StudentMongoRepository`, mantenendo anche un riferimento diretto alla collezione.

In questo modo, possiamo inserire documenti noti nel database quando testiamo i meccanismi di lettura del nostro Repository. Inoltre, possiamo verificare la presenza dei documenti attesi nel database quando testiamo i meccanismi di scrittura del nostro Repository.

L'utilizzo di un database in-memory ed il Mocking rappresentano due approcci distinti nel testing. Con il Mocking, simuliamo e facciamo lo Stubbing dei metodi astratti e delle dipendenze, presupponendo che le implementazioni finali siano corrette. In questo caso, è responsabilità dello sviluppatore garantire che le dipendenze siano correttamente implementate. D'altra parte, l'utilizzo di un database in-memory implica l'uso di un'implementazione fittizia e meno potente del database, che può differire dall'implementazione del database reale. Infatti, i database in-memory generalmente non supportano tutte le funzionalità ed i meccanismi presenti nel database reale.

I nostri test potrebbero sembrare positivi quando utilizziamo un database in-memory, ma potrebbero fallire quando utilizziamo successivamente un database reale.

N.B: tipicamente, quando si implementano e si testano operazioni CRUD o query semplici, un database in-memory può essere una buona soluzione. Tuttavia, spesso dobbiamo includere test per scenari complessi che devono coinvolgere il database reale. Pertanto, risulta più efficace scrivere i test utilizzando il database reale.

In generale, la pratica consigliata è quella di scrivere i test utilizzando il database reale fin dall'inizio, evitando l'uso di un database in-memory. Pur essendo tecnicamente Integration Test, possiamo considerarli come Unit Test, in quanto stiamo implementando un componente di una libreria di terze parti che rappresenta il diretto interessato. Infatti, questo approccio viene adottato solo quando testiamo lo `StudentMongoRepository`, il quale rappresenta il componente che interagisce direttamente con il database. Nel

resto dell'applicazione, ci affidiamo all'astrazione dall'implementazione reale mediante l'utilizzo dell'interfaccia **StudentRepository**. In questo modo, simuliamo il comportamento di **StudentRepository** per testare le altre parti dell'applicazione.

Analogamente, per implementare gli Integration Test e verificare l'integrazione tra le singole unità, potremmo pensare di utilizzare un database in-memory perché è meno pesante di installare un vero e proprio server MongoDB sul computer. Tuttavia, possiamo sfruttare la potenza dei container Docker. Iniziamo quindi a scrivere gli Integration Test, eseguendo effettivamente un'istanza di MongoDB all'interno di un contenitore Docker.

Una pratica comune è mantenere separati gli Integration Test dagli Unit Test. Per fare ciò, possiamo includere il termine **IT** nel nome del file Java (convenzione del plug-in Maven Failsafe). Inoltre, possiamo mantenere questi file di test in una cartella separata (ad esempio, **src/it/java**).

Adesso, dobbiamo gestire l'avvio del container Docker con MongoDB durante l'esecuzione degli Integration Test. Abbiamo due alternative:

- Utilizziamo la libreria di testing **Testcontainers**. In particolare, configuriamo i test in modo tale che, prima di essere eseguiti, venga avviato un container Docker con un'istanza di MongoDB. Alla fine dei test, il container deve essere fermato.
- Configuriamo la build Maven con il plug-in **Fabric8**. In particolare, nella fase di **pre-integrazione-test** tale plug-in deve avviare un container Docker con MongoDB. Nella fase di **post-integrazione-test**, tale plug-in deve interrompere il container.

N.B.: quando mappiamo la porta del container Docker con la porta del computer locale, evitiamo di assegnare una porta fissa all'Host, poiché potrebbe essere occupata da altri servizi. Invece, dobbiamo mappare la porta del container Docker con una porta casuale del computer, la quale cambierà ad ogni esecuzione. Questa pratica è estremamente importante nei server di Continuous Integration (CI Server), dove non possiamo garantire la disponibilità di porte specifiche e occorre evitare conflitti. Se viene utilizzata una porta già occupata, i test potrebbero fallire.

Capitolo 12

UI TEST

Gli **User Interface Test (UI Test)** sono progettati per verificare il corretto funzionamento dell'interfaccia utente di un'applicazione. Questi test possono essere applicati a diversi tipi di applicazioni, tra cui applicazioni da command line, applicazioni con interfaccia grafica utente (GUI) ed applicazioni web. In particolare, ci concentriamo sulla verifica e sull'implementazione di un'interfaccia grafica (GUI) utilizzando il framework **Java Swing**.

Gli UI Test devono verificare l'interazione dell'utente con l'interfaccia grafica (GUI). Ad esempio:

- Quando l'utente esegue un clic su un pulsante, ci si aspetta che si verifichi un comportamento specifico.
- Quando alcuni campi non sono compilati correttamente, alcune parti dell'interfaccia utente (ad esempio, i pulsanti) devono essere disattivate.

. In generale, dovremmo verificare che lo stato dell'interfaccia utente cambi in base ad alcune interazioni specifiche dell'utente.

Gli UI test e gli End-to-End Test sono concettualmente ortogonali. Infatti, gli End-to-End Test possono essere considerati UI Test poiché verificano l'intera applicazione e la sua interfaccia utente. Tuttavia, il contrario non è necessariamente vero. Infatti, l'interfaccia utente può essere testata anche attraverso Unit Test, isolandola dalle sue dipendenze.

N:B: negli Unit Test dell'interfaccia utente, è comune fare il Mocking delle dipendenze.

Tipicamente, gli UI Test sono difficili da scrivere a causa della natura event-driven delle interfacce grafiche utente (GUI). Tuttavia, è possibile semplificare questo compito utilizzando dei framework di test dedicati, come **AssertJ Swing**. Questo framework fornisce un'API che permette di simulare gli eventi dell'utente ed accedere ai controlli della GUI.

N.B: gli UI Test sono intrinsecamente più lenti a causa dell'overhead introdotto dalla GUI durante l'esecuzione dei test.

AssertJ Swing è un framework di test che facilita la scrittura di test per interfacce utente implementate con Java Swing. Questo strumento consente di simulare l'interazione dell'utente con una GUI (consentendo azioni come inserimento di testo, clic, trascinamento, ecc...). Inoltre, consente di ricercare i componenti all'interno della GUI (la ricerca può essere fatta per tipo, per nome o con criteri personalizzati).

AssertJ Swing fornisce un supporto completo per tutti i componenti Swing inclusi nel JDK. La sua API agevola la creazione e la manutenzione di test GUI.

Inoltre, AssertJ Swing garantisce che le operazioni svolte sull'interfaccia utente siano gestite correttamente, evitando problemi legati al threading.

Infine, AssertJ Swing consente di generare screenshot dei test GUI che falliscono, semplificando l'analisi e la comprensione degli errori.

N.B.: dobbiamo rimuovere gli screenshot dal repository Git, includendo la cartella nel file `.gitignore`. Se i test vengono eseguiti su un CI Server, dobbiamo rendere disponibili gli screenshot per il download.



ES:

Consideriamo il precedente esempio di Integration Test, in cui abbiamo implementato uno `StudentController` ed un `StudentMongoRepository`.

Adesso ci concentriamo sull'implementazione dell'interfaccia `StudentView` mediante la creazione di uno `StudentSwingView`. Questa implementazione seguirà l'approccio TDD, utilizzando il framework AssertJ Swing per semplificare la scrittura e l'esecuzione dei test degli UI Test.

N.B.: essendo `StudentView` un'interfaccia astratta, la modularità dell'architettura MVC ci consente di implementare diverse versioni grafiche (come ad esempio un'interfaccia a riga di comando o una interfaccia web) senza la necessità di modificare il Controller e il Repository.

La scrittura di codice Java per implementare una GUI, può risultare complessa e richiedere molto tempo. Per semplificare questo processo si consiglia l'utilizzo di un editor visuale, come **WindowBuilder** (disponibile per Eclipse).

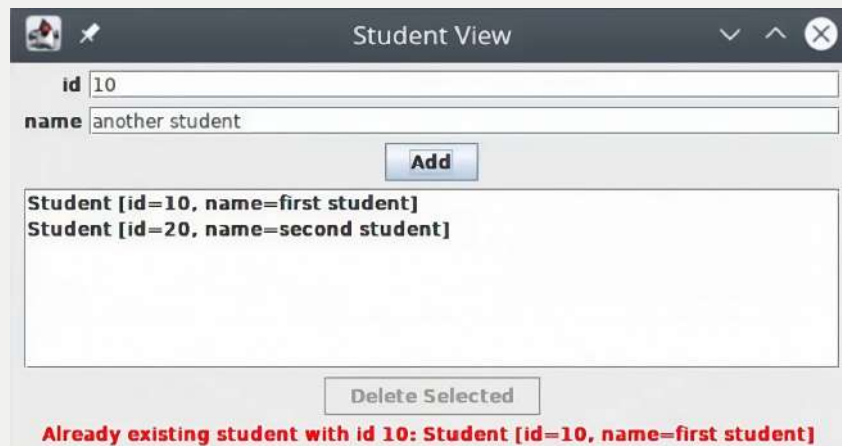
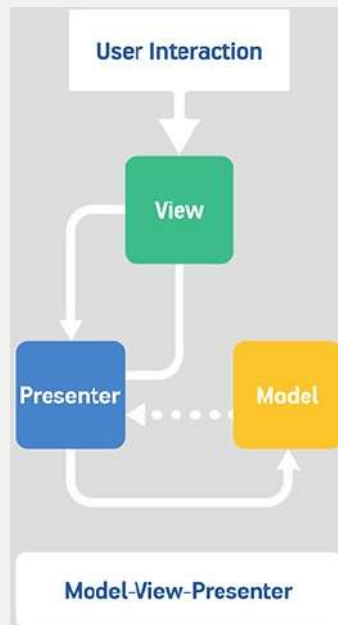


Figura 12.1: Forma finale della nostra GUI, ottenuta mediante l'implementazione di `StudentSwingView`.

L'interazione dell'utente con l'applicazione avviene attraverso la View, che permette di eseguire operazioni specifiche (come l'inserimento di un nuovo studente o la rimozione di uno studente esistente). La View, a sua volta, delega queste richieste al Controller. Quindi, il Controller è responsabile di elaborare le richieste dell'utente, delegando le operazioni necessarie per interagire con il database al Repository. Una volta completate le operazioni

sul database, il Controller restituisce i risultati alla View, che si occuperà di presentarli all'utente richiamando i metodi appropriati. Questa catena di interazioni tra View, Controller e Repository consente una gestione efficiente delle operazioni utente e delle operazioni sul database, mantenendo una separazione delle responsabilità all'interno dell'architettura MVC.



Requisiti funzionali:

- Il pulsante “Add” deve essere attivo solo quando entrambi i campi di testo sono compilati. Quando viene premuto il pulsante “Add”, questo deve chiamare il metodo `newStudent` del Controller, passando un oggetto `Student` con i valori specificati nei campi di testo.
- Il pulsante “Delete Selected” deve essere attivo solo quando uno studente è selezionato nell’elenco. Quando viene premuto il pulsante “Delete Selected”, questo deve chiamare il metodo `deleteStudent` del Controller.
- Il metodo `showError` dell’interfaccia View deve visualizzare il messaggio di errore e le informazioni sullo studente nell’apposita etichetta in fondo alla GUI.
- I metodi `studentAdded` e `studentRemoved` dell’interfaccia View devono aggiungere e rimuovere rispettivamente lo studente dall’elenco. Entrambi i metodi devono anche disabilitare l’etichetta di errore.

L’implementazione di queste funzionalità di basso livello nella View seguirà una metodologia TDD, con la scrittura iniziale di Unit Test utilizzando AssertJ Swing. Durante questa fase, il Controller sarà sostituito da un oggetto simulato con il Mocking.

Requisiti di alto livello:

- Il pulsante “Add” consente all’utente di inserire un nuovo studente nel database, utilizzando i valori specificati nei campi di testo. Se l’operazione ha successo, l’elenco visualizzato viene aggiornato per riflettere l’aggiunta dello studente.
- Il pulsante “Delete Selected” permette all’utente di eliminare dal database lo studente attualmente selezionato nell’elenco. Se l’operazione ha successo, l’elenco visualizzato viene aggiornato per rimuovere lo studente selezionato.
- Gli errori derivanti da operazioni di aggiunta o eliminazione, vengono mostrati nella parte inferiore della vista per informare l’utente sulle eventuali problematiche.

La verifica di queste caratteristiche di alto livello avverrà attraverso l’esecuzione di Integration Test.

N.B.: come descritto dalla Test Pyramid, scriveremo numerosi Unit Test (uno per ogni possibile scenario della GUI) ed alcuni Integration Test (non dovranno verificare nuovamente i dettagli interni della GUI, come lo stato di abilitazione/disabilitazione dei pulsanti).

Iniziamo lo sviluppo della GUI a partire dagli Unit Test. A tale scopo, applichiamo il TDD alle specifiche di implementazione della GUI attraverso tre fasi:

- Test per i controlli della GUI: verifichiamo che i controlli siano parte integrante della GUI. Inoltre, verifichiamo che essi siano abilitati/disabilitati correttamente in base alle condizioni specificate. In particolare, verifichiamo che il pulsante “Add” sia abilitato solo quando entrambi i campi di testo non sono vuoti. Inoltre, verifichiamo che il pulsante “Add” sia disabilitato quando almeno un campo di testo è vuoto.
- Test per l’implementazione dell’interfaccia **StudentView**: verifichiamo che i metodi implementati dell’interfaccia **StudentView**, quando vengono chiamati, apportino le modifiche previste alla GUI. In particolare, se viene chiamato il metodo **studentAdded(Student)**, allora lo studente passato deve apparire nell’elenco. Invece, se viene chiamato il metodo **studentRemoved(Student)**, allora lo studente passato deve essere rimosso dall’elenco.
N.B.: in questa fase, non ci interessa sapere chi chiamerà effettivamente questi metodi (ad esempio, il Controller).
- Test per la logica dell’UI: verifichiamo che il clic sui pulsanti corrisponda all’invocazione corretta dei metodi sul Controller simulato.

In particolare, se viene premuto il pulsante “Add”, allora deve essere richiamato il metodo “newStudent” del Controller (simulato con il Mocking) passando un oggetto “Student” con i valori specificati nei campi di testo. Invece, quando viene premuto il pulsante “Delete Selected”, non dobbiamo verificare che lo studente venga aggiunto all’elenco della View. Infatti, la View si limita a delegare i compiti al Controller, supponendo che quest’ultimo funzioni correttamente (la correttezza del metodo `studentAdded` viene verificata in un altro test). Inoltre, non è necessario verificare se lo studente viene aggiunto al database. Infatti, la View delega il Controller, che a sua volta delega il Repository.

N.B: questi scenari dovranno essere verificano con gli Integration Test.

Adesso, vediamo come implementare gli Integration Test per le interfacce utente, concentrandoci solo su alcuni casi interessanti:

- Interazione tra la View ed il Controller utilizzando un Repository simulato: quando vengono inseriti un ID ed un nome nei campi di testo e si fa clic sul pulsante “Add”, lo studente aggiunto dovrebbe apparire nell’elenco. Questo test verifica l’integrazione corretta tra la View (`StudentSwingView`) e il Controller (`SchoolController`), senza considerare lo stato del database. Per questo motivo, possiamo simulare l’interfaccia `StudentRepository` con il Mocking o implementando un database in-memory (poiché lo stato del database è già stato testato in modo isolato con gli Unit Test).

La test fixture comprende tutti e tre i componenti del programma, ma il SUT è ancora la View. Infatti, stiamo verificando il comportamento della View quando viene integrata con gli altri componenti reali del programma.

- Interazione tra la View ed il Controller utilizzando un Repository reale: quando vengono inseriti un ID ed un nome nei campi di testo e si fa clic sul pulsante “Add”, lo studente aggiunto dovrebbe essere presente nel database. Questo test verifica la corretta implementazione del pattern MVC.

In questo contesto, i casi interessanti sono rappresentati soltanto dai metodi di test che riguardano l’aggiunta di uno studente e la rimozione di uno studente selezionato. Il SUT è costituito dalla View, dal Controller e dal Repository.

N.B: questo test ha somiglianze con un End-to-End Test. Tuttavia, mentre negli End-to-End Test si interagisce solo con l’interfaccia utente dell’applicazione, in questo caso stiamo utilizzando tutti i componenti (cioè, sia l’interfaccia utente che i componenti interni).

N.B: non ha senso scrivere Integration Test per scenari che coinvolgono esclusivamente la View. Ad esempio, evitiamo di scrivere Integration Test

per l'abilitazione dei pulsanti, poiché queste situazioni sono già state testate negli Unit Test della vista.

N.B: non è necessario scrivere numerosi Integration Test. In generale, l'obiettivo è mantenere un numero di Integration Test inferiore rispetto agli Unit Test, concentrandosi sugli scenari critici che coinvolgono l'interazione tra le componenti del sistema.

I framework per l'UI Test non consentono di verificare se l'aspetto dell'interfaccia utente è esteticamente piacevole. Infatti, se ci sono modifiche nel codice che influiscono sulla disposizione dei componenti all'interno della GUI, i test potrebbero continuare ad essere positivi anche se il layout è stato alterato. La valutazione dell'estetica dell'interfaccia utente rimane un compito che richiede l'intervento umano, poiché non può essere automatizzato facilmente. Tuttavia, il comportamento corretto dell'interfaccia utente può essere testato in modo automatico, seguendo i principi del TDD.

Tipicamente, gli UI Test vengono eseguiti nei server di Integrazione Continua (CI) a causa del loro costo temporale significativo. Su GitHub Actions, è possibile eseguire gli UI Test negli ambienti virtuali Windows e macOS, ma non direttamente nell'ambiente virtuale Linux fornito da GitHub Actions (poiché la macchina virtuale Linux non dispone di un ambiente grafico).

In particolare, durante l'esecuzione degli UI Test nell'ambiente virtuale Linux di GitHub Actions, sorge un problema specifico. Infatti, AssertJ Swing richiede un ambiente grafico, rappresentato in Linux dal server di visualizzazione X11 (non installato di default nella macchina virtuale Linux di GitHub Actions).

Tuttavia, possiamo risolvere questo problema avviando un display virtuale X come **Xvfb**, cioè un'implementazione del protocollo X11 che consente operazioni grafiche senza uno schermo fisico. A tale scopo, GitHub Actions fornisce il programma **xvfb-run**, che consente di eseguire un comando in un server virtuale X (dobbiamo prefissare la build Maven con **xvfb-run**)

Per le macchine virtuali di Windows e macOS, non è necessario alcuno strumento aggiuntivo poiché l'ambiente grafico è già installato e la compilazione di Maven può essere eseguita direttamente.

12.1 Multi-threading

In generale, anche se la concorrenza o il multithreading non sono esplicitamente necessari, quando si utilizza un framework per implementare la GUI (come Swing) dobbiamo gestire implicitamente almeno due thread. Uno è il thread principale dell'applicazione e l'altro è il thread utilizzato dalla GUI. Anche se la concorrenza non è richiesta, è essenziale eseguire tutte le operazioni che modificano la GUI all'interno del thread opportuno.

Swing introduce questa necessità a causa della gestione degli eventi (tutti gli eventi devono essere gestiti nel thread dell'interfaccia utente). Se non rispettiamo questo requisito si potrebbero verificare comportamenti imprevisti o eccezioni. AssertJ Swing offre un'API specifica per verificare l'uso corretto dei thread durante gli UI Test (ad esem-

pio, possiamo verificare che un test fallisca se proviamo a modificare l'interfaccia utente nel thread sbagliato).

Tuttavia, l'introduzione di multi-threading può portare a non determinismo nei test, il che rappresenta una sfida. Infatti, gli Automated Test devono essere ripetibili e deterministici.

L'utilizzo di timeout nei test può contribuire a renderli più deterministici, ma è importante evitare attese eccessive che potrebbero rallentare inutilmente l'esecuzione dei test.

Capitolo 13

END-TO-END TEST

Ricapitolando, gli Unit Test testano le singole unità di un programma (come classi o metodi). Questi test mirano a verificare il corretto funzionamento di una singola unità in isolamento, senza dipendere da altri componenti. A tale scopo, sostituiscono i componenti esterni con *test double* (ad esempio, mock).

Invece, gli Integration Test combinano le singole unità per garantire che funzionino come previsto quando sono integrate. In particolare, testano la collaborazione di almeno due componenti reali, supponendo che le singole unità siano già state testate in modo isolato con gli Unit Test.

Gli **End-to-End** Test, noti anche come **System Test**, verificano l'intera applicazione attraverso la sua interfaccia utente (UI). Tali test verificano che tutti i componenti interagiscano correttamente (rappresentano una forma avanzata di Integration Test, in cui tutti i componenti del sistema sono integrati insieme).

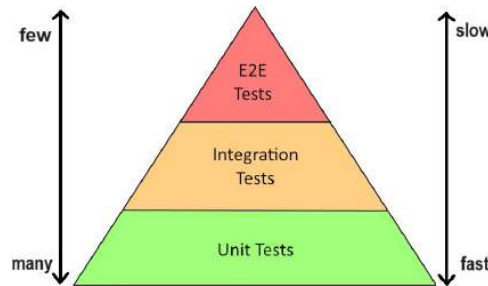
Il SUT (System Under Test) rappresenta il componente soggetto al test. Il tipo di SUT varia in base al tipo di test effettuato:

- Negli Unit Test, il SUT è rappresentato da una singola classe Java. Pertanto, invochiamo i metodi su un'istanza della classe.
- Negli Integration Test, il SUT è rappresentato da una singola entità che interagisce con altri componenti oppure da diverse istanze di classi.
- Negli End-to-End Test, il SUT è rappresentato dall'intera applicazione. Pertanto, interagiamo con l'interfaccia utente dell'applicazione.

Ricordiamo la struttura della Test Pyramid:

- Alla base, dobbiamo sviluppare numerosi Unit Test, caratterizzati da dimensioni ridotte e tempi di esecuzione nell'ordine dei millisecondi. Gli Unit Test devono essere facili da scrivere e devono concentrarsi su singole unità di codice. Gli Unit Test sono tipicamente test **white-box**, basati su una conoscenza dettagliata della logica interna del componente.
- Successivamente, dobbiamo implementare alcuni Integration Test con una granularità maggiore, accettabilmente più lenti rispetto agli Unit Test. Gli Integration Test possono leggermente più complessi da scrivere e coinvolgono la connessione di componenti diversi per verificarne l'interazione.
- In cima, occorre scrivere solo un numero limitato di End-to-End Test, che operano ad un livello più alto e richiedono più tempo per essere eseguiti. Gli End-to-End Test possono risultare i più complessi da scrivere e coinvolgono l'interazione

con l'interfaccia utente dell'applicazione, tralasciando i dettagli interni. Gli End-to-End Test sono tipicamente test **black-box**, ignorando i dettagli di basso livello (osserviamo solamente gli input ed i relativi output).



Gli Unit Test costituiscono la base della piramide. Infatti, potrebbe non avere senso scrivere Integration Test ed End-to-End Test per componenti che non sono stati precedentemente testati in modo isolato con Unit Test.

Gli Integration Test e gli End-to-End Test dovrebbero verificare solo il comportamento di componenti già testati in modo isolato. Pertanto, ci aspettiamo che abbiano successo fin dall'inizio (al contrario, gli Unit Test sviluppati con l'approccio TDD sono progettati per fallire inizialmente).

Gli Integration Test e gli End-to-End Test non mirano a coprire tutti i possibili cammini (*path*), ma piuttosto si concentrano sui casi rilevanti che dipendono dall'applicazione. Questo perché i casi limite e le eccezioni dovrebbero essere già state verificate dagli Unit Test, poiché in essi è più semplice ricreare questi contesti (sfruttando tecniche come il Mocking e lo Stubbing). Ricostruire tali scenari negli Integration Test e negli End-to-End Test potrebbe risultare piuttosto complesso.

Se si apportano modifiche all'implementazione di singoli componenti, è improbabile che gli Integration Test e gli End-to-End Test falliscano, a meno che l'interfaccia esterna non subisca modifiche significative. Invece, se si apporta una piccola modifica ad un singolo componente, ci si aspetta che almeno uno Unit Test fallisca (in caso contrario, si potrebbero verificare problemi).

Gli UI Test e gli End-to-End Test sono concetti ortogonali. Gli End-to-End Test sono generalmente riconducibili agli UI Test, in quanto valutano l'intera applicazione come una black-box, concentrandosi sull'interfaccia utente. Tuttavia, l'opposto non è sempre vero. Infatti, è possibile testare l'interfaccia utente attraverso Unit Test, mantenendola isolata dalle sue dipendenze.

Gli End-to-End Test rappresentano un tipo speciale di Integration Test, in cui tutti i componenti del sistema vengono integrati. Tuttavia, a differenza degli Integration Test, in cui la test fixture include poche istanze delle nostre classi connesse manualmente, negli End-to-End Test eseguiamo i test direttamente sull'intera applicazione in esecuzione, interagendo con l'interfaccia utente. Questo significa che, in generale, negli End-to-End Test non è necessario invocare direttamente il codice Java della nostra applicazione. Invece, interagiamo esclusivamente attraverso l'interfaccia utente verificando che l'applicazione si comporti correttamente come una black-box (cioè, ci comportiamo come utenti finali che interagiscono con l'UI senza conoscere l'implementazione interna).



ES:

Consideriamo il precedente esempio, in cui abbiamo già implementato la GUI e testato l'applicazione sia con Unit Test che Integration Test. Per semplicità, escludiamo le funzionalità di multithreading.

L'idea principale è avviare la nostra applicazione, interagire con la sua finestra Swing utilizzando AssertJ Swing e verificare solo ciò che farebbe l'utente.

Pertanto, abbiamo bisogno di una classe con un metodo `main`, in quanto AssertJ Swing supporta l'avvio di un'applicazione dal suo metodo `main`.

N.B: la classe `main` dovrebbe essere esclusa dalla Code Coverage in quando non viene mai eseguita durante i test (viene usata semplicemente per avviare l'applicazione Java).

Pertanto, iniziamo aggiungendo una classe Java con un metodo `main`, consentendo l'inserimento di alcuni argomenti dalla linea di comando. Per gestire questa funzionalità, utilizzeremo la libreria Java **Picocli**.

Per creare i contesti dei nostri test, avremo ancora bisogno di un accesso diretto al database. Tuttavia, a differenza degli Integration Test (in cui utilizzavamo un'istanza di `StudentMongoRepository`), negli End-to-End Test useremo direttamente un'istanza di `MongoClient` e non menzioneremo mai la classe `Student`.

N.B: gli End-to-End Test devono essere mantenuti separati dagli altri test. Pertanto, dovrebbero essere collocati in una directory distinta (ad esempio, `src/e2e/java`).

N.B: gli End-to-End Test possono essere eseguiti con il plugin Maven Failsafe, poiché rappresentano una categoria specifica di Integration Test. A tal fine, è necessario nominare i file in modo diverso rispetto agli Integration Test (ad esempio, `**/*E2E.java`, `**/E2E*.java` o `**/*E2ECase.java`) per consentire due esecuzioni separate di Failsafe.

N.B: negli End-to-End Test, così come negli Integration Test, non ha senso eseguire il Mutation Testing. Inoltre, negli End-to-End Test non è necessario raggiungere una completa Code Coverage.

Capitolo 14

BEHAVIORAL-DRIVEN DEVELOPMENT

Quando elaboriamo specifiche di alto livello, potrebbe essere preferibile utilizzare un linguaggio di alto livello che risulti accessibile anche a chi non è un programmatore (come clienti e utenti finali). Spesso, la documentazione fornita tramite gli Unit Test risulta insufficiente. Infatti, poiché nella scrittura degli Unit Test utilizziamo un linguaggio di più basso livello (ad esempio Java), essi risultano efficaci per documentare singole classi e metodi. Tuttavia, si concentrano su dettagli specifici, rischiando di far perdere la visione complessiva del quadro generale.

Il **Behavioral-Driven Development (BDD)** è una metodologia progettata per mantenere l'attenzione sull'utente finale durante l'intero sviluppo del progetto. Pertanto, poiché la metodologia BDD si rivolge all'utente finale, le specifiche dovrebbero essere scritte in un linguaggio vicino a quello naturale. A tale scopo, esistono diversi framework che consentono di scrivere specifiche di alto livello utilizzando un semplice **Domain-Specific Language (DSL)**. Questi framework adottano frasi simili all'inglese per esprimere il comportamento ed i risultati attesi. Un esempio di tale framework è **Cucumber**.

Il Behavioral-Driven Development (BDD) costituisce una forma di Test-Driven Development (TDD), poiché le specifiche vengono definite in anticipo. Successivamente, i componenti vengono implementati basandosi su tali specifiche. Tuttavia, a differenza del TDD, dove i test rappresentano specifiche di basso livello, nel BDD i test diventano specifiche di alto livello.

Inizialmente, per soddisfare le specifiche definite, vengono implementate piccole componenti utilizzando la metodologia TDD e gli Unit Test. Queste componenti vengono successivamente integrate tramite gli Integration Test per verificare l'interazione tra di esse. Infine, gli End-to-End Test sono sviluppati per dimostrare che le specifiche di alto livello vengono soddisfatte.

Nel TDD si parte da specifiche di basso livello e si procede verso l'alto (bottom-up), mentre nel BDD si parte da specifiche di alto livello e si procede verso il basso (top-down).

Quando si parte da specifiche di alto livello definite in anticipo (come nel BDD), il ciclo di sviluppo cambia radicalmente. Mentre nel TDD il passaggio dallo stato Red allo stato Green può richiedere pochi minuti, nell'approccio BDD potrebbe impiegare anche alcuni giorni.

N.B: BDD e TDD possono essere utilizzati insieme.

Il processo BDD inizia definendo specifiche di alto livello per gli scenari che vogliamo implementare. Successivamente, si focalizza l'attenzione su uno specifico scenario, procedendo con l'implementazione dei singoli componenti attraverso il TDD e la scrittura degli Unit Test per verificare lo scenario in questione. Quindi, si controlla l'integrazione dei singoli componenti attraverso gli Integration Test.

Una volta che sono stati testati i singoli componenti e la loro integrazione, si procede a verificare l'intero scenario attraverso gli End-to-End Test. Una volta che uno scenario è stato completamente testato, si passa al successivo.

N.B.: il BDD può essere applicato anche agli Unit Test, ma tipicamente viene utilizzato nella descrizione di scenari di livello superiore (come negli End-to-End Test).

Utilizzeremo Cucumber per scrivere specifiche di alto livello da utilizzare come End-to-End Test dopo aver implementato tutti i componenti mediante TDD.

Le specifiche testuali di alto livello utilizzano un semplice Domain-Specific Language (DSL) che impiega frasi simili all'inglese per esprimere il comportamento ed i risultati attesi. Tali specifiche consentono di descrivere scenari seguendo la tipica struttura:

```
1 | Given [some initial context]
2 | When [some event occurs]
3 | Then [something should happen]
```

N.B.: tali specifiche possono essere scritte insieme al cliente (o utente finale), poiché non richiedono la conoscenza di un linguaggio di programmazione (è praticamente inglese).

Nel file Cucumber, con estensione `.feature`, si delineano i test utilizzando un linguaggio descrittivo noto come **Gherkin**. Questo DSL funge sia script per gli Automated Test che come da documento di specifica in tempo reale. Ciascun file `.feature` dovrebbe descrivere una singola caratteristica del sistema o un aspetto specifico di una caratteristica. Questo approccio fornisce una descrizione di alto livello delle funzionalità del software e raggruppa gli scenari correlati.

Uno **scenario** definisce degli esempi di comportamento atteso e può essere considerato equivalente ad un test nel nostro processo di sviluppo standard. Questo scenario è composto da step e può essere suddiviso principalmente in tre parti, utilizzando le parole chiave **Given**, **When** e **Then**. Ulteriori step possono essere aggiunti utilizzando le parole chiave **And** e **But**.

N.B.: a parte queste specifiche keywords, tutto il resto del testo è scritto in inglese semplice.

N.B.: **Given**, **When** e **Then** rappresentano un'alternativa di **Setup**, **Exercise** e **Verify**.



ES:

Consideriamo il seguente esempio di test scritto in Gherkin. Questo scenario definisce il comportamento atteso della View **Student** e presenta specifiche dettagliate riguardo allo stato iniziale della View quando viene visualizzata (ovvero, descrive cosa deve accadere quando l'utente avvia l'applicazione).

N.B.: queste specifiche di alto livello nascondono i dettagli interni dell'applicazione. Tuttavia, sarà necessario mapparle in test JUnit per eseguirle.

La tecnica BDD è comoda per gli utenti, ma richiede uno sforzo aggiuntivo

da parte degli sviluppatori per associare ogni frase ad un test JUnit.

N.B.: poiché queste specifiche rappresentano test End-to-End e vengono scritte prima dell'implementazione dell'applicazione, è probabile che falliscano nella fase iniziale ed avranno successo solo al termine dello sviluppo. Pertanto, il fallimento di questi test non dovrebbe causare la rottura della build nel CI Server o durante la compilazione automatica (altrimenti la build fallirebbe costantemente).

```
1 | Feature: Student View
2 |   Specifications of the behavior of the Student View
3 |
4 |   Scenario: The initial state of the view
5 |     Given The database contains a student with id "1" and name "first student"
6 |     And The database contains a student with id "2" and name "second student"
7 |     When The Student View is shown
8 |     Then The list contains an element with id "1" and name "first student"
9 |     And The list contains an element with id "2" and name "second student"
```

Capitolo 15

CODE QUALITY

Possiamo applicare diversi meccanismi di analisi statica per tenere traccia della qualità del codice Java. Questi meccanismi sono complementari a quelli già eseguiti dal compilatore Java (ad esempio, il controllo statico dei tipi).

La *qualità del codice* (**Code Quality**) misura l'affidabilità e la manutenibilità di un'applicazione durante l'arco di vita del progetto. L'obiettivo principale è evitare che un codice di scarsa qualità diventi costoso da mantenere nel tempo.

SonarQube è una piattaforma open-source progettata per l'ispezione continua della qualità del codice. Questa piattaforma definisce un modello di Code Quality che comprende diverse metriche. Le tre principali misure si concentrano su:

- **Affidabilità (reliability)**: questa misura si occupa di individuare i **bug** nel codice, dove il software potrebbe comportarsi in modo diverso da quanto previsto (cioè, il codice viene compilato ma contiene qualche bug, compromettendo l'affidabilità del software).
- **Sicurezza (security)**: questa misura si concentra sul rilevare le **vulnerabilità** nel codice, dove il software potrebbe essere esposto a violazioni e attacchi.
- **Manutenibilità (maintainability)**: questa misura si occupa di identificare i **code smells**, indicando situazioni in cui il software potrebbe diventare difficile da mantenere nel tempo.

SonarQube offre un'interfaccia web completa con report dettagliati sulle tre misure principali menzionate in precedenza, oltre a fornire informazioni su:

- **Codice duplicato**: indica la presenza di porzioni di codice identiche o simili all'interno del progetto.
- **Codice standard**: valuta il codice rispetto agli standard definiti, evidenziando eventuali deviazioni.
- **Test**: fornisce un'analisi sulla qualità dei test implementati nel progetto.
- **Copertura del codice**: mostra la percentuale del codice sorgente che risulta essere coperta dai test.



Figura 15.1: Interfaccia web di SonarQube in cui vengono sintetizzate le analisi raccolte.

In particolare, SonarQube mantiene un registro storico delle metriche sopra citate, accompagnate da grafici che offrono una rappresentazione chiara dell'evoluzione nel tempo. La piattaforma offre analisi completamente automatizzate e si integra senza problemi con Maven. Per gli sviluppatori che utilizzano Eclipse, è disponibile un plug-in noto come **SonarLint**.

N.B: esiste una versione cloud di SonarQube, chiamata **SonarCloud**, che facilita l'accesso e la gestione delle analisi focalizzate sulla Code Quality.

SonarQube offre anche una stima del tempo necessario per risolvere i problemi individuati. La somma di queste stime costituisce il **debito tecnico** (o **technical debt**). Come in un debito finanziario, il debito tecnico deve essere saldato con il tempo. Pertanto, più questo debito è alto e più sarà difficile ripagarlo.

N.B: pagare il debito tecnico significa affrontare e risolvere i problemi di Code Quality individuati, evitando che questi accumulino interessi nel tempo.

SonarQube consiglia di risolvere tempestivamente i problemi di Code Quality individuati, seguendo il **paradigma della perdita d'acqua** (o **water leak paradigm**). In altre parole, è consigliato risolvere prontamente i problemi, altrimenti potrebbero peggiorare nel tempo (cioè, le perdite d'acqua devono essere risolte molto presto oppure allagheremo). Anche se il debito tecnico non è obbligatorio da saldare, evitare di farlo può portare all'accumulo di ulteriore debito, rendendo più difficile la gestione e la correzione in futuro.

SonarQube è disponibile come file JAR autonomo che può essere scaricato ed eseguito localmente. Tuttavia, richiede un database per conservare la cronologia delle analisi dei progetti. Di default, SonarQube utilizza un database H2 incorporato che non è adatto per un ambiente di produzione. Pertanto, per un'implementazione più robusta è consigliabile utilizzare un database tradizionale. Una soluzione pratica per l'esecuzione locale di SonarQube è l'utilizzo di un'immagine Docker. In particolare, utilizzando un file `docker-compose.yml` è possibile avviare un container SonarQube collegato ad un container PostgreSQL. Una volta attivato, SonarQube è accessibile all'indirizzo `http://localhost:9000`.

Per integrare SonarQube nella build di Maven, è necessario configurare il plug-in SonarQube con il goal `sonar:sonar`. Ecco un esempio di configurazione:

```
1 <build>
2   <pluginManagement>
3     <plugins>
4       <plugin>
5         <groupId>org.sonarsource.scanner.maven</groupId>
6         <artifactId>sonar-maven-plugin</artifactId>
7         <version>3.8.0.2131</version>
8       </plugin>
9     </plugins>
10  </pluginManagement>
11 </build>
```

N.B: durante la compilazione di Maven, le informazioni sulla Code Coverage non vengono eseguite direttamente da SonarQube, ma vengono solo raccolte. Per generare queste informazioni, è necessario configurare il plugin JaCoCo come al solito.

SonarQube valuta il codice assegnando un voto ed un colore basati sull'analisi. La configurazione delle soglie permette di definire quando l'analisi può essere considerata fal-

lita o passata. In caso di problematiche rilevate, è possibile fare click su di esse per visualizzare la porzione di codice interessata e stimare il tempo necessario per risolvere il problema (che contribuisce al debito tecnico).

SonarQube offre anche regole standard per correggere le violazioni, fornendo consigli di correzione. Dopo aver apportato le correzioni, è possibile riavviare l'analisi e monitorare l'evoluzione della qualità del codice nel tempo.

N.B.: alcuni problemi potrebbero essere considerati falsi positivi. In questi casi è possibile configurare alcune proprietà nella build di Maven per escludere una specifica regola oppure uno specifico file Java.

N.B.: il codice principale ed il codice di test sono analizzati con regole differenti, tenendo conto delle specifiche esigenze. Ad esempio, una classe vuota nel codice di test potrebbe essere valutata in modo diverso rispetto a una classe vuota nel codice principale.

SonarCloud rappresenta la versione basata su servizio Cloud di SonarQube. Questo servizio può essere integrato con GitHub Actions. In particolare, SonarCloud ha la capacità di commentare direttamente nelle Pull Request nel caso in cui si verifichi un calo della qualità del codice.